

Computer Graphics Study

컴퓨터 그래픽스 : 컴퓨터를 사용하여 그림을 생성하는 기술

2021/03/05 ~

박지원

목차

*** 본 필기는 이정진 교수님의 강의와 'OpenGL로 배우는 3차원 컴퓨터 그래픽스'를 기반으로 작성되었습니다. ***

- OpenGL 기초
- 1주차 : 컴퓨터그래픽스를 위한 기초 수학 이론
- 2주차 : OpenGL 기초
- 3주차 : OpenGL에 포함된 기초 Method
- 4주차 : Raster Graphics
- 5주차 : Image Processing(영상 처리)
- 6주차 : Image Warping(이미지 변경)
- 7주차 : 중간고사 질문
- 8주차 : 3D Modeling
- 9주차 : Polygonal Mesh
- 10주차_1 : Subdivision Surface
- 10주차_2 : Rendering
- 10주차_3 : Rendering Pipeline 강의 및 기초
- Rendering Pipeline 독학 및 심화
- 11주차 : Ray Tracing
- 12주차 : Lighting and Reflectance
- 13주차 : Rasterization
- 14주차 : 기말고사 질문
- 학기말 프로젝트 : OpenGL

<OpenGL 기초>

1. Library vs API vs SDK

1) Library

= 응용 프로그램 개발을 위해 필요한 기능을 모아 놓은 소프트웨어

ex) C++ STL

2) API (Application Programming Interface)

= API는 프로그램들 끼리(APP 과 APP OR 서비스와 서비스) 사용하는 통신 수단이며 연결다리로 볼 수 있습니다.

- Interface이므로 요청을 한 플랫폼에게 요청만 할 뿐 자세한 내부 로직은 알지 못하는 추상적인 성질을 갖고 있습니다.

- 개발자는 API를 통해 필요한 것을 요청(request)하고 , 요청을 받은 플랫폼은 해당 요청에 대한 Response를 줍니다.

- 동물 정보를 보여주는 Application을 비유로 사용하면 , 동물 정보를 실제로 저장하고 있는 Cloud에게 호랑이에 대한 정보를 물어보는 것이 API 입니다. Cloud는 엄청나게 긴 코드로 구성되어 있지만 API는 기본적으로 Interface이기 때문에 질문을 한 개발자는 Cloud가 어떻게 호랑이 정보를 알려주는 지에 대해 알 필요가 없습니다. 알아서 API를 통해 물어본 request에 대해서 cloud가 response를 해줍니다.

- 간단하게 기억하기 : 개발자가 응용 프로그램에서 어떤 기능이 필요해서 다른 플랫폼에게 method(코드)를 적어 해당 기능을 요청하는 것을 API 라고 합니다.

ex) 날씨 API , Instagram API , OpenGL

ex) 개발자가 어떤 에디터에서 OpenGL API를 이용하여 , OpenGL API에 존재하는 method를 이용해 어떤 플랫폼에 요구를 합니다.

- Reference : <https://www.youtube.com/watch?v=kG-fLp9BTRo>

3) SDK (Software Development Kit)

- = 소프트웨어 개발 도구이며 , 도구 상자 이미지를 연상하면 좋을 것입니다.
- 개발 도구 프로그램(코드), 디버깅 프로그램 , 문서 , API로 구성되어 있습니다.
- 개발자가 실제로 Editor에서 API를 호출하는 코드(Method)입니다.
- 코드이기 때문에 다양한 언어로 된 SDK가 존재합니다.
- ex) java로 작성한 SDK인 JDK

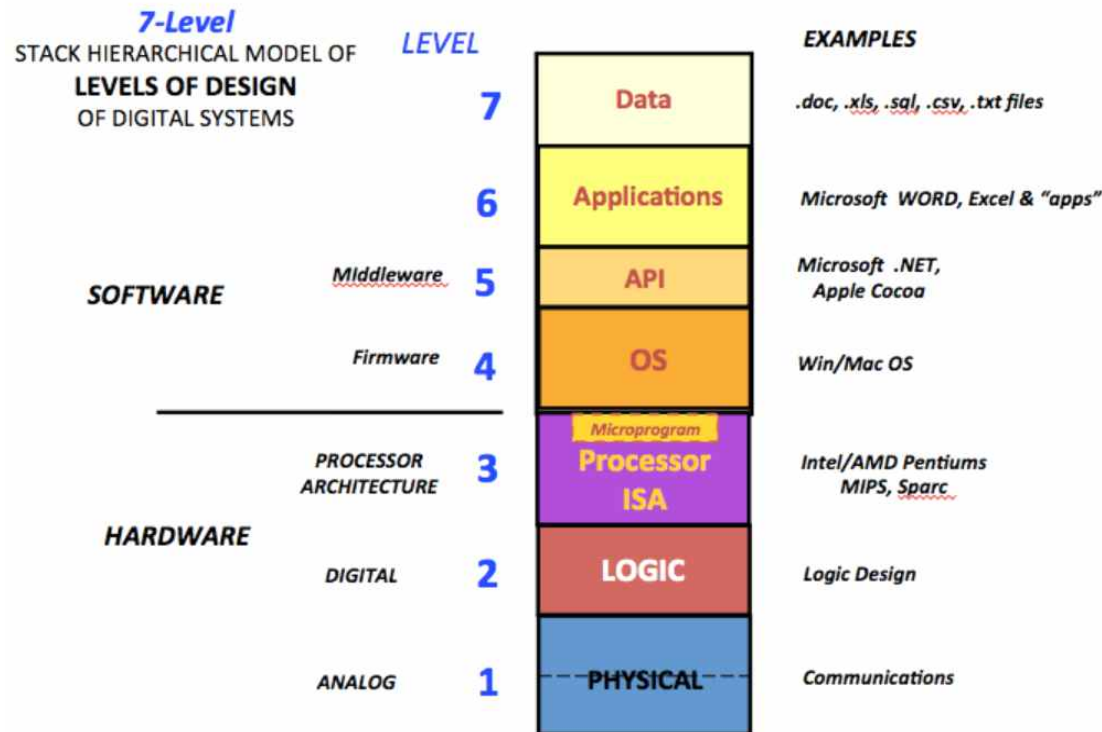
4) API vs Library

라이브러리에서 비유에 사용했던 공구박스를 여기서 다시 사용하려니 참 애매하다. 그래도... 굳~이 비유하자면, 공구박스의 망치를 예로 들겠다. 라이브러리에서는 공구박스 자체를 내가 가지고 있고, 거기서 망치를 꺼내 활용했다. 하지만 API는 다르다. 공구박스가 나에게 없는 것이다. 나에게 공구박스가 없어, 공구박스가 있는 다른 사람에게 나무판자를 주며 부탁을 하는 것이다. 나무판자에 망치를 이용해 못을 박아달라. 그리고 난 못이 박힌 나무판자만 얻는 것이다.

2. OpenGL

= **Application**이 자신의 하부에서 동작하는 장치의 그래픽스 서브시스템에 접근하고 제어하기 위해 사용하는 인터페이스(=API)입니다.

- 프로그램에서 만들어진 명령들이 OpenGL에 전달되고, 하부의 Graphics H/W로 보내집니다.
- OpenGL은 2차원 및 3차원 그래픽스 표준 API 규격으로, 프로그래밍 언어 간 플랫폼 간의 교차 응용 프로그래밍을 지원한다. OpenGL은 약 250여개 가량의 함수 호출을 이용하여 단순한 기하도형에서부터 복잡한 삼차원 장면을 생성할 수 있다.
- OpenGL은 어떤 플랫폼에서도 동작하지만, DirectX는 Windows에서만 동작하는 API입니다.
- OpenGL은 Layer_6인 Application과 Layer_1 ~ 3의 어딘가에 존재하는 Graphics H/W를 연결시켜주는 추상화 Layer인 Layer_5의 API입니다.



<1주차 : 컴퓨터그래픽스를 위한 기초 수학 이론>

벡터공간 V 에서 벡터공간 W 로의 함수 T 가 모든 벡터 $\mathbf{u}, \mathbf{v} \in V$ 와 스칼라 α 에 대하여 다음을 만족할 때 T 를 **선형변환**(또는 **일차변환**, linear transformation)이라 한다.

$$(i) \quad T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$$

$$(ii) \quad T(\alpha \mathbf{u}) = \alpha T(\mathbf{u})$$

- 벡터 공간 : 차원 r^2 r^3

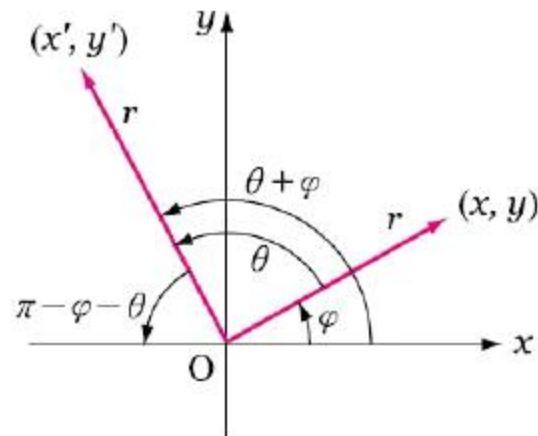
<특수한 선형 변환>

- 영변환 : 변환했을 때 모든 점이 영변환
- 항등변환 : 변환했을 때 모든 점이 자기 자신으로 변환
- 위의 두 변환도 선형변환을 만족하므로 선형변환 만족.

<2차원에서 회전변환 : 매우 중요>

예제

xy 평면 위의 벡터 $\mathbf{v} = (x, y)$ 를 시계반대방향으로 원점을 중심으로 하고 θ 만큼 회전하여 벡터 $\mathbf{v}' = (x', y')$ 이 되었다. $\mathbf{v}$ 는 오른쪽 그림과 같이 x 축 양의 부분과 이룬 각이 φ 이고 $\|\mathbf{v}\| = r$ 이다.



다음 물음에 답하여라.

- (1) x', y' 을 x, y, θ 로 나타내어라.
- (2) (1)의 관계를 행렬로 나타내어라.
- (3) $T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ 가 (2)와 같이 정의될 때 T 는 선형변환인가?

$$\sin(\alpha + \beta) = \sin\alpha\cos\beta + \cos\alpha\sin\beta$$

$$\sin(\alpha - \beta) = \sin\alpha\cos\beta - \cos\alpha\sin\beta$$

$$\cos(\alpha + \beta) = \cos\alpha\cos\beta - \sin\alpha\sin\beta$$

$$\cos(\alpha - \beta) = \cos\alpha\cos\beta + \sin\alpha\sin\beta$$

$$\tan(\alpha + \beta) = \frac{\tan\alpha + \tan\beta}{1 - \tan\alpha \cdot \tan\beta}$$

$$\tan(\alpha - \beta) = \frac{\tan\alpha - \tan\beta}{1 + \tan\alpha \cdot \tan\beta}$$

(1) : 위의 삼각함수 공식을 이해하면 풀수있음

(2) : R2에서 이동하는 선형 변환은 임의의 벡터에 행렬을 곱해서 새로운 벡터를 만드는 것과 동일

(3) : 선형변환의 정의로 증명

모든 변환이 선형변환이 되는 것은 아니다. 변환 $T : \mathbb{R} \rightarrow \mathbb{R}$ 을

$$T(x) = ax + b \quad (b \neq 0)$$

으로 하면 T 는 선형변환이 아니다. 왜냐하면

$$T(x_1 + x_2) = a(x_1 + x_2) + b = ax_1 + ax_2 + b$$

$$T(x_1) + T(x_2) = ax_1 + b + ax_2 + b = ax_1 + ax_2 + 2b$$

이므로

$$T(x_1 + x_2) \neq T(x_1) + T(x_2)$$

이기 때문이다.

<아주 중요한 정리>

정리

$T : V \rightarrow W$ 가 선형변환이면 $\mathbf{v}_i \in V$, 스칼라 $\alpha_i (i=1, 2, \dots, n)$ 에 대하여

$$T(\alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \dots + \alpha_n \mathbf{v}_n) = \alpha_1 T(\mathbf{v}_1) + \alpha_2 T(\mathbf{v}_2) + \dots + \alpha_n T(\mathbf{v}_n)$$

<2주차 : OpenGL 기초>

*** 2주차는 간단히 OpenGL을 소개해주시고 , OpenGL의 몇 가지 reference를 이용해 보았습니다. OpenGL reference는 다른 언어처럼 암기하지 않고 필요할 때 마다 웹 사이트에 들어가서 검색합니다. ***

1. visual studio 2019 community에서 OpenGL 설치하기
 - API이므로 해당 파일을 다운로드 받아서 사용해야 합니다.
 - 교수님은 visual studio 2015에서 하셨습니다.
 - 아래의 영상을 그대로 따라서 진행하면 됩니다.

<https://www.youtube.com/watch?v=XOtY4yzitdk>

2. OpenGL 개요

- OpenGL은 개발자가 응용 프로그램에서 어떤 기능이 필요해서 다른 플랫폼에게 method(코드)를 적어 해당 기능을 요청하는 API입니다.
- 3주차에 API에 포함된 다양한 method를 배웁니다.
- DirectX와 비교했을 때 매우 간단하게 C++ 코드로 작성할 수 있습니다. DirectX는 매우 복잡합니다.
- OpenGL은 학습용 , DirectX는 기업의 프로젝트용 이라고 표현하셨습니다.
- 컴파일을 하면 그래픽을 나타내는 프로그램이 함께 출력됩니다. 헤더를 반드시 아래의 순서대로 작성합니다.

```
#include <windows.h>
#include <glut.h>
```

- 기본 Header
- <glut.h>는 특수하게 앞에 gl/을 붙이지 않아도 동작합니다.

```
#include <gl/glut.h>
#include <gl/glu.h>
#include <gl/gl.h>
```

3. OpenGL Reference & GLUT API

1) OpenGL

<https://www.khronos.org/registry/OpenGL-Refpages/gl4/>

2) GLUT(OpenGL Utility Toolkit library)

<http://freeglut.sourceforge.net/docs/api.php#GeometricObject>

4. 중요 method

`glPointSize(GLfloat size)`

- `glGetFloatv(GL_POINT_SIZE_RANGE)`

- Returns the range of the point size that the hardware supports

`glLineWidth(GLfloat width)`

- `glGetFloatv(GL_LINE_WIDTH_RANGE)`

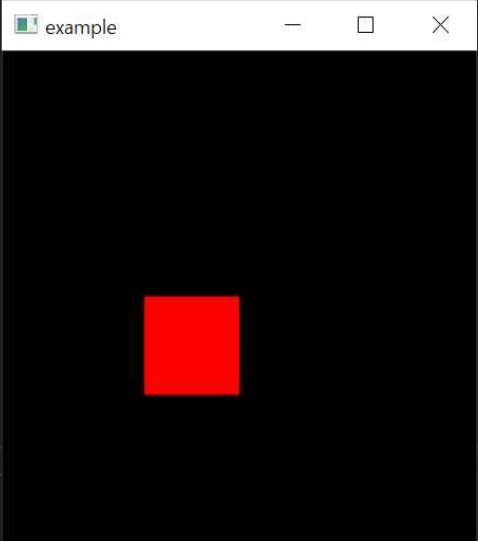
- Returns the range of the line width that the hardware supports

- `glPointSize` : 점의 크기
- `glLineWidth` : 선의 두께

5. 예제 코드

```
Main.cpp  X
StudyOpenGL  (전역 범위)  main(int argc, char ** argv)

1  #include <windows.h>
2  #include <glut.h>
3
4  void Reshape(int w, int h)
5  {
6      glLoadIdentity();
7      glViewport(0, 0, w, h);
8      gluOrtho2D(0, 100, 0, 100); //프로그램 창의 크기
9  }
10
11 void Display(void)
12 {
13     glClear(GL_COLOR_BUFFER_BIT); //화면 지우기
14     glColor3f(1.0, 0.0, 0.0); //RGB
15     glRectf(30.0, 30.0, 50.0, 50.0); //사각형의 좌표 2개(좌하단 좌표의 x,y 와 우상단 좌표의 x,y)
16     glutSwapBuffers();
17 }
18
19 void main(int argc, char** argv)
20 {
21     glutInit(&argc, argv);
22     glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
23     glutCreateWindow("example"); //프로그램 이름
24     glutReshapeFunc(Reshape); //설정한 프로그램창의 크기 호출
25     glutDisplayFunc(Display); //유저 데이터(화면에 그릴 그림) 호출
26     glutMainLoop();
27 }
```

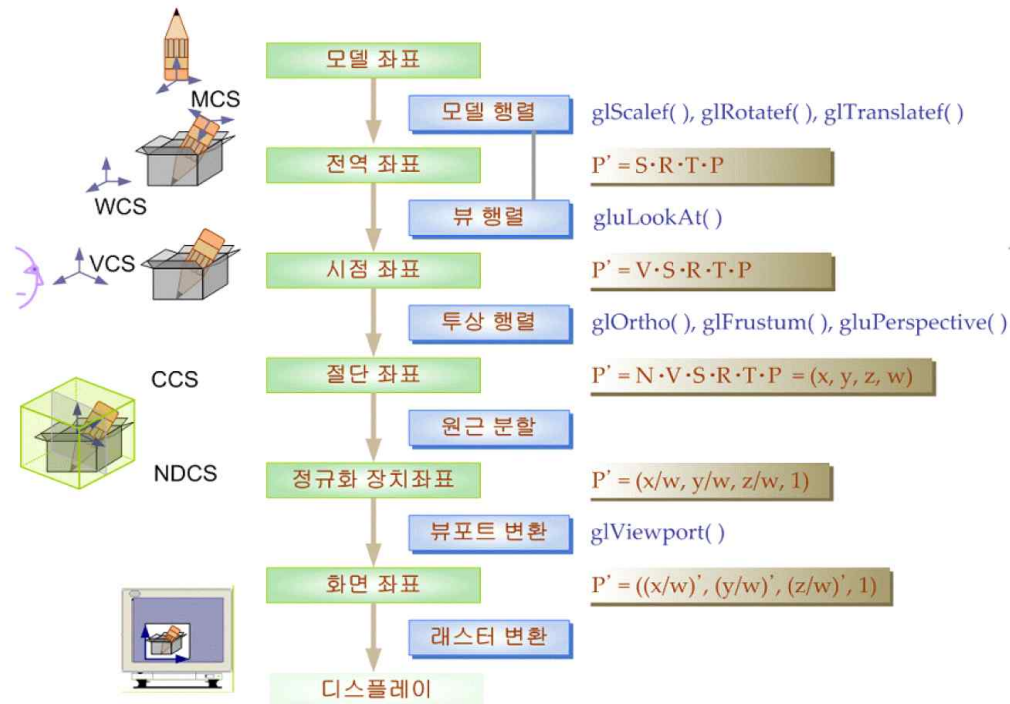


<3주차 : OpenGL에 포함된 기초 Method>

*** OpenGL(API)에 포함된 기초 Method 들을 배웁니다. 이 들은 행렬과 변환과 연관이 깊은 method입니다. ***

<컴퓨터 그래픽스에서 매우 중요한 내용>

GL Pipeline



<예제 프로그램 : 3주차_1>

*** 주석에 적힌 해당 코드의 설명을 자세히 읽읍시다. ***

```
#include <gl/glut.h>
#include <gl/glu.h>
#include <gl/gl.h>

using namespace std;

void Display()
{
    glClear(GL_COLOR_BUFFER_BIT); //화면 지우기
    glViewport(0, 0, 300, 300); //윈도우의(0,0) 좌표부터 (300,300)의 크기만큼 디스플레이를 사용합니다.
    glMatrixMode(GL_MODELVIEW); //사용할 행렬의 모드를 MODELVIEW로 합니다.

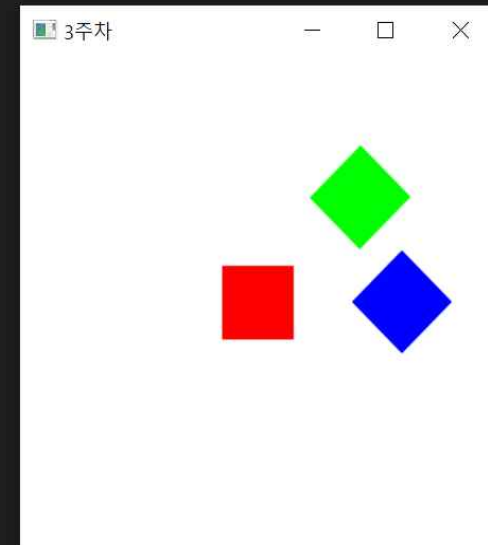
    //Red
    glColor3f(1, 0, 0); //float type이지만 int형을 적어도 알아서 형변환 됩니다. //색상 조절
    glLoadIdentity();
    glutSolidCube(0.3); //0.3의 길이를 갖는 정사각형

    //Green : 행렬의 성질로 인해 이동을 먼저하고 회전을 합니다.
    glColor3f(0, 1, 0);
    glLoadIdentity();
    glRotatef(45.0, 0.0, 0.0, 1.0); //z축 기준 45도 회전
    glTranslatef(0.6, 0.0, 0.0); //x축에서 0.6만큼 이동
    glutSolidCube(0.3);

    //Blue : 행렬의 성질로 인해 회전을 먼저하고 이동을 합니다.
    glColor3f(0, 0, 1);
    glLoadIdentity();
    glTranslatef(0.6, 0.0, 0.0);
    glRotatef(45.0, 0.0, 0.0, 1.0);
    glutSolidCube(0.3);
    glFlush();
}
```

```
int main(int argc , char** argv)
{
    glutInitDisplayMode(GLUT_RGBA); //기본 디스플레이 모드
    glutInitWindowSize(300, 300); //전체 windows(여기서는 3주차_1프로그램을 의미)의 크기
    glutCreateWindow("3주차");
    glClearColor(1.0, 1.0, 1.0, 1.0); //화면을 하얀색으로 초기화
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
    glutDisplayFunc(Display);
    glutMainLoop();

    return 0;
}
```



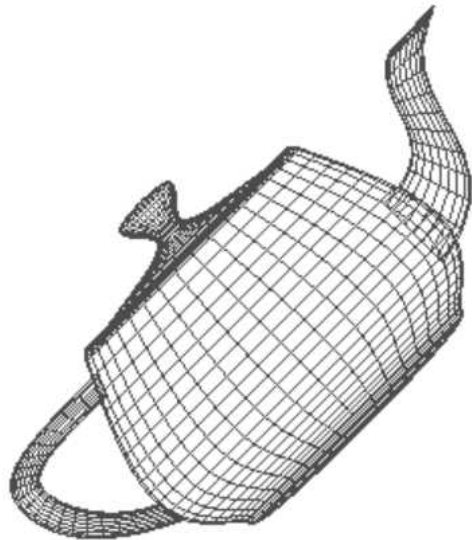
<예제 프로그램 : 3주차_2>

```
/*
그림을 보면 주전자가 회전해 있습니다.
보통 객체(주전자)를 회전(glRotatef)시킵니다.
하지만 이 코드에서는 객체를 촬영하는 카메라를 회전시켰습니다.
카메라를 회전 시켜도 객체가 회전이 됩니다.
*/
void Display()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glMatrixMode(GL_MODELVIEW); //GL_MODELVIEW가 가장 많이 사용됩니다. GL_PROJECTION | GL_TEXTURE는 빈도가 낮습니다.
    glEnable(GL_DEPTH_TEST);
    glLoadIdentity();
    glColor3f(0.3, 0.3, 0.3);
    gluLookAt(0.0, 0.0, 0.0, 0.0, 0.0, -1.0, 1.0, 1.0, 0.0); //주전자의 북동쪽 방향을 위로 나타나게 활용.
    glutWireTeapot(0.5);
    glFlush();
}

int main(int argc , char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGBA | GLUT_DEPTH);
    glutInitWindowSize(400, 400);
    glutInitWindowPosition(0, 0);
    glutCreateWindow("3주차_2 : gluLookAt");
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glutDisplayFunc(Display);
    glutMainLoop();

    return 0;
}
```


3주차_2 : gluLookAt



*** 지금부터 Function을 설명할 때 매개변수를 포함하지 않고 설명한 것이 대다수 존재하며 설명 또한 매우 간략합니다. Function의 이름만 명시해 놓았으므로 직접 공식문서에서 찾아보거나 교수님 PPT를 참조 합시다. ***

1. Model Transformation Method

- 모델의 변환을 위해 행렬을 사용합니다.
- OpenGL자체가 행렬 관련 함수가 많습니다.
- 일단 단위행렬로 초기화
- glMatrixMode = 어떤 행렬을 사용할지 설정하는 method
- glLoadIdentity() = 행렬을 단위행렬(I)로 초기화하는 method
- glMultMatrixf(const GLfloat *M) : 행렬을 곱하는 method
- *M은 4x4 행렬입니다.
- glRotatef(angle , x , y , z) : 각도에 따라 객체를 회전시킵니다.
- glTranslatef(dx , dy , dz) : 객체를 이동시킵니다.

2. Multiple Transformation Method

- 일반적으로 여러 개의 변환을 객체에 적용시킵니다.
- 코드의 순서와 실제 Multiple Transformation의 순서는 다릅니다.

- glMatrixMode(GL_MODELVIEW);
- glLoadIdentity();
- glRotatef(45, 0.0, 0.0, 1.0); 물체 변환의 역순
- glTranslatef(10.0, 0.0, 0.0);
- glVertex3f(Px, Py, Pz);

■ $P' = T \bullet P$

■ $P'' = R \bullet P' = R \bullet T \bullet P$

- 코드의 순서로 따지면 glRotatef가 glTranslatef보다 먼저 실행되어야 합니다.
- 하지만 행렬의 성질로 인해 glTranslatef를 먼저 하고 , glRotatef를 나중에 합니다.

3. Viewing Method

- view direction : 카메라가 바라보는 방향
- field of view : 카메라의 각도
- up vector : 카메라의 가로와 수직의 위 방향
- **gluLookAt(매개변수 9개)** : Vector3 3개($3 \times 3 = 9$) (카메라 위치 , 카메라가 바라보는 방향 , 카메라 up-vector의 좌표)

gluLookAt()

: view transform matrix를 정의

- 시점 좌표계를 정의하는 함수로, (eyeX, eyeY, eyeZ)에 있는 카메라가 (centerX, centerY, centerZ)에 있는 초점을 바라보는 것이다.

- (upX, upY, upZ)는 카메라 상향 벡터이다.

```
1 void gluLookAt(  
2     GLdouble eyeX,  
3     GLdouble eyeY,  
4     GLdouble eyeZ,  
5     GLdouble centerX,  
6     GLdouble centerY,  
7     GLdouble centerZ,  
8     GLdouble upX,  
9     GLdouble upY,  
10    GLdouble upZ  
11 );  
12
```

- gluLookAt()으로 회전하는 것이 헛갈리므로 잘 사용하지 않습니다.

4. View Volume

= 얼마만큼을 그릴 것 인지 결정하는 method

- glOrtho() : 원근감이 존재하지 않기 때문에 멀리 있는지 가까이 있는지 똑같이 보입니다.
- glFrustum OR gluPerspective : 원근감이 존재하는 사람이 보는 방식이기 때문에 멀리 있는 물체가 작게 보입니다.
- default가 원근감이 없기 때문에 glFrustum OR gluPerspective를 사용해서 원근감을 구현해야 합니다.

5. glViewport()와 glOrtho()

```
glClear(GL_COLOR_BUFFER_BIT);  
glViewport(50, 50, 150, 150);  
glColor3f(1.0, 0.0, 0.0);  
glBegin(GL_POLYGON);  
    glVertex3f(0.5f, -0.5f, 0.0);  
    glVertex3f(0.5f, 0.5f, 0.0);  
    glVertex3f(-0.5f, 0.5f, 0.0);  
    glVertex3f(-0.5f, -0.5f, 0.0);
```

```
glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
```

- 윈도우 창에서 (50,50)을 좌측 하단으로 시작해서 (150,150)만큼을 도화지로 사용하겠습니다. 그러므로 도화지의 우측 상단은 (200,200)가 됩니다.

- glVertex3f를 이용해서 점을 4개 찍습니다. glOrtho를 이해해야 해당 코드를 이해할 수 있습니다.

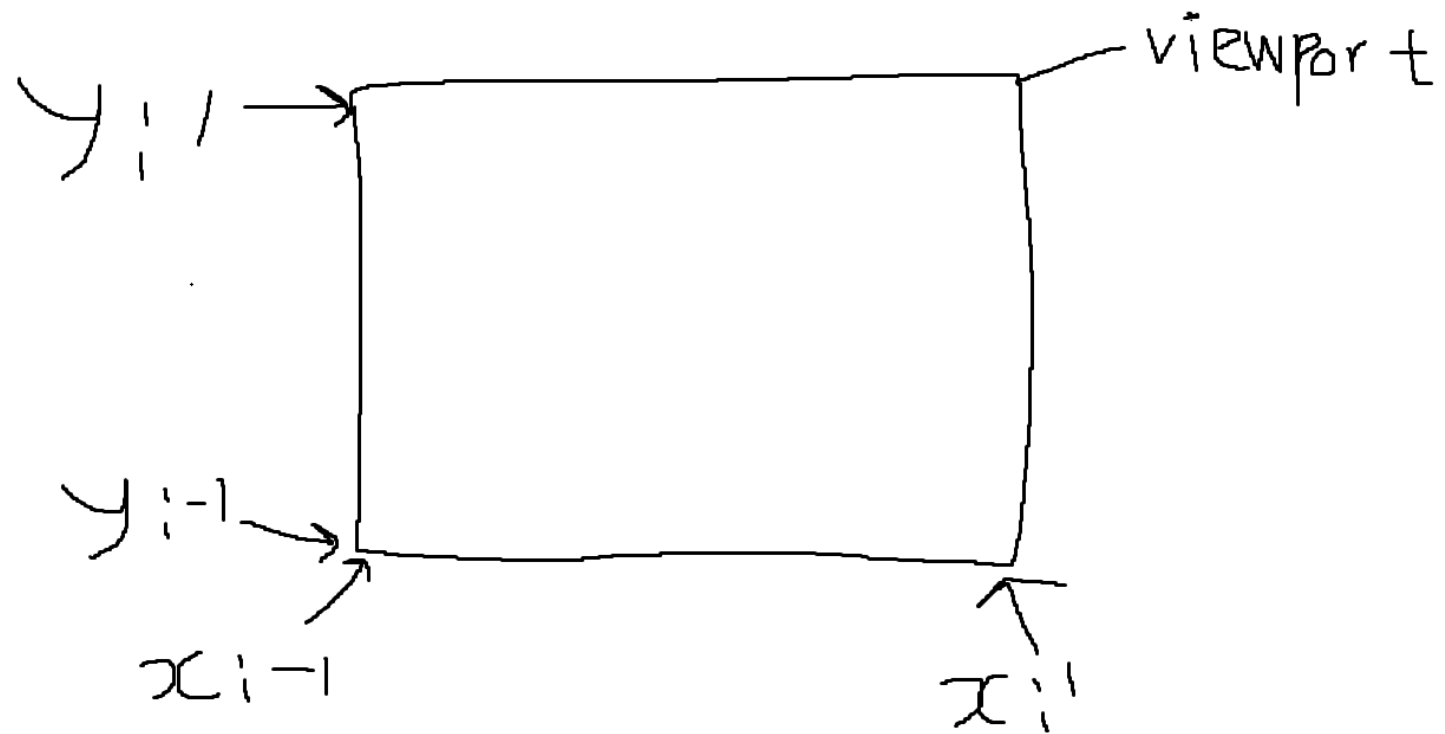
- glOrtho와 glVertex3f를 이용하면 물체를 확대 축소 시킬 수 있습니다.

- glOrtho(x좌표 최솟값, x좌표 최댓값, y좌표 최솟값, y좌표 최댓값, z좌표 최솟값, z좌표 최댓값)입니다.

- z좌표는 아직 2D라 무시해도 괜찮아 보입니다.

- 상대적 값으로 도화지를 표현한다고 생각해도 될 것 같습니다.

<glOrtho 그림으로 표현>



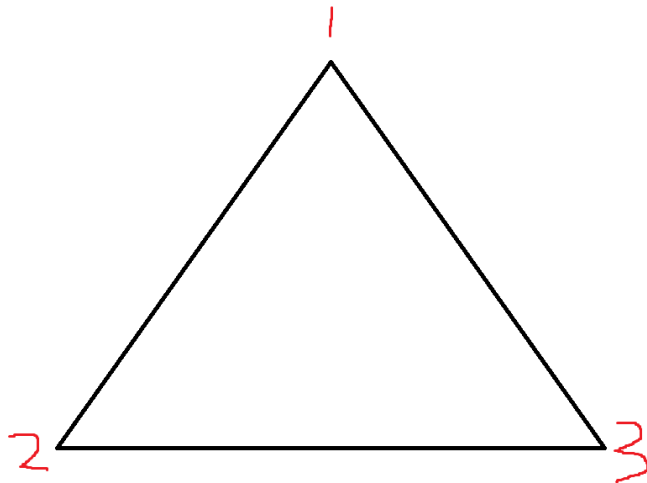
6. Callback Function

= 어떤 일이 시행되면 자동으로 호출되는 함수입니다.

- glutReshapeFunc()
- glutMouseFunc() : 마우스의 입력에 반응하는 함수
- glutMotionFunc() : 마우스 드래그에 반응하는 함수
- glutPassiveMotionFunc() : 마우스의 움직임에 반응하는 함수
- glutIdleFunc() : vertex를 갱신하기 때문에 animation 효과를 나타내는 함수
- glutKeyboardFunc() : 키보드 또는 마우스의 입력에 반응하는 함수

ex) esc 누르면 프로그램 종료

7. Front Face Detection



- 삼각형을 점 3개로 나타내면
- triangle(1번 인자, 2번 인자, 3번 인자)입니다.
- 이렇게 반시계로 삼각형의 점을 읽는 것을 FrontFace라고 합니다.
- 컴퓨터 그래픽스에서 정방향을 반시계 방향입니다.

<마우스로 OpenGL에서 사각형 그리기>

- 주의사항 : windows 사이즈(400,400)에서 화면에 마우스로 그리면 좌표가 (200,100) 같이 찍히게 됩니다.
- 하지만 glOrtho()를 이용해서 x값은 0 ~ 1이고 , y값은 0 ~ 1로 설정했기 때문에 값의 차이가 생깁니다. 그러므로 점을 찍을 때 400을 나누어서 좌표를 설정합니다.

```
#include <glut.h>
```

```
GLint topLeftX, topLeftY, bottomRightX, bottomRightY;
```

```
void myDisplay()  
{  
    glViewport(0, 0, 400, 400);  
    glClear(GL_COLOR_BUFFER_BIT);  
    glColor3f(1, 0, 0);  
    glBegin(GL_POLYGON);  
        glVertex3f( , , 0);  
        glVertex3f( , , 0);  
        glVertex3f( , , 0);  
        glVertex3f( , , 0);  
    glEnd();  
    glFlush();  
}
```

```
#include <glut.h>
```

```
GLint topLeftX, topLeftY, bottomRightX, bottomRightY;
```

```
void myDisplay()  
{  
    glViewport(0, 0, 400, 400);  
    glClear(GL_COLOR_BUFFER_BIT);  
    glColor3f(1, 0, 0);  
    glBegin(GL_POLYGON);  
        glVertex3f(topLeftX/400.0, (400-topLeftY)/400.0, 0);  
        glVertex3f(topLeftX/400.0, (400-bottomRightY)/400.0, 0);  
        glVertex3f(bottomRightX/400.0, (400-bottomRightY)/400.0, 0);  
        glVertex3f(bottomRightX/400.0, (400-topLeftY)/400.0, 0);  
    glEnd();  
    glFlush();  
}
```

<4주차 : Raster Graphics>

1. Pixel & 해상도

1) Pixel (=화소)

= 컴퓨터 image를 구성하고 있는 최소 단위

= Picture Element에서 탄생한 단어

- 1 pixel은 약 0.26mm

- pixel은 다양한 polygon이나 원으로도 만들 수 있으나 효율성을 위해 정사각형이 pixel의 표준이 되었습니다.

2) 해상도와 관련된 단위

① DPI



Image printed at 300 DPI



Image printed at 72 DPI

왼쪽은 300DPI로 인쇄한 사진이고, 오른쪽은 72DPI로 인쇄한 사진이다. 전자가 밀도가 높기에 더 선명하다.

= Dots Per Inch

- 주로 인쇄 출력물에 사용하는 단위입니다.

- DPI가 높을수록 해상도가 좋습니다.

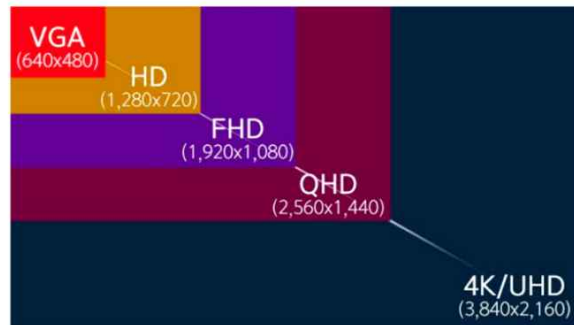
- 1 inch = 2.54cm
- 300DPI : 1 inch에 1/300 크기의 점이 300개 들어간다는 의미
- 72DPI : 1 inch에 1/72 크기의 점이 300개 들어간다는 의미

② PPI

= Pixels Per Inch

- TV , 모니터 , 스마트폰 같은 기계에서 사용하는 단위입니다.
- PPI가 높을수록 해상도가 좋습니다.

3) 해상도



= 화면의 화질을 의미합니다.

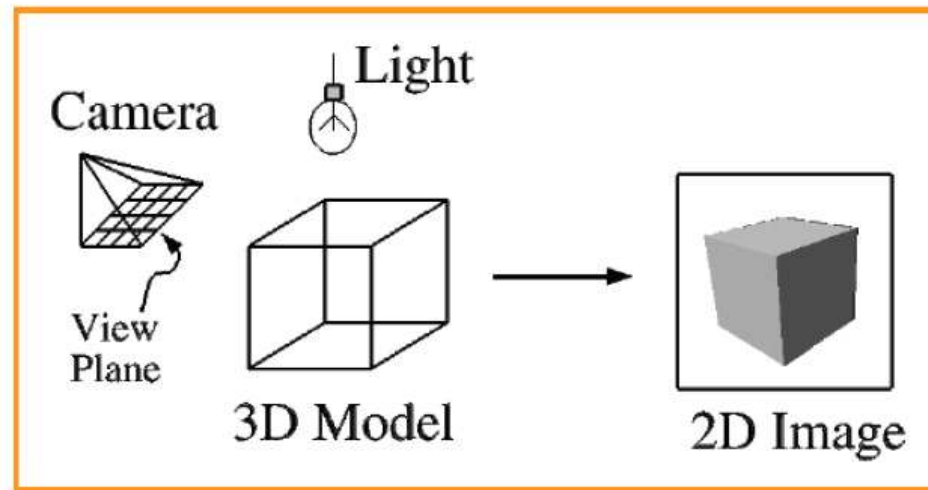
- 100 DPI & 가로 16인치 & 세로 9인치인 이미지 파일을 200 DPI 로 바꾸면 가로 8인치 & 세로 4.5인치로 작아집니다. 이 이미지 파일은 가로 1600 pixels & 세로 900pixels로서 총 1,440,000pixels로 이루어져 있습니다. 흔히 쓰는 표현으로는 1600 × 900 해상도입니다.
- 3840x2160이 1600x900 보다 해상도가 좋은 것은 절대 아닙니다. 먼저 영화관의 초대형 스크린은 스마트폰에 비해 수 억 배는 크다고 가정합니다. 영화관의 초대형 스크린에 3840x2160화면을 띄우면 스크린 전체에 3840x2160개의 pixel을 채우기 때문에 스크린의 1인치에 들어가는 pixel의 개수는 적을 것입니다. 하지만 스마트폰에서 1600x900화면을 띄우면 작은 화면에 1600x900개의 pixel을 채우기 때문에 스마트폰의 1인치에 들어가는 pixel의 개수는 많을 것입니다. 그러므로 해상도(=화면의 화질)는 스마트폰의 1600x900 해상도가 훨씬 선명합니다.
- Youtube의 720P는 1280x720(HD)이며 , 1080P는 1920x1080(FHD)입니다.
- 하드웨어의 크기는 변하지 않으므로 , 화면의 해상도를 낮추면 동일한 크기의 화면에 들어가는 절대적인 pixel의 개수가 적어지기 때문에 화질이 낮아지고 , 심하면 pixel의 정사각형 윤곽이 보이게 됩니다.

2. Image

= 2차원 공간에서의 pixel 집합(2차원 배열)

- Image는 pixel의 집합이므로 크게 보면 연속적(continuous)으로 깔끔하게 보이지만 확대를 하면 pixel들이 보입니다.
- 사진을 찍는 세상은 빛이 존재하는 3D이고 , 이걸 카메라로 촬영했을 때 2D로 표현하는 방법에 대해서 추후에 배웁니다.

<Computer Graphics에서 매우 중요한 그림>



<Image를 컴퓨터에서 표현하는 H/W>

- 영상 위주로 강의하셨습니다.

3. Color

1) 색의 기초

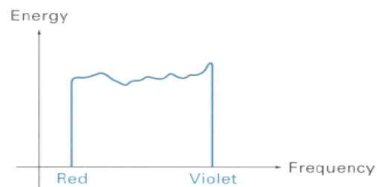
① 명도 : 색의 밝은 정도



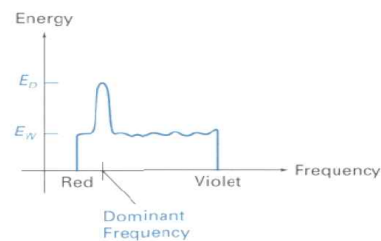
② 채도 : 색의 탁한 정도이며 , 원색(물감 12색)에 가까울수록 채도가 높습니다.



2) Visible Light를 나타내는 HSL Model



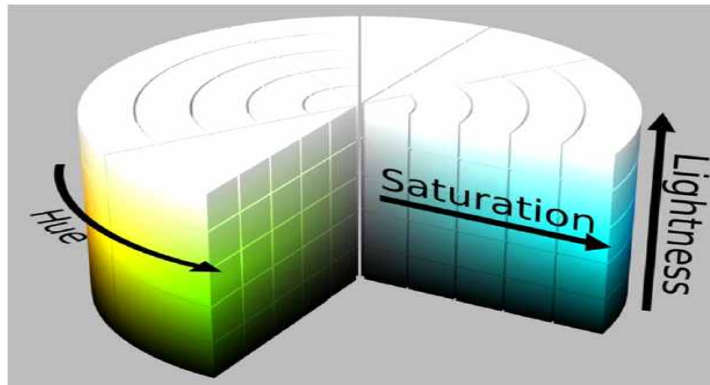
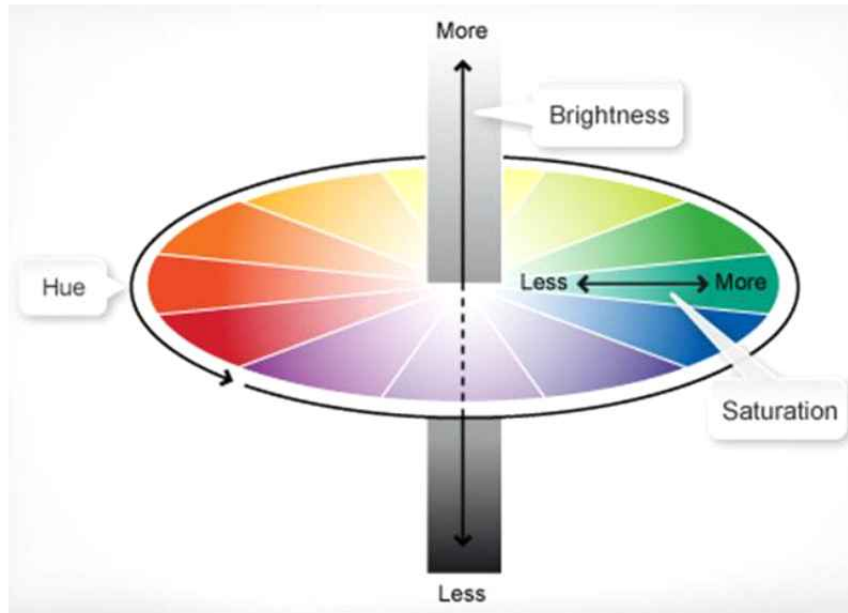
White Light



Orange Light

- 사람이 볼 수 있는 빛의 스펙트럼을 진동수로 나타내었습니다.
- 진동수(Frequency , HZ단위)와 파장(람다)은 반비례 관계입니다.
- 작은 진동수이며 큰 파장을 갖는 색은 Red입니다.
- 큰 진동수이며 작은 파장을 갖는 색은 Violet입니다.

<HSL Model>



① H : Hue

= 색상

ex) Orange : 오렌지 부분의 energy가 확 올라갑니다.

② S : Saturation

= 채도

③ L : Lightness (=Brightness)

= 명도

= 그래프의 면적

3) Color Model

= 각각의 모델의 멤버 변수들의 값을 섞어서 색을 표현합니다.

① RGB : 기본 삼원색 (Red , Green , Blue)

② XYZ : 인간은 파장마다 다르게 받아들이는 원추세포가 존재합니다. 이를 기반으로 만들어진 Color Model입니다.

③ CMY : 고화질 인쇄에서 사용하는 삼원색 (Cyan , Magenta , Yellow)

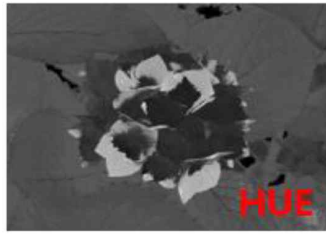
④ HSV : 실제 컬러 영상에서 처리할 때 유리한 색 모델

- H(Hue) : 색상 , 0에서 360사이의 값을 가집니다.

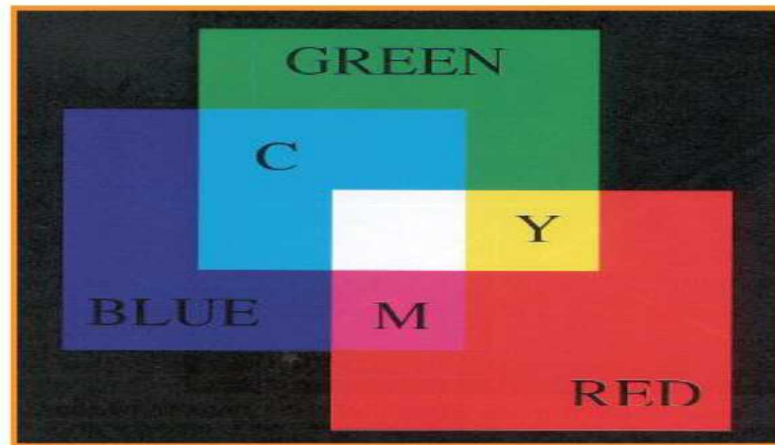
- S(Saturation) : 채도 , 0에서 100사이의 값을 가지며 0에 가까울수록 무채색에 가깝습니다.

- V(Value) : 명도 , 0에서 100사이의 값을 가지며 0은 검정이고 100은 흰색입니다.

<HSV의 요소 3가지를 각각 변경했을 때>



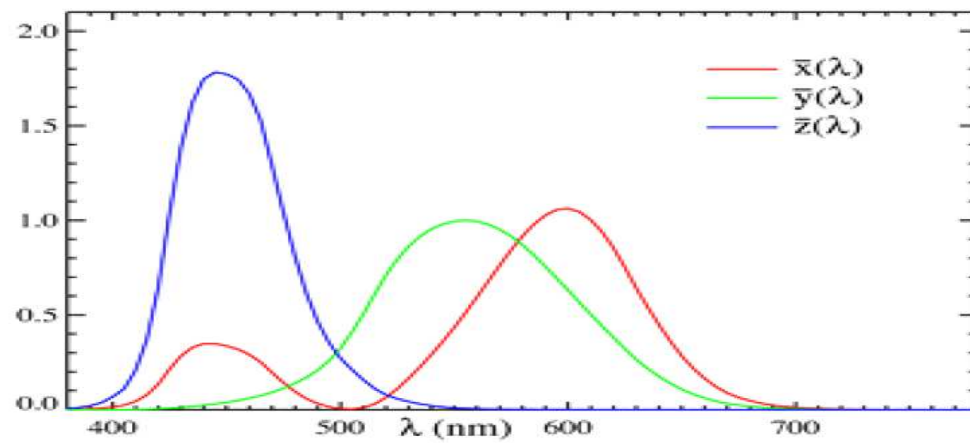
4) Color Model Example



RGB



CMY



XYZ

Red color (R)

7

Green color (G)

255

Blue color (B)

30

Convert Reset Swap

Hex color code

#07FF1E

RGB color code

rgb(7,255,30)

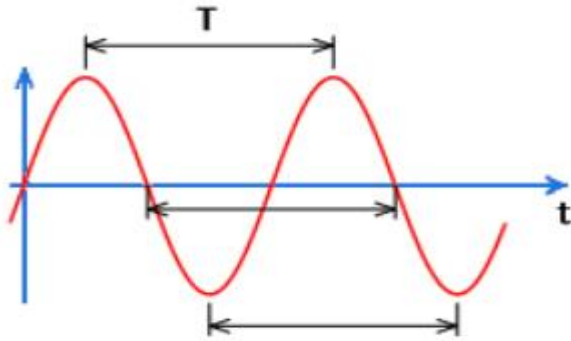
RGB를 섞어서 만든
상수 Hex 값

<5주차 : Image Processing(영상 처리)>

*** 이미지를 다양하게 변화시키는 기법에 대해서 학습 합니다. ***

1. Signal And Frequency (=신호와 진동수)

1) 신호

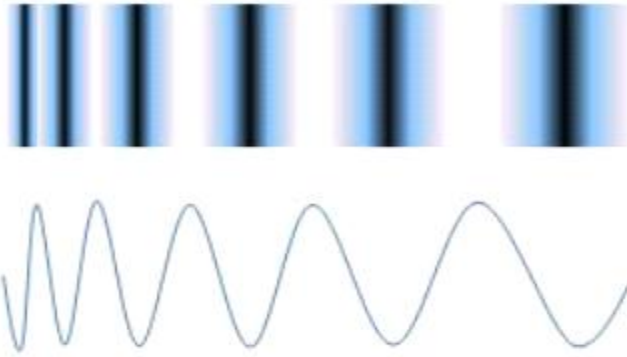


- 물체의 운동을 그래프로 표현한 것을 신호라고 하는 것 같습니다.
- Image 영역에서의 Signal은 pixel **밝기**의 변화율을 표현하는 그래프입니다.

2) 진동수

- = 1초 동안 동일한 신호 모양이 몇 번 반복하는지에 대한 수치
- = 엄밀하게 따지면 1초 동안 반복하는 진동수
- Frequency 이며 , f로 표기하고 , 단위는 Hz 입니다.
- 파동의 개념에서는 주파수라고 부릅니다.
- 1초에 그래프에서 4번 같은 파장이 반복되면 4Hz입니다.
- 위의 그림에서는 몇 초인지는 알 수 없지만 , 파장이 2번 반복되었습니다.

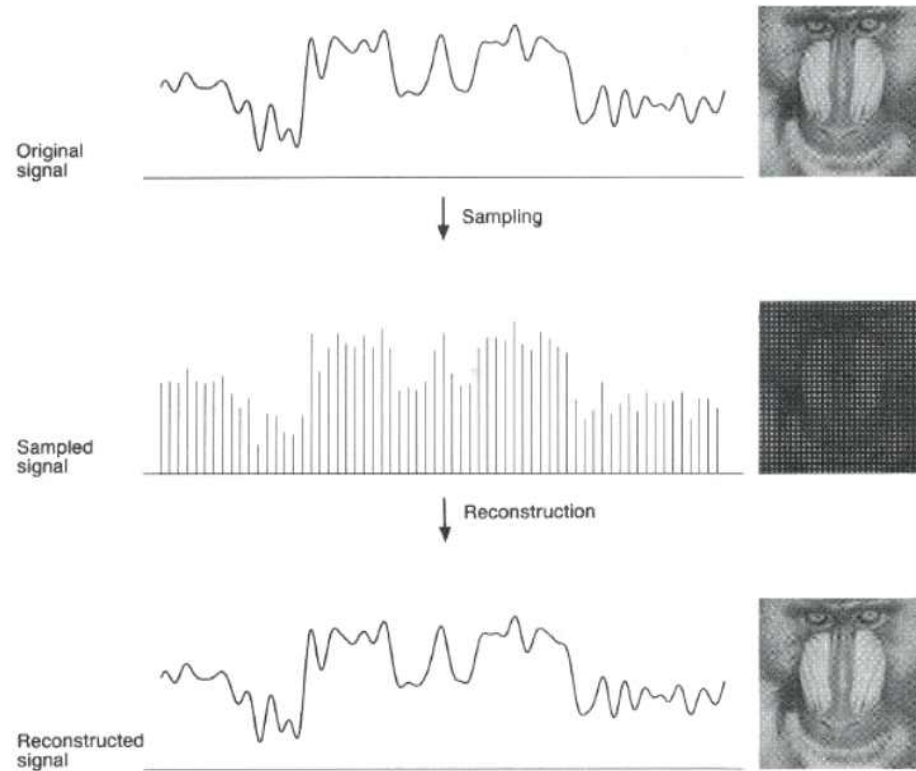
<Image에서 Frequency(진동수)>



- Image 영역에서 pixel 밝기의 변화율을 진동수라고 표현합니다.
- 밝기가 빨리 변화하는 영역을 고주파 영역 , 밝기가 거의 변하지 않는 영역을 저주파 영역으로 나타냅니다.

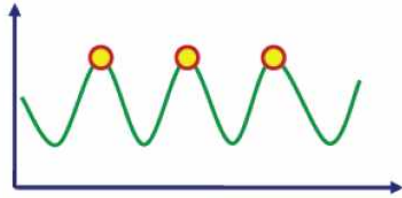
2. Image

- Image는 실제로는 **discrete** 하지만 continuous하게 보일 뿐입니다. 그러므로 Continuous한 실제 객체를 discrete 하게 변경할 때 Image에서는 손실이 발생하고 이를 다시 복원하는 reconstruct를 합니다.



- continuous한 것은 자연계의 물체이고 그걸 sampling을 해서 discrete 한 image로 만들어 줍니다.
- Sampling : continuous $\rightarrow$ discrete
- Reconstruction : continuous $\rightarrow$ discrete
- 그래프의 x값은 x축에서 위치, y값은 밝기
- 얼마나 많은 sample을 만들어야 오리지널과 비슷하게 복원이 될까

3. Sampling Theory : 얼마나 많은 양의 sample을 만들어야 데이터의 손실 없이 Reconstruction이 가능할까?

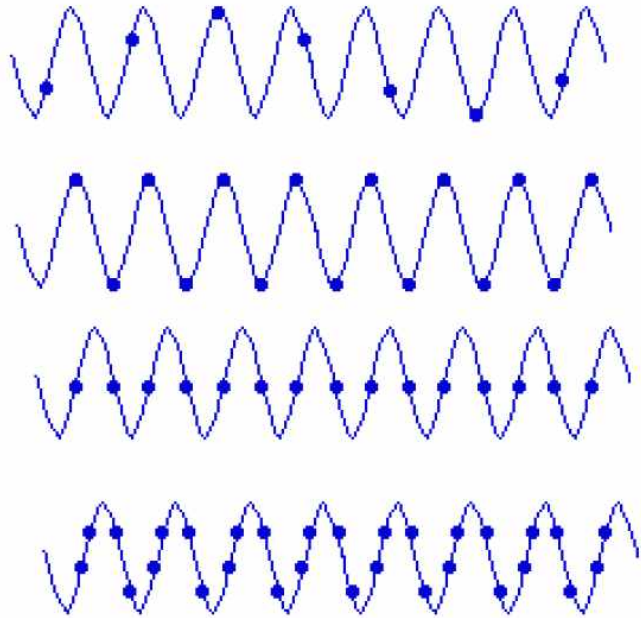


- Sample은 그림에서 노란 동그라미입니다.
- 그림에서 원본 Signal(신호)은 곡선이지만 3개의 Sample로 이루어진 signal은 동등한 y값을 가지므로 직선 형태로 그려지게 됩니다.
- Sample을 조금 갖는 Image에서는 Aliasing(계단 현상)이 발생합니다.
- 당연하지만 Sample이 많을수록 점점 정확해집니다.
- 모든 신호는 주기 함수의 합으로 표현할 수 있습니다. 신호에 대해서 다양한 진동수로 나눈 다음에 모두 더하면 원본의 신호와 같아집니다.

<질문에 대한 Shannon의 답>

A signal can be reconstructed from its samples,
if the original signal has no frequencies
above $1/2$ the sampling frequency - Shannon

- Original Signal의 진동수가 Sampling Signal의 개수의 $1/2$ 를 초과하지 않는다면 해당 Sample Signal을 이용해서 Original Signal로 복원할 수 있습니다.



$$f_s < 2f$$

$$f_s = 2f$$

$$f_s = 2f$$

$$f_s > 2f$$

- Sample의 개수는 점의 개수이며 , fs로 표기합니다.
- 진동수는 f로 표기합니다.
- Sample을 진동수의 2배보다 많이 찍어야 복원이 가능합니다.

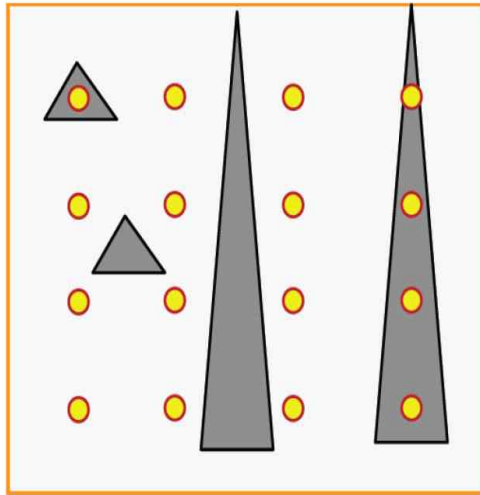
<2배와 같은 경우>

- 1) 그림_1 : Sample이 부족한 경우를 under-sampled 라고 부릅니다. 이런 경우 Aliasing이 발생합니다. 우리는 Aliasing이 발생하지 않기 위해서 높은 진동수만큼 Sample을 만들어야 합니다.
- 2) 그림_2 : 올바르게 Original Signal을 복원 합니다.
- 3) 그림_3 : 올바르게 Original Signal을 복원 하지 못하고 직선으로 복원됩니다.

4. Aliasing (=계단 현상)

= 샘플에서 재구성된 신호가 원래 연속 신호와 다를 때 발생하는 왜곡 또는 아티팩트를 가리킵니다.

1) Spatial Aliasing (공간 Aliasing)



- 좌측의 Original Image를 16개의 Sample을 사용하여 나타내면 우측의 그림처럼 올바르게 그려지지 못한다.

2) Temporal Aliasing (시간 Aliasing)

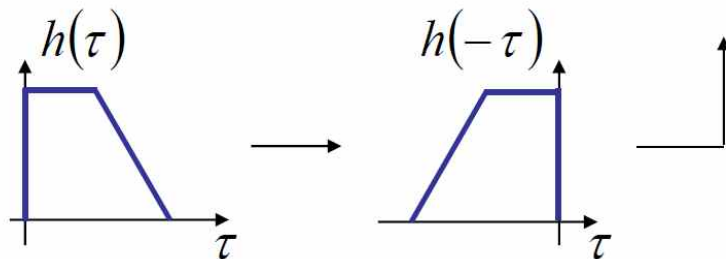
= 카메라에서 빠르게 연속적으로 이동하는 Image에서 Sample이 잘못되어서 이상하게 렌더링 되는 현상

5. Anti-Aliasing

- Sample을 진동수보다 2배 이상 높은 값으로 만들어냅니다. 하지만 이렇게 해도 언제나 성공적으로 aliasing을 방지하지는 못합니다.
- 원래 Signal의 진동수를 Filtering을 통해 낮추면, 그 진동수의 2배 초과로 Sample을 만들면 올바르게 복원됩니다.
- 진동수가 높으면 복구 가능한 Sample을 만들기 힘들기 때문에, 이론적으로는 진동수의 제한을 둡니다.

6. Convolution

$$g(x) = \int_{-\infty}^{\infty} f(\tau)h(x-\tau)d\tau \quad g = f * h$$



- 연산기호 : *
- T같이 생긴 기호를 타우라고 합니다. x축을 타우로 놓고 적분한 식을 해석해 보겠습니다.
- 두 함수 f와 h를 그래프 상에 표현하고 함수 f는 움직이지 않고 고정시킵니다. 함수 h는 x값에 따라 이동시키고 대칭시켜 $-\infty \sim \infty$ 만큼 이동시킵니다.
- 함수 h를 이동시키다보면 함수 f와 겹치는 부분이 나타나는데 , 겹치는 부분의 면적을 Convolution이라고 합니다.
- 적분 공간이 무한대(연속 공간)이므로 컴퓨터에서 계산이 불가능하므로 , 유한 공간(이산 공간)으로 변화시켜서 구합니다.
- 많이 겹칠수록 g의 값이 커지고 , 적게 겹칠수록 g의 값이 0에 가까워집니다.
- linear & shift invariant를 만족합니다.
- convolution에 대해 교환법칙이 성립합니다.

ex) $g(2)$: 고정된 함수 f와 x축에서 2만큼 이동시키고 대칭시킨 함수 h와 그래프에서 겹친 부분의 넓이

<Convolution을 불과 연기로 설명>

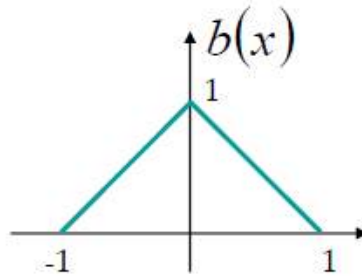
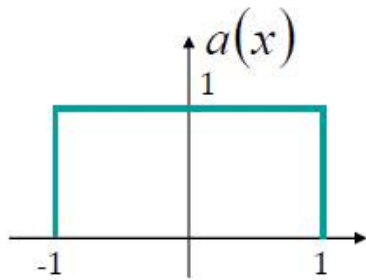
$$f * s = \sum_{i=0}^5 f(i) \cdot S(5 - i)$$

$$\int_0^t f(\tau) S(t - \tau) d\tau$$

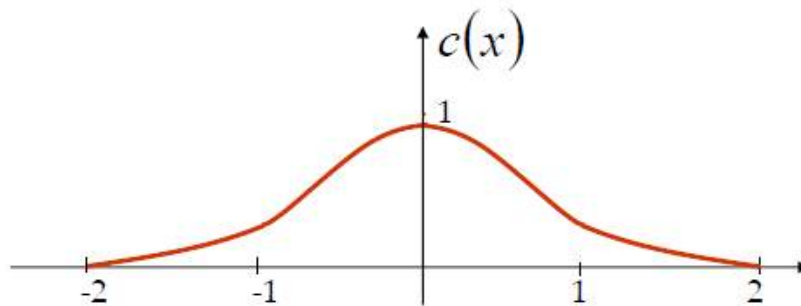
Question : 1분마다 성냥으로 불을 피웁니다. 불이 피울 때 마다 연기가 발생하게 되는데 5분 뒤에 공기 중에 연기의 양을 구하시오.

- Function_f = 분마다 피는 불의 개수
- Function_s = 공기 중의 성냥으로 인한 연기의 양
- 0분에 불을 1개 만들면 f(1)이고 , 57분에 불을 2개 만들면 f(2)입니다.
- s(값)에서 값은 연기가 공기 중에 존재한 시간
- 4시 0분에 불 1개 펴 -> 연기는 1 * s(0)
- 4시 1분에 불 2개 펴 -> 연기는 1 * s(1) + 2 * s(0)
- 4시 2분에 불 4개 펴 -> 연기는 1 * s(2) + 2 * s(1) + 4 * s(0)
- 4시 2분에 구한 식을 해석하면 , 1 * s(2)에서 1은 4시 0분에 피운 불의 개수 이고 그 때 피운 불로 발생한 연기는 공기 중에 존재한 지 2분이 지났습니다. 2 * s(1)에서 2는 4시 1분에 피운 불의 개수 이고 그 때 피운 불로 발생한 연기는 공기 중에 존재한 지 1분이 지났습니다. 4 * s(0)에서 4는 4시 2분에 피운 불의 개수 이고 그 때 피운 불로 발생한 연기는 공기 중에 존재한 지 0분이 지났습니다.
- 5분 뒤에 연기의 양은 좌측 그림의 식이고 , 이걸 이산적으로 생각하지 않고 연속적으로 생각하면 우측 그림의 식이 되며 적분을 이용합니다.

Q&A : 컨벌루션의 결과는?



$$\downarrow c = a * b$$

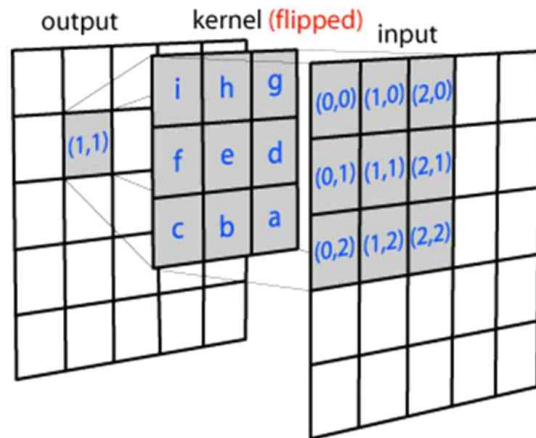


7. Convolution을 이용한 Filtering

- Filtering : 입력 영상에 filter mask를 convolution 해서 나오는 결과 영상
- Input Image에 filter mask를 입혀 연산을 하여 새로운 Output Image를 만드는데 이걸 변환(Transformation)으로 생각해도 괜찮은 것 같습니다. 그러므로 어떤 벡터에 변환을 가할 때 다양한 변환 행렬에 따라 다양한 결과 벡터가 출력되는 것처럼 다양한 filter mask를 입혀 filtering을 하면 다양한 Output Image가 출력됩니다.
- 동의어 : Filter Mask = Convolution Mask = Convolution Filtering Mask = Mask = Filter = kernel(filter의 하위 개념)
- 저는 Filter Mask로 사용하겠습니다.

1) Filtering Overview

<그림>



- Filter Mask가 (3x3)입니다.
- Output의 (1,1)을 구하기 위해 input의 (1,1)에 Mask의 중점을 올리고 겹치는 부분에 대해서 convolution을 진행합니다.
- 연산을 할 때 Mask는 Flip 시킵니다.
- 그 후 (Input의 0,0의 값 * i) + (input의 1,0의 값 * h) + etc + (Input의 2,2의 값 * a)를 구합니다.
- 이 계산 과정을 Convolution이라고 합니다,
- 예제 연습해서 맞추었습니다!!!

2) Filter Mask 특징

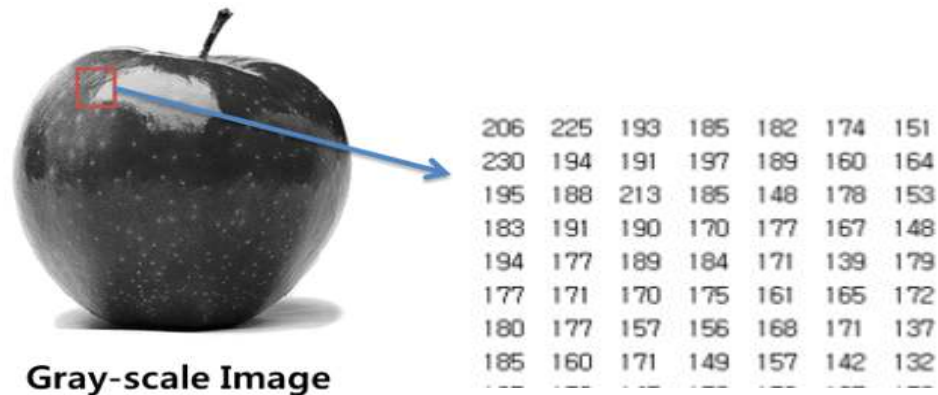
- Filter Mask의 크기는 홀수(3x3, 5x5, 7x7)
- Filter Mask의 모든 원소 합이 1이 되도록 설계하면 필터링 결과 영상의 평균 밝기가 입력 영상 평균 밝기와 같게 유지됩니다.
- Filter Mask의 모든 원소 합이 1 보다 작을 경우 영상의 평균 밝기가 어두워지고 1보다 크면 좀 더 밝아집니다.
- Filter Mask의 연산 결과 값이 음수일 수 있습니다. 음수라면 0으로 만들어 버리거나, 전체 값에 일정한 상수를 더하는 방식을 이용합니다.
- 1D Filter Mask로 2번을 연산하는 것($O(2n)$)이 2D Filter Mask 1번으로 연산하는 것($O(n^2)$) 보다 훨씬 빠릅니다.

3) Filter Mask를 input 영상과 겹쳤을 때 겹치지 않는 부분 해결책

- ① Filter Mask에서 input과 겹치지 않는 Filter Mask에서 h값은 0으로 합니다. 왜곡이 발생합니다.
- ② Filter Mask가 최대한 겹치도록 Filter Mask를 이동시킵니다. output 영상의 크기가 작아집니다.
- ③ Input 영상의 크기를 감소시킵니다.

4) Filtering 예시

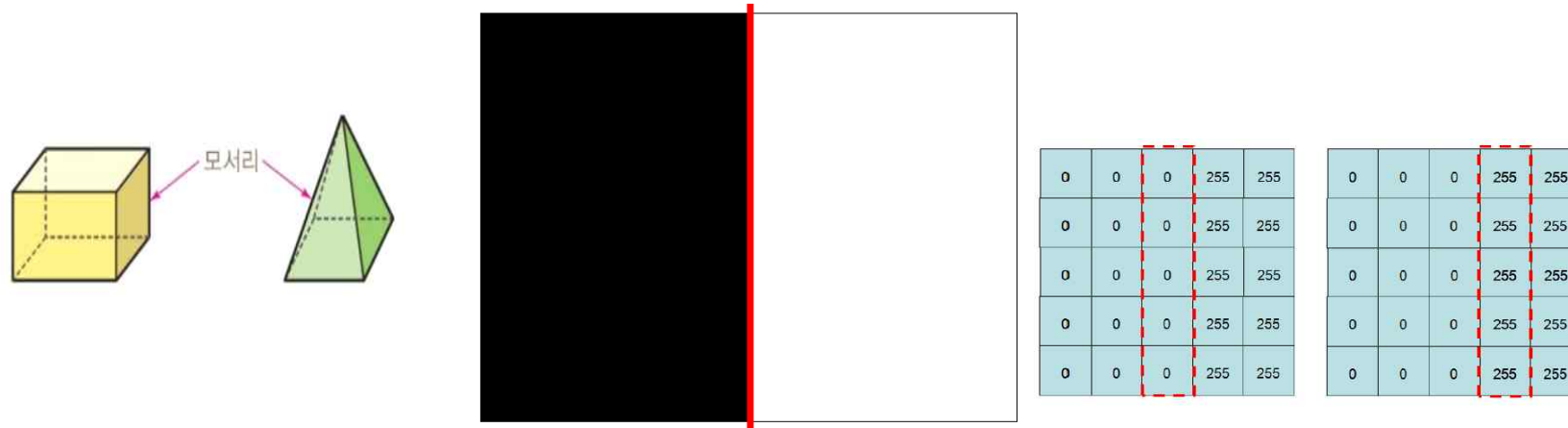
- 흑백 image의 Input, Output에는 밝기 값이 들어갑니다.



-1	-1	-1
-1	9	-1
-1	-1	-1

- Input의 (1,1)의 밝기 값은 194입니다. 거기에 위의 Filter를 이용해서 convolution을 해봅시다.
- 121로 밝기가 변해서 더욱 어두워집니다.

8. 경계 검출 (Edge Detection)



1) 경계의 정의

= 객체의 외각선의 집합

- 영어로 Edge는 모서리이며 , 모서리는 객체의 외각에 존재하는 선이고 , 결국 이는 전체 Image에서 객체와 배경을 구분하는 경계가 됩니다.
- 전체 영상의 픽셀 비율로 따지면 매우 적습니다.
- 데이터는 이산적이고 잡음이 존재합니다.
- 좌측은 실제 그림이며 pixel로 나타낸 그림이 오른쪽 그림입니다. 여기서 컴퓨터가 경계를 어떻게 판단하는지 알아보시다.
- 컴퓨터는 미분을 이용해서 경계를 판단합니다.
- 열 집합의 밝기 값 차이가 크면 y축과 평행한 경계를 구할 수 있고(위의 그림), 행 집합의 밝기 값 차이가 크면 x축과 평행한 경계를 구할 수 있습니다.

$$\frac{\Delta I}{\Delta x} \approx I(x + \Delta x) - I(x) \approx I(x) - I(x - \Delta x)$$

$$\frac{\Delta I}{\Delta x} \approx I(x + \Delta x) - I(x - \Delta x)$$

- 교수님께서 좌측의 식을 매우 강조하셨습니다.
- 식을 위의 그림에서의 3열로 설명하면 , 3열은 2열과 비교하면 밝기 값의 차이가 없고 , 4열과 비교하면 모든 pixel에서 밝기 값의 차이가 존재합니다.
- 아래 식은 3열과 4열과의 밝기 차이 - 3열과 2열과의 밝기 차이의 차 입니다.

2) 경계의 위치에 따른 미분

 $\nabla I = \left[\frac{\partial I}{\partial x}, 0 \right]$	 $\nabla I = \left[0, \frac{\partial I}{\partial y} \right]$	 $\nabla I = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right]$
경계가 y축과 평행	경계가 x축과 평행	경계와 수직방향

- 위의 그림은 무조건 기억해야 합니다.
- 모든 경계가 x축 OR y축에 평행하지 않으므로 Gradient 개념을 도입합니다.
- Gradient : 임의의 경계(축과 평행하지 않음)를 구하는 방식

9. (Image) Noise (잡음)

1) 정의



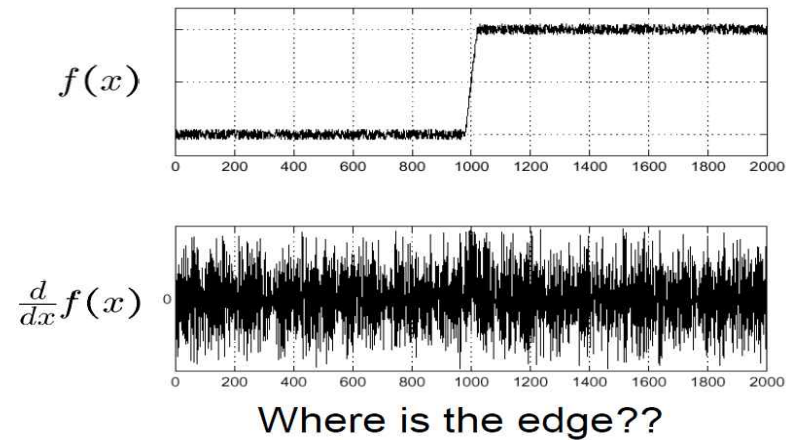
Image with noise



Image without noise

- = 이미지의 밝기 또는 색상 정보의 무작위 변화이며 일반적으로 전자 노이즈의 한 측면입니다. 이미지 센서와 스캐너 또는 디지털 카메라의 회로로 생성 할 수 있습니다. 이미지 노이즈는 필름 그레인과 이상적인 광자 검출기의 피할 수 없는 샷 노이즈에서도 발생할 수 있습니다.
- Discrete(불연속적인)한 Image는 언제나 noise 존재합니다.

2) Noise의 문제점

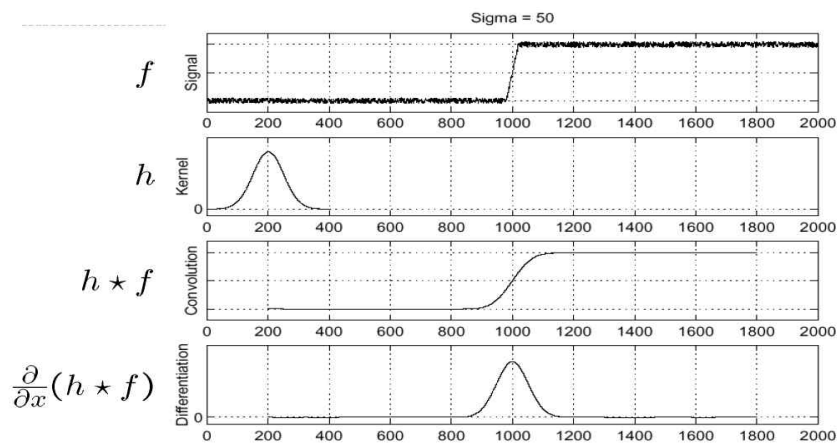


- $f(x)$ 는 위치에 따른 밝기 값 그래프입니다.
- 인간은 x 가 1000일 때 갑자기 밝기 값이 변하는 것을 확인할 수 있으므로 해당 지점이 경계라는 것을 쉽게 알 수 있습니다.
- 하지만 컴퓨터가 실제로 이 그래프를 미분하게 되면 noise로 인해 미분한 그래프의 결과를 보고 어디가 경계인지 찾을 수 없습니다.

3) 해결책

- 3가지 수학적 kernel로 해결하기 : 그래프를 교수님께서 매우 강조하셨습니다.

① 단순 Convolution

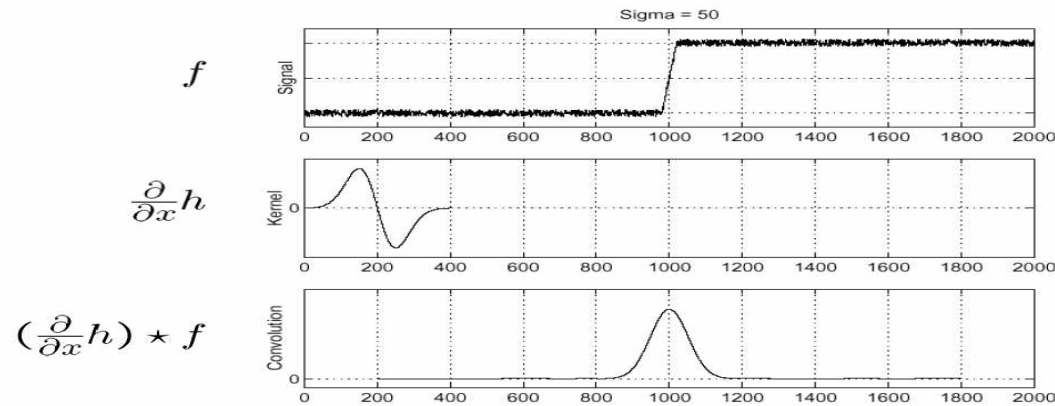


- 밝기 그래프를 Gaussian smoothing kernel과 convolution을 해서 새로운 그래프를 만들고 이를 미분하면 정확하게 경계(edge)를 검출할 수 있습니다.

- 3가지 그림(강, 1차미분, 2차미분)
- 2차 미분은 변곡점이 경계다.

② 미분 + Convolution

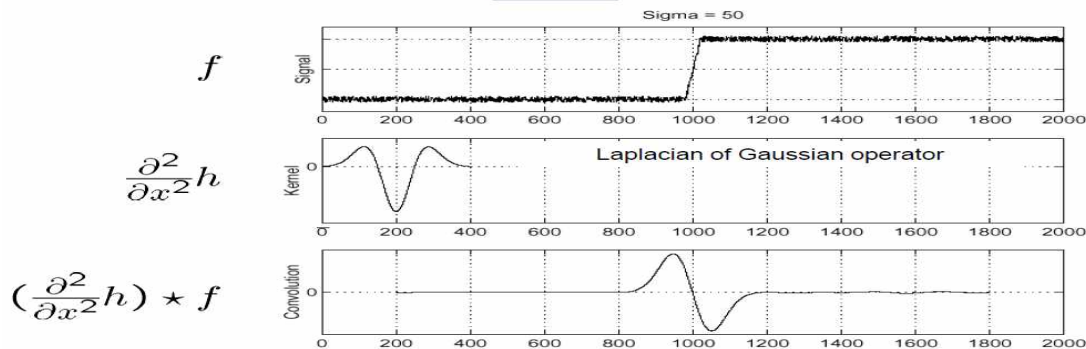
$$\frac{\partial}{\partial x}(h \star f) = \left(\frac{\partial}{\partial x}h\right) \star f \quad \dots \text{saves us one operation.}$$



- 밝기 그래프를 미분한 Gaussian smoothing kernel과 convolution을 해서 새로운 그래프를 만들고 이를 미분하면 정확하게 경계(edge)를 검출할 수 있습니다.

③ 2번 미분 + Convolution

$$\frac{\partial^2}{\partial x^2}(h \star f) = \left(\frac{\partial^2}{\partial x^2}h\right) \star f \quad \leftarrow \text{Laplacian of Gaussian}$$



Where is the edge?

Zero-crossings of bottom graph !

- 밝기 그래프를 2번 미분한 Gaussian smoothing kernel과 convolution을 해서 새로운 그래프를 만들고 이를 미분하면 정확하게 경계(edge)를 검출할 수 있습니다.
- 결과 그래프에서 변곡점(극점 -> 극점)이 경계 입니다.

10. Quantization (양자화)

1) 정의



$I(x,y)$



$P(x,y)$

- = 고화질의 Image의 밝기 값을 단계별(단계 마다 범위 설정) 밝기 값으로 구분합니다.
- 그림에서 $I(x,y)$ 밝기 값은 8bit로 표현하여 255개의 밝기 값을 표현합니다.
- 그림에서 $I(x,y)$ 밝기 값은 2bit로 표현하여 4개의 밝기 값을 표현합니다.
- $I(x,y)$ 밝기 값을 $P(x,y)$ 밝기 값으로 변경합니다.
- Quantization은 결국 밝기 값을 나타내는 값이 한정된 범위(ex :: 0 ~ 255)를 가져서 탄생한 결과입니다.

2) 문제점



8 bits



4 bits



2 bits



1 bit

- 8bit는 255 단계지만 연속적인 밝기라고 가정하면 , 이를 1bit로 만들면 2단계(0 OR 1)의 단계별 밝기 값으로 만들어버립니다. 그로 인해 영상에서 표시할 수 있는 밝기가 2단계뿐이라 **영상의 품질이 매우 감소**합니다.

3) 해결책

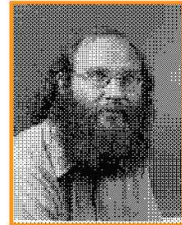
① Dithering Algorithm



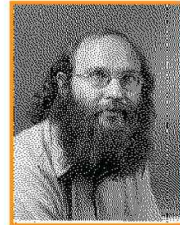
Original
(8 bits)



Random
Dither
(1 bit)



Ordered
Dither
(1 bit)



Floyd-Steinberg
Dither
(1 bit)



Uniform
Quantization
(1 bit)

- Original 보다는 Image의 성능을 개선시킬 수 없습니다.
- 하지만 Uniform Quantization Image보다는 대폭 성능을 개선시킬 수 있습니다.

<Random Dither>

= Filtering을 통해 바꾼 Image의 pixel 마다 Random 값(noise)을 더해 줍니다.

- 성능 : 하

<Ordered Dither>

$$\frac{1}{5} \begin{bmatrix} 1 & 3 \\ 4 & 2 \end{bmatrix} \quad \frac{1}{10} \begin{bmatrix} 3 & 7 & 4 \\ 6 & 1 & 9 \\ 2 & 8 & 5 \end{bmatrix} \quad \frac{1}{17} \begin{bmatrix} 1 & 9 & 3 & 11 \\ 13 & 5 & 15 & 7 \\ 4 & 12 & 2 & 10 \\ 16 & 8 & 14 & 6 \end{bmatrix} \quad \frac{1}{65} \begin{bmatrix} 1 & 49 & 13 & 61 & 4 & 52 & 16 & 64 \\ 33 & 17 & 45 & 29 & 36 & 20 & 48 & 32 \\ 9 & 57 & 5 & 53 & 12 & 60 & 8 & 56 \\ 41 & 25 & 37 & 21 & 44 & 28 & 40 & 24 \\ 3 & 51 & 15 & 63 & 2 & 50 & 14 & 62 \\ 35 & 19 & 47 & 31 & 34 & 18 & 46 & 30 \\ 11 & 59 & 7 & 55 & 10 & 58 & 6 & 54 \\ 43 & 27 & 39 & 23 & 42 & 26 & 38 & 22 \end{bmatrix}$$

= 인접한 숫자가 최대한 떨어지도록 행렬에서 밝기 값을 분포합니다. 이전에는 랜덤 값을 더해주었지만 이 기법에서는 바꾼 image의 pixel 마다 규칙이 있는 행렬을 값에 더해줍니다.

- 행렬 사이즈 마다 격자가 바꾼 image에 나타납니다.
- 성능 : 중

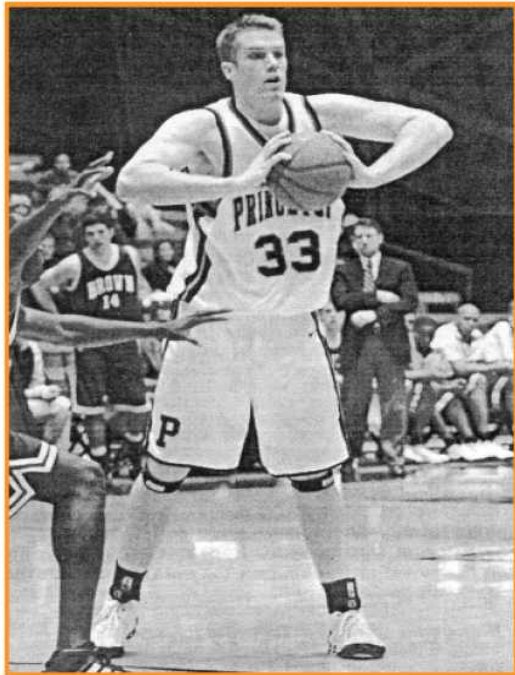
<Error Diffusion Dither : Floyd-Steinberg Dither>

= Error를 분산시켜서 만든 식을 바꾼 Image에 더해주는 방식

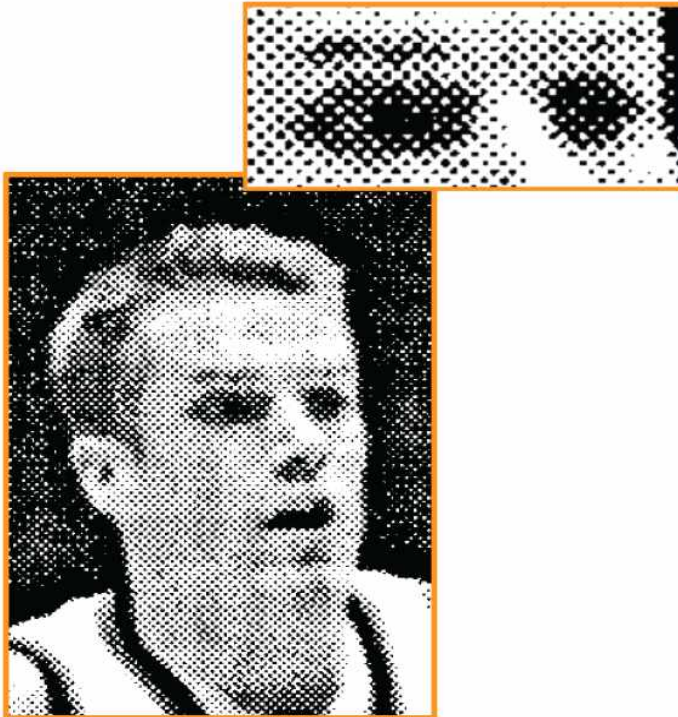
- 성능 : 상
- 당연히 , Original Image 보다는 성능이 좋지 않습니다.

② Classical Halftoning Algorithm

- = 어두울수록 큰 원으로 표현하고 , 밝을수록 작은 원으로 표현하는 기법입니다.
- 신문에서 많이 사용합니다.



From Town Topics, Princeton



11. 다양한 Image Processing

1) Sharpen (선명하게)

= Image를 선명하게 하는 기법

- Image의 Edge 성분을 모두 더합니다.

0	-1	0
-1	5	-1
0	-1	0

마스크 1

-1	-1	-1
-1	9	-1
-1	-1	-1

마스크 2

10	10	10	10	10	10	10	10	10
10	10	10	10	10	10	10	10	10
10	10	10	10	20	10	10	10	10
10	10	10	20	30	20	10	10	10
10	10	20	30	40	30	20	10	10
10	10	10	20	30	20	10	10	10
10	10	10	10	20	10	10	10	10
10	10	10	10	10	10	10	10	10
10	10	10	10	10	10	10	10	10



0	10	10	10	10	10	10	10	10
10	10	10	10	0	10	10	10	10
10	10	10	10	40	10	10	10	10
10	10	0	20	50	20	0	10	10
10	0	40	50	80	50	40	0	10
10	10	0	20	50	20	0	10	10
10	10	10	10	40	10	10	10	10
10	10	10	10	0	10	10	10	10
10	10	10	10	10	10	10	10	10

- 마스크 1을 Filter로 하여 Input에 Filtering을 한 결과입니다.
- Output Image에서 십자가 부분의 밝기가 증가하여 Image의 선명도가 증가했습니다.

2) Emboss

= Image를 특정 방향의 gradient가 강조된 필터로 합성시키는 기법

$$\text{Filter} = \begin{bmatrix} -1 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

3) Non-Linear Filtering

ex) Oil , Stain Glass

<6주차 : Image Warping>

1. 정의



- = Source image를 뒤틀려서(warp) Destination image를 만드는 기법
- Pixel들은 알고리즘에 따라서 새로운 Pixel의 위치로 이동합니다.
- Image Warping은 Mapping(색은 그대로 , 위치만 변화) + Re-sampling(색이 변함)

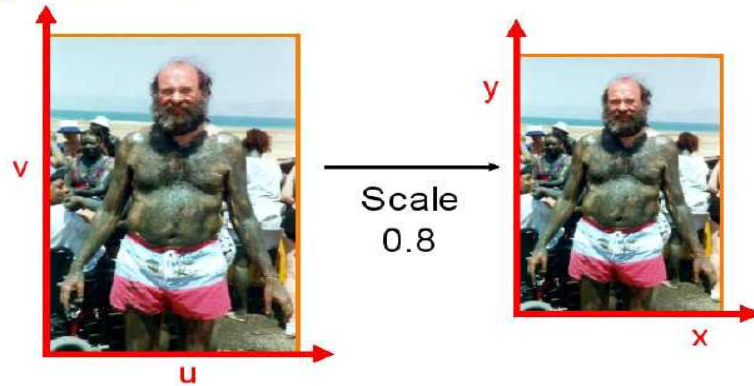
2. Mapping (=Pixel의 이동)

- = Source image의 pixel이 destination image의 어떤 위치로 이동하는지
- forward mapping : source image를 통해 destination image를 구합니다.
- reverse mapping : destination image를 통해 source image를 구합니다.

1) Parametric mapping

① 크기

- $x = factor * u$
- $y = factor * v$

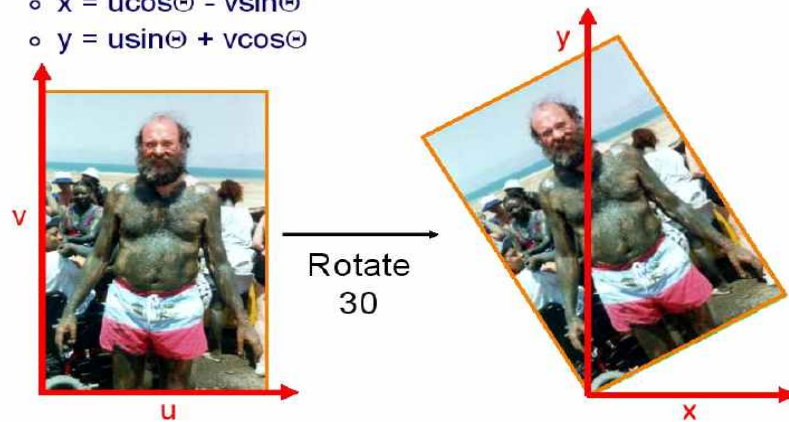


- Image의 가로 , 세로의 크기가 달라질 때 상수 factor를 곱해서 올바르게 크기를 변환합니다.
- GPU에서 이런 계산들을 행렬을 통해 쉽게 처리합니다.

② 회전

- Rotate by Θ degrees:

- $x = u \cos \Theta - v \sin \Theta$
- $y = u \sin \Theta + v \cos \Theta$

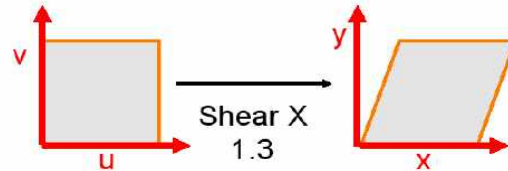


- 2D는 회전변환으로 쉽게 계산합니다.
- 3D라면 좌표가 x,y,z이므로 2D와는 다릅니다. 그리고 x,y,z 말고 1개의 인자를 더 사용하여 4D로 계산식을 만듭니다. 행렬 계산에서 4D가 유리합니다.

③ Shear

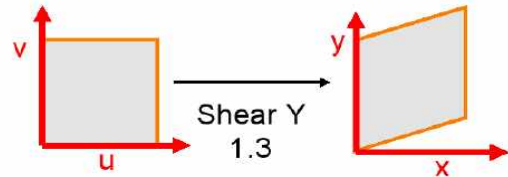
- Shear in X by *factor*:

- $x = u + \text{factor} * v$
- $y = v$

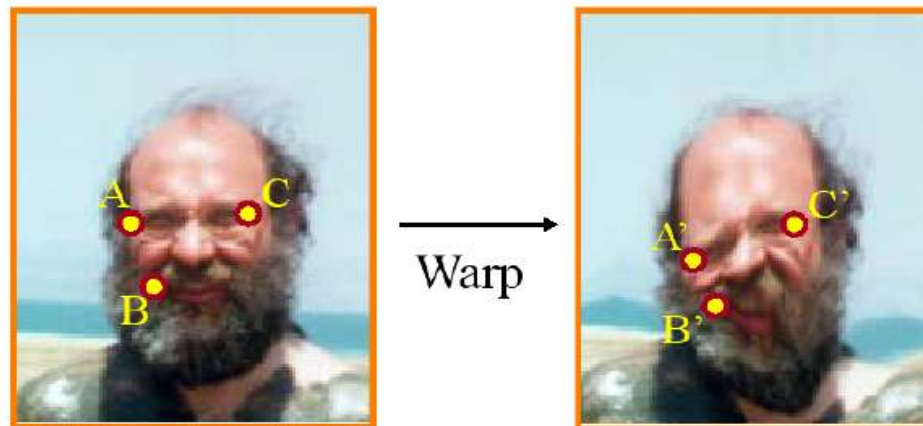


- Shear in Y by *factor*:

- $x = u$
- $y = v + \text{factor} * u$



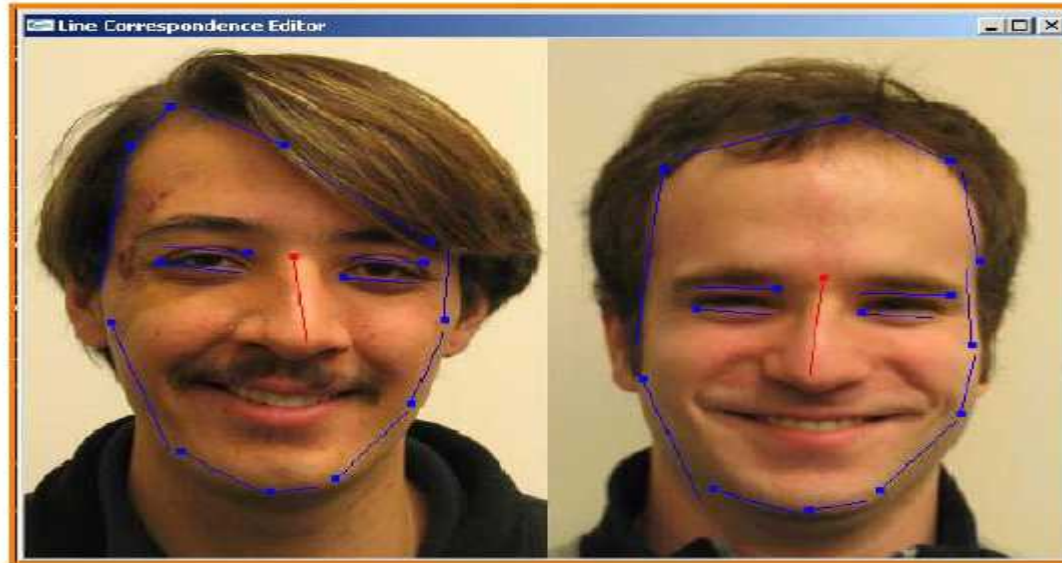
2) Point Correspondence Mapping (=점과 점의 대응)



- 그림에서 특정 vertex의 이동을 기준으로 나머지 pixel들을 이동시킵니다.
- 점 A,B,C를 A',B',C'으로 이동하면 P는 P'로 이동합니다.
- P가 A에 가까울수록 Warp의 결과인 P'이 A'에 가깝습니다. (가중치)

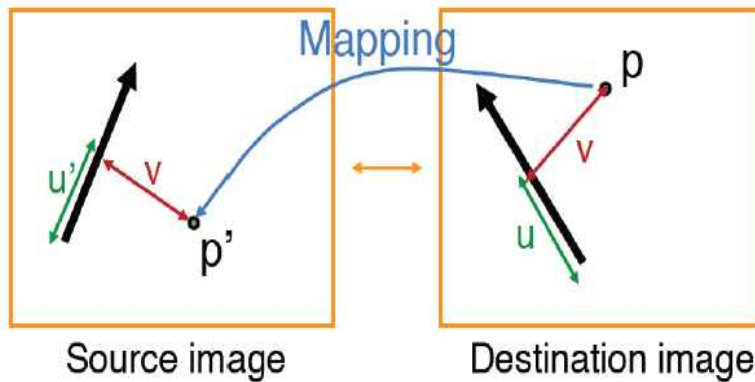
3) Line Correspondence Mapping (=선과 점의 대응)

- 그림에서 특정 Line의 이동을 기준으로 나머지 pixel들을 이동시킵니다.



<Example>

- 사람 얼굴 변화
- 사람의 늙는 과정
- 다른 사람의 얼굴로 변화



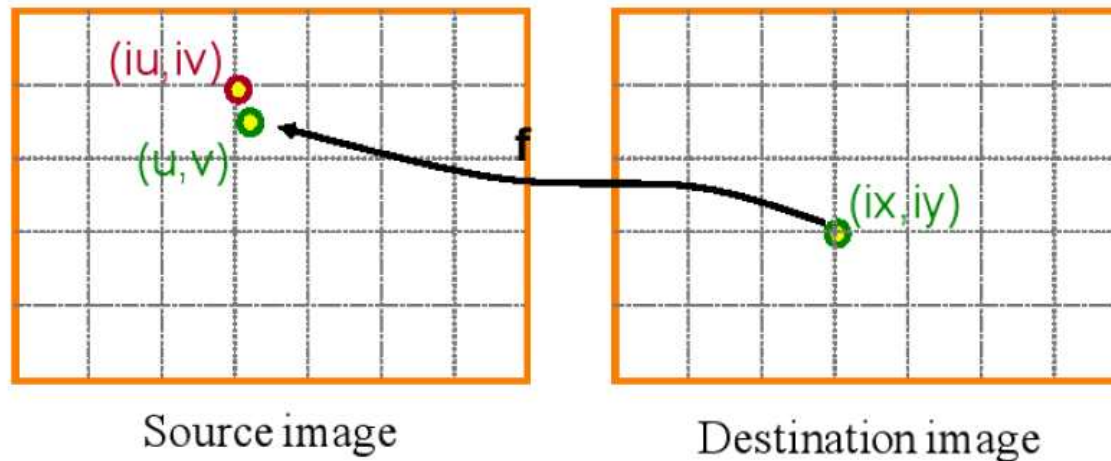
<원리>

- 선이 하나라고 할 때, 어떤 점과 해당 선의 수직 거리를 v . 선의 시작점에서 수선의 발까지 거리를 u 라고 합시다.
- 선을 이동시키면 이동한 선에서 u 만큼 이동하고, v 만큼 수직으로 수선의 발을 그려 destination image의 pixel을 구합니다.
- 선이 많아지면 같은 원리를 이용해서 적당한 점을 찾아서 Mapping 합니다.

3. Re-sampling

= destination image에 존재하는 pixel들에게 어떤 색상을 입힐까?

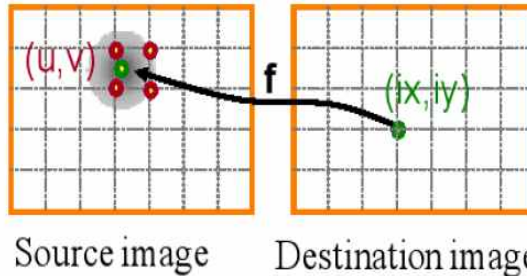
1) Point Sampling : Poor Re-sampling



- 그림은 격자로 이루어져 있고 격자의 꼭짓점만이 모두 pixel입니다. 그러므로 (iu, iv) , (ix, iy) 는 pixel 이지만 (u, v) 는 pixel이 아닙니다.
- Source image의 (u, v) 의 근처에는 꼭짓점(pixel)이 4개가 있고 , 가장 근처에는 (iu, iv) 가 존재합니다.
- Destination image의 (ix, iy) 을 reverse matching으로 Source image의 pixel을 찾는다면 (u, v) 가 mapping됩니다. 하지만 (u, v) 의 위치에는 pixel이 존재하지 않습니다. 그렇기 때문에 Poor Re-sampling은 (u, v) 를 기준으로 가장 근처에 있는 pixel의 밝기 값을 사용합니다. 그로 인해 가장 가까운 (iu, iv) pixel의 밝기 값을 (ix, iy) 로 사용합니다.
- Poor Re-sampling은 알고리즘이 간단하므로 계산이 매우 빠르지만 (iu, iv) pixel의 밝기 값이 (u, v) 위치의 밝기 값과 같음을 보장할 수 없기 때문에 sampling을 했을 때 결과 이미지의 상태가 좋지 않습니다.
- 물결 그림이 나타날 수 있습니다.

2) Re-Sampling with filter

```
float Resample(src, u, v, k, w)
{
    float dst = 0;
    float ksum = 0;
    int ulo = u - w; etc.
    for (int iu = ulo; iu < uhi; iu++) {
        for (int iv = vlo; iv < vhi; iv++) {
            dst += k(u,v,iu,iv,w) * src(u,v)
            ksum += k(u,v,iu,iv,w);
        }
    }
    return dst / ksum;
}
```



*** 코드를 시험에 출제하십니다. ***

<코드 설명>

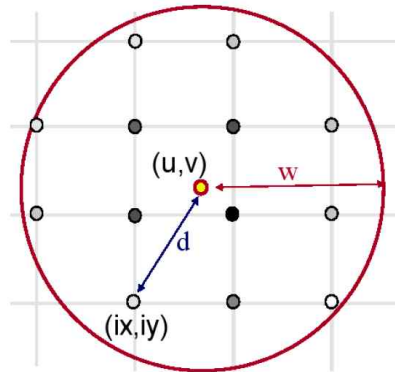
- vlo = v-w
- uhi = u+w
- vhi = v+w
- src(u,v) -> src(iu,iv)

<Return 값에서 ksum으로 나누는 이유>

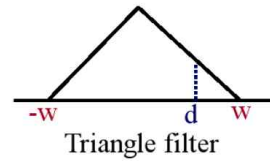
- dst는 밝기 값을 더하게 되므로 값이 255를 넘어가 범위를 초과할 수 있기 때문입니다. 255를 넘어가면 존재하지 않는 밝기 값 입니다.

- Poor re-sampling 보다 훨씬 정확한 방식으로 (ix,iy)의 밝기 값을 결정합니다.
- pixel이 아닌 (u,v) 기준으로 일정 거리 이내에 존재하는 모든 pixel의 밝기 값을 가중치를 사용하여 섞어서 평균을 냅니다.
- (u,v)를 기준으로 일정 거리 이내의 pixel이므로 원을 그려 이용합니다. 원의 반지름은 w이며, (u,v)와 pixel의 거리는 d이고, w이내에 존재하는 pixel에 대해서 가중치를 선정합니다. w를 키워버리면 많은 pixel을 검사하게 되며 영상에서 blur와 smoothing이 발생합니다.
- 가까울수록 가중치가 커지는데, 가중치를 정하는 방식이 2가지가 존재합니다.

① Triangle filter



Filter Width = 2

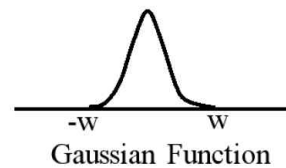
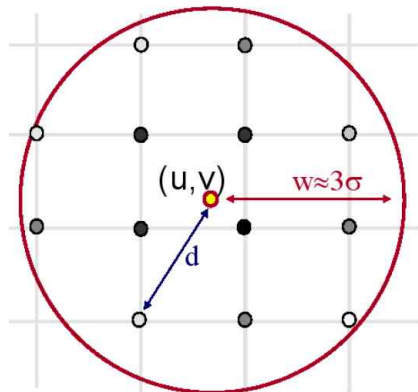


$$k(i,j) = \max(1 - d/w, 0)$$

② Gaussian Filter

Kernel is Gaussian function

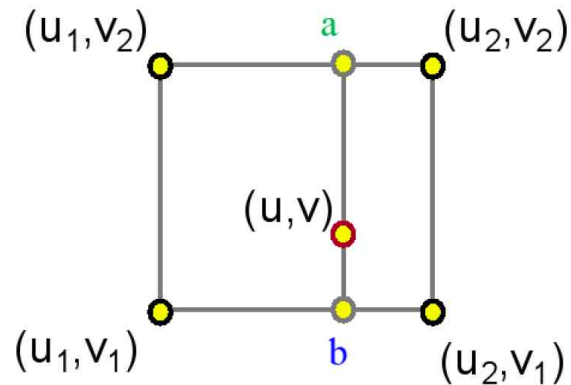
$$G(d, \sigma) = e^{-d^2 / (2\sigma^2)}$$



- Drops off quickly, but never gets to exactly 0
- In practice: compute out to $w \sim 2.5\sigma$ or 3σ

<w가 작아서 원 내부에 pixel이 0개일 때 해결 방법 : 3 Linear interpolation>

- a = linear interpolation of $\text{src}(u_1, v_2)$ and $\text{src}(u_2, v_2)$
- b = linear interpolation of $\text{src}(u_1, v_1)$ and $\text{src}(u_2, v_1)$
- $\text{dst}(x, y)$ = linear interpolation of " a " and " b "



Filter Width < 1

- 원 내부에 pixel이 없으므로 가장 근처의 pixel 4개를 이용합니다.
 - 원리 : 위치가 0일 때의 값이 10이고 위치가 1일 때의 값이 20이면 , 위치가 0.5일 때의 값은 15입니다. (=비례식)
- ① (u_1, v_2) 의 밝기 값과 (u_2, v_2) 의 밝기 값의 거리 비율(그림에서는 대충 3:1)에 대해서 a 의 밝기 값을 구합니다.
 - ② (u_1, v_1) 의 밝기 값과 (u_2, v_1) 의 밝기 값의 거리 비율(그림에서는 대충 3:1)에 대해서 b 의 밝기 값을 구합니다.
 - ③ a 의 밝기 값과 b 의 밝기 값의 거리 비율(그림에서는 대충 3:1)에 대해서 (u, v) 의 밝기 값을 구합니다.

3) Sampling의 예시



Point



Triangle



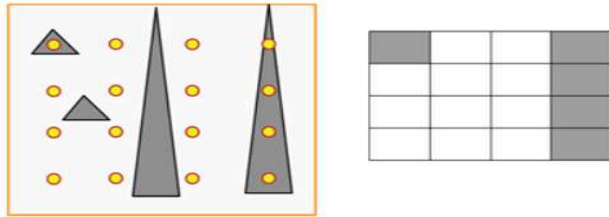
Gaussian

- ① point : 속도가 빠르고 , blur가 발생하지 않지만 aliasing이 발생합니다.
- ② Triange : 중간
- ③ Gaussian : 속도가 느리고 , aliasing이 발생하지 않지만 시그마 값이 작으면 blur가 발생합니다.

<7주차 : 중간고사 질문>

*** 교수님의 답변을 빨간색으로 처리 ***

1. sample과 pixel의 관계에 대해 명확히 이해하지 못했습니다. 둘은 엄밀히 말하면 포함관계 인가요?



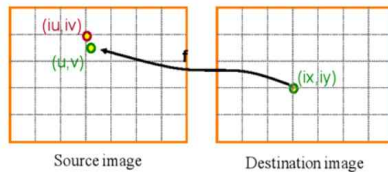
- 좌측 그림은 sample의 결과와 우측 그림은 pixel의 결과입니다. 그러면 결국 sample은 매우 작은 점이고 , 해당 점이 가리키는 위치와 동일한 밝기(색상)를 pixel이라는 정사각형 전체에 채워 넣는다고 이해해도 괜찮을까요? 앞의 가정이 옳다면 이미지에서 sample의 개수와 pixel의 개수는 동일한가요?

- 네 맞습니다.

- sample은 매우 작은 점이고 , 해당 점이 가리키는 위치와 동일한 밝기(색상)를 pixel이라는 정사각형 전체에 채워 넣습니다.

2. 아래의 그림을 보시면 (ix, iy) 가 source image의 pixel이 존재하지 않는 위치인 (u, v) 로 mapping이 된 건지 , sample이 존재하지 않는 위치인 (u, v) 로 mapping이 된 건지 모르겠습니다.

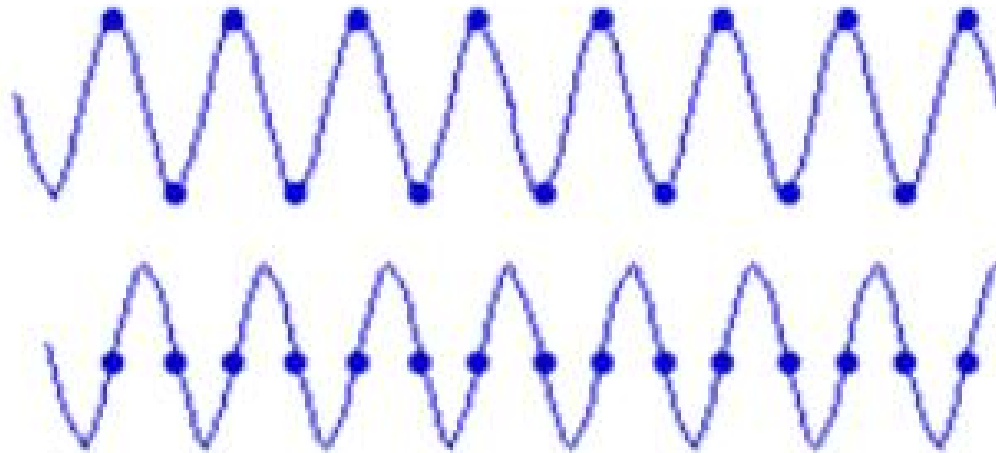
1) Point Sampling : Poor Re-sampling



- 둘 다 맞습니다.

- Pixel과 Sample은 포함관계 이므로 굳이 구분하지 않아도 괜찮습니다.

3. 진동수의 개수



$$f_s = 2f$$

$$f_s = 2f$$

- 여기서 f 는 8이고, $2f$ 는 16이 아닌가요? 진동의 개수를 셀 때  이 모양의 반복이 아닌가요?
- 그림에서 f_s 는 15개인데 제 생각대로라면 $2f$ 는 16이라 질문 드립니다.
- 학생의 생각이 맞아요. 진동수의 개수는 같은 패턴이 동일 구간에서 반복하는 횟수예요.

4. kernel vs filter

Kernel vs Filter

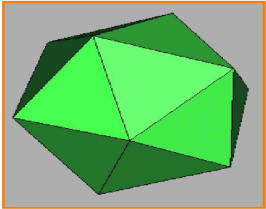
Before we dive into it, I just want to make the distinction between the terms 'kernel' and 'filter' very clear because I have seen a lot of people use them interchangeably. A kernel is, as described earlier, a matrix of weights which are multiplied with the input to extract relevant features. The dimensions of the kernel matrix is *how the convolution gets its name*. For example, in 2D convolutions, the kernel matrix is a 2D matrix.

A filter however is a concatenation of multiple kernels, each kernel assigned to a particular channel of the input. Filters are always one dimension more than the kernels. For example, in 2D convolutions, filters are 3D matrices (which is essentially a concatenation of 2D matrices i.e. the kernels). So for a CNN layer with kernel dimensions $h \times w$ and input channels k , the filter dimensions are $k \times h \times w$.

- filter가 1차원이 왜 높은지 이해가 가지 않아 kernel과 filter의 차이점을 명확하게 이해하지 못하겠습니다.
- 교수님께서도 강의에서 kernel을 강조하셨는데 그 때 저는 filter랑 동의어라고 생각하고 filter가 중요하다고 이해했었습니다.
- Kernel은 filter의 하위개념으로 이해하면 좋을 듯해요.
- 1차원 gaussian kernel 을 x축, y축, z축에 적용하면 3차원 필터가 됩니다.

<8주차 : Modeling>

<강의 개요>



- 현실 세계는 3D 세계입니다. 3D 세계에 존재하는 **3D Object들을 어떻게 그릴지에 대해서 알아보시다.**
- 녹색 Object는 간단하게 삼각형의 집합으로 2D에서 나타낼 수 있지만 , 훨씬 더 복잡한 헬리콥터 , 인간 뼈는 어떻게 표현하는지 알아보시다.

<강의 전체 Outline>

3D Object Representations

- Points
 - Range image
 - Point cloud
- Surfaces
 - Polygonal mesh
 - Subdivision
 - Parametric
 - Implicit
- Solids
 - Voxels
 - BSP tree
 - CSG
 - Sweep
- High-level structures
 - Scene graph
 - Application specific

- 3D Object를 좌측의 자료를 참고하면 수많은 방식으로 표현할 수 있습니다.
- 방식마다 image를 얻는 방식(Acquisition)이 다릅니다.
- 컴퓨터에서 데이터를 효과적으로 저장할 때 그에 맞는 자료구조가 있듯이 3D Object Representation(표현)에서도 그에 알맞은 기법이 존재합니다.

1. Modeling

*** **Object** = Unity의 GameObject처럼 Scene에 보이는 **물체**입니다. ex) 짱구 ***

*** Scene = 컴퓨터 화면 ***

1) 정의

= Scene에 무엇(Object)을 그릴 것인지

= Object를 선분의 집합으로 정의하려면 선분의 양 끝점 위치를 명시하는 작업

= 평면 다각형(Polygon)의 집합으로 **다각형 정점의 위치를 명시**하는 작업

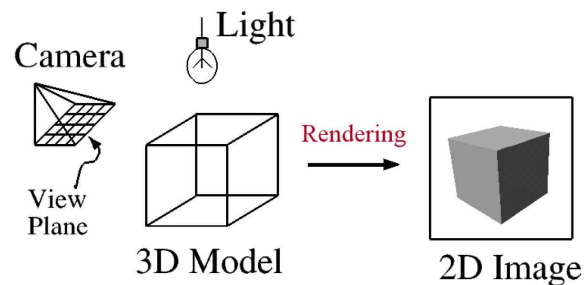
Modeling with GLU and GLUT

Prerequisites

An understanding of the simple modeling with polygon primitives.

2) Modeling VS Rendering

Construct image from 3D model



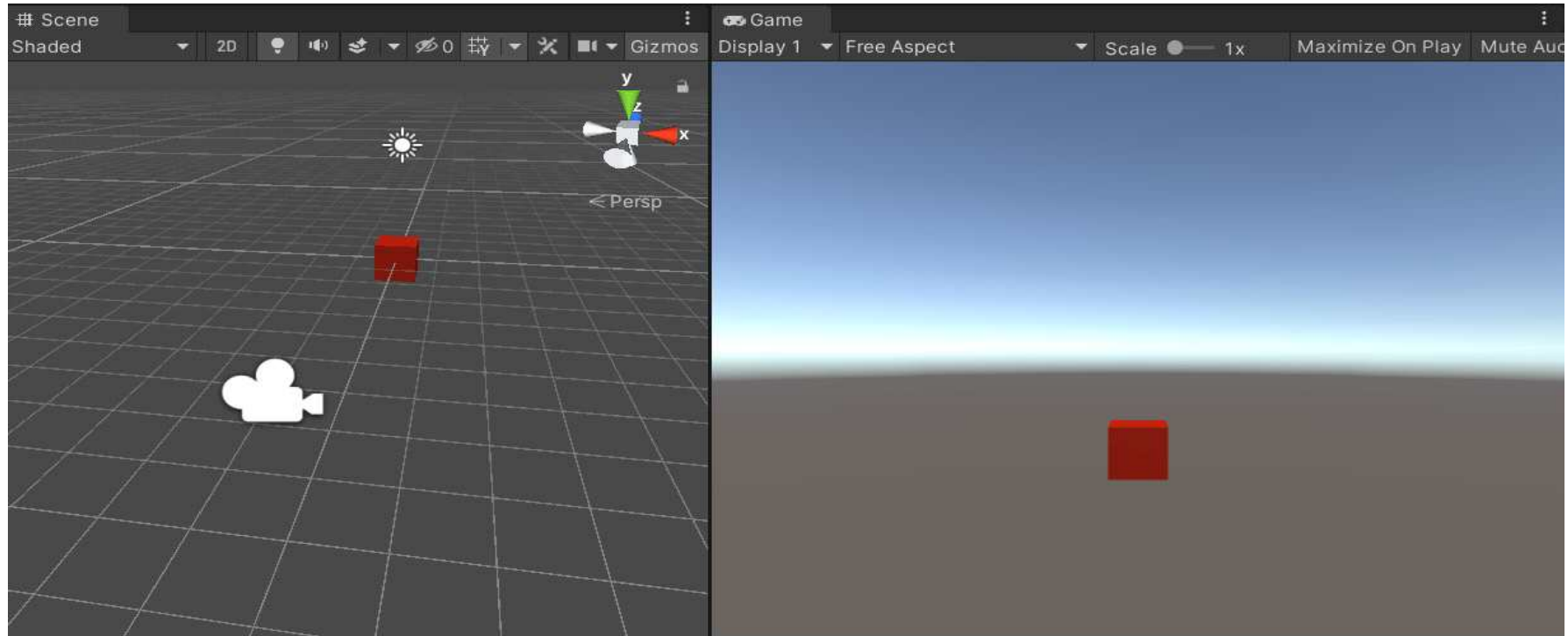
- ① **Modeling** : 3D Model을 좌표계에 Point를 찍어 구현하는 것.
- ② **Rendering** : Modeling 에서 만든 3D Model을 카메라로 촬영할 때 카메라의 View Plane에 2D Image로 3D Model이 그려지는 것.

- **Rendering** = *constructing 2D images from 3D models*
- **Modeling** = *representing 3D objects*

Modeling is the manipulation of a 3D model to the way the user wants or needs it to look (posing, animating, etc.), and rendering is the process of taking that finished modeling product and turning it into a high quality image, video, or a compiled and fully animated model for a game like a customized gun or an NPC. Modeling usually depends on the GPU because you're working in a 3D space, whereas rendering usually relies on your CPU (I say usually because it depends on the program and/or the rendering engine you choose to use).

- 先(선) 모델링 -> 後(후) 렌더링
- 물체의 모델링을 완벽히 처리했다라도 조명이 없으면(조명은 렌더링과 관련된 개념) 화면의 모든 물체는 검게 보입니다.
- 모델링은 물체를 만들어 내는 것을 말합니다. 이에 비해 렌더링은 모델링된 물체 영상을 그려내는 것을 의미합니다.

<Example : Unity>



- **Modeling** : 붉은색 Cube(=GameObject)를 Scene에 표현하는 기법
- **Rendering** : 붉은색 Cube가 Camera와 Light와 그 외에 Rendering에서 고려해야 할 여러 가지 사항을 고려해서 2D Game 창에 그려지는 기법
- Unity Asset Store로 인해 개발자가 modeling 하는 경우가 적어지는 것 같고 , Rendering을 더 공부해야 사실감 있게 표현할 수 있을 것 같습니다.

3) Rendering Vs Shader

Rendering is a very generic term which is basically can be defined as "creating an image", which can be, obviously, created in many ways. For example if you fill memory with color values (software rendering) it is also a "rendering".

Shaders are completely other thing. They are related to how modern GPUs work. Say you want to render a triangle. You supply triangle coordinates to GPU. You also load a shader to GPU. Then GPU is rendering data you supplied, it is executing your shader on every triangle vertex (vertex shader) and every triangle pixel (pixel shader). In shader you basically transform coordinates (vertex) and change color (pixel) of your final image. That's VERY basic on what shaders are. Shaders are very powerful and very fast, most effects in modern games are possible because of shaders.

As [@tkausl](#) correctly mentioned, modern GPU pipeline actually requires shaders do draw anything. They are often hidden behind frameworks like, for example, XNA. They can be called "Identity" shaders, which only transform your "screen coordinates" to OpenGL or DirectX coordinates and do not modify pixels, except for handling transparency.

So to answer your questions:

I am confused what's the difference between rendering and shaders?

They are not comparable.

and how rendering is performed using shaders?

I guess I answered that above.

Can we perform rendering without shaders?

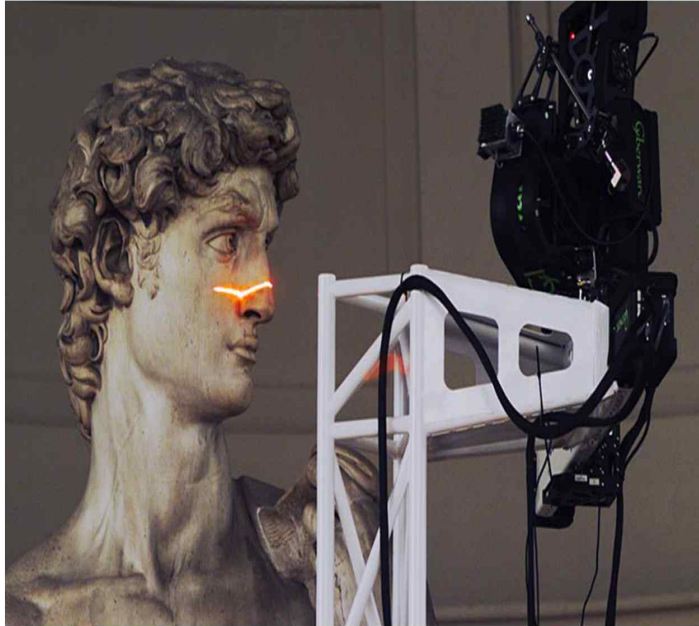
Absolutely we can.

are shaders used in both films(like toy story) and games?

Yes. Though I would guess they differ significantly since Pixar is not using your typical OpenGL or DirectX API, but the concept of "shader" should be the same. Also Pixar is not rendering in real-time, so they could use much more complex algorithms.

2. Point

1) Range Image



= Image는 객체의 강도와 색을 표현하지만 Range Image는 3D Object를 **스캐너로 측정하여 Object의 Depth(깊이)를 측정**해서 저장합니다.

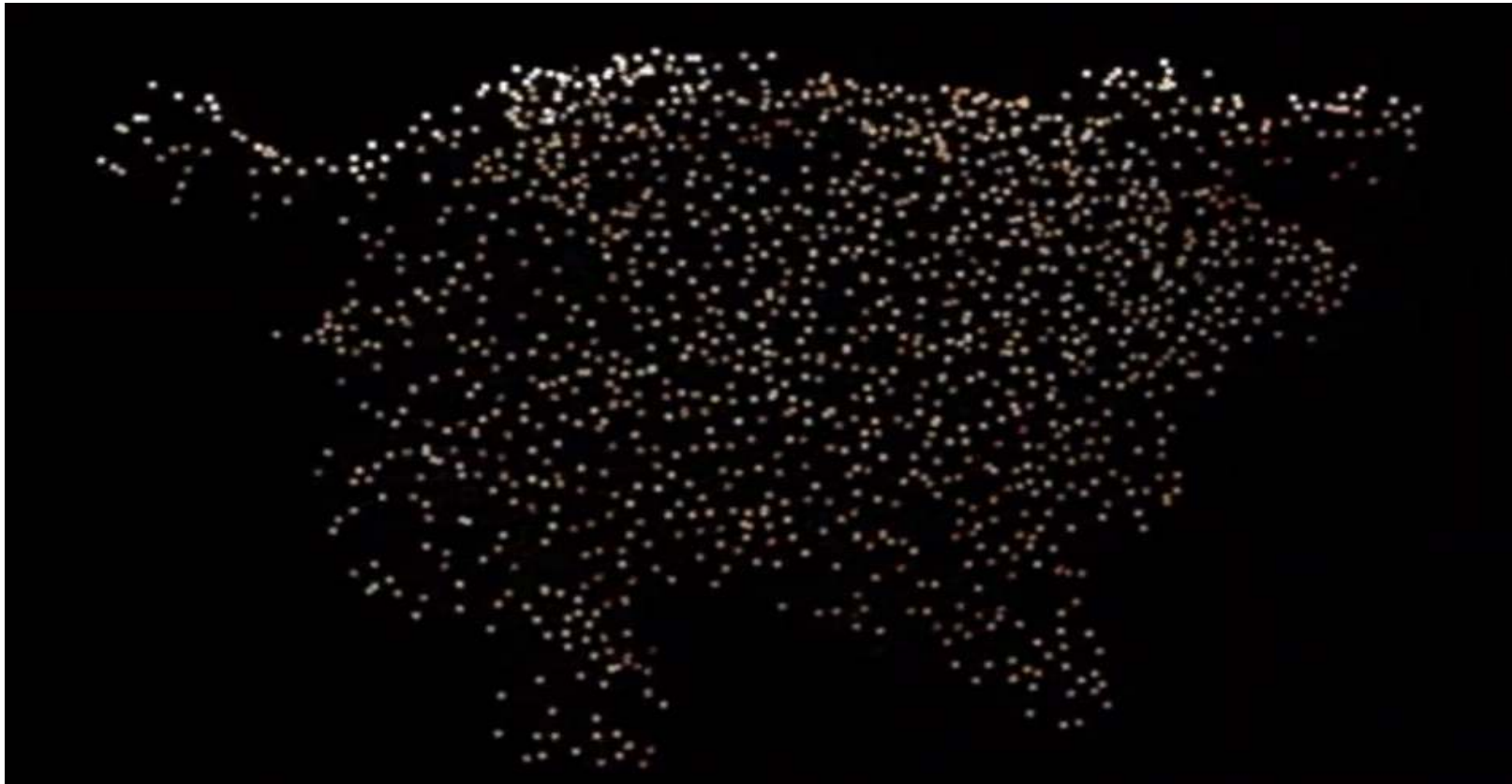
- 스캐너를 사용해서 Object를 측정하기 때문에 가려진 부분은 표현할 수 없습니다.
- 스캐너로 측정할 때 보이는 부분만 측정하므로 View에 의존하는 표현 방식입니다.

ex) 좌측 그림은 저명한 대학교수가 2년간 미켈란젤로 석상을 스캐너로 Range Image를 만드는 과정입니다.

2) Point Cloud

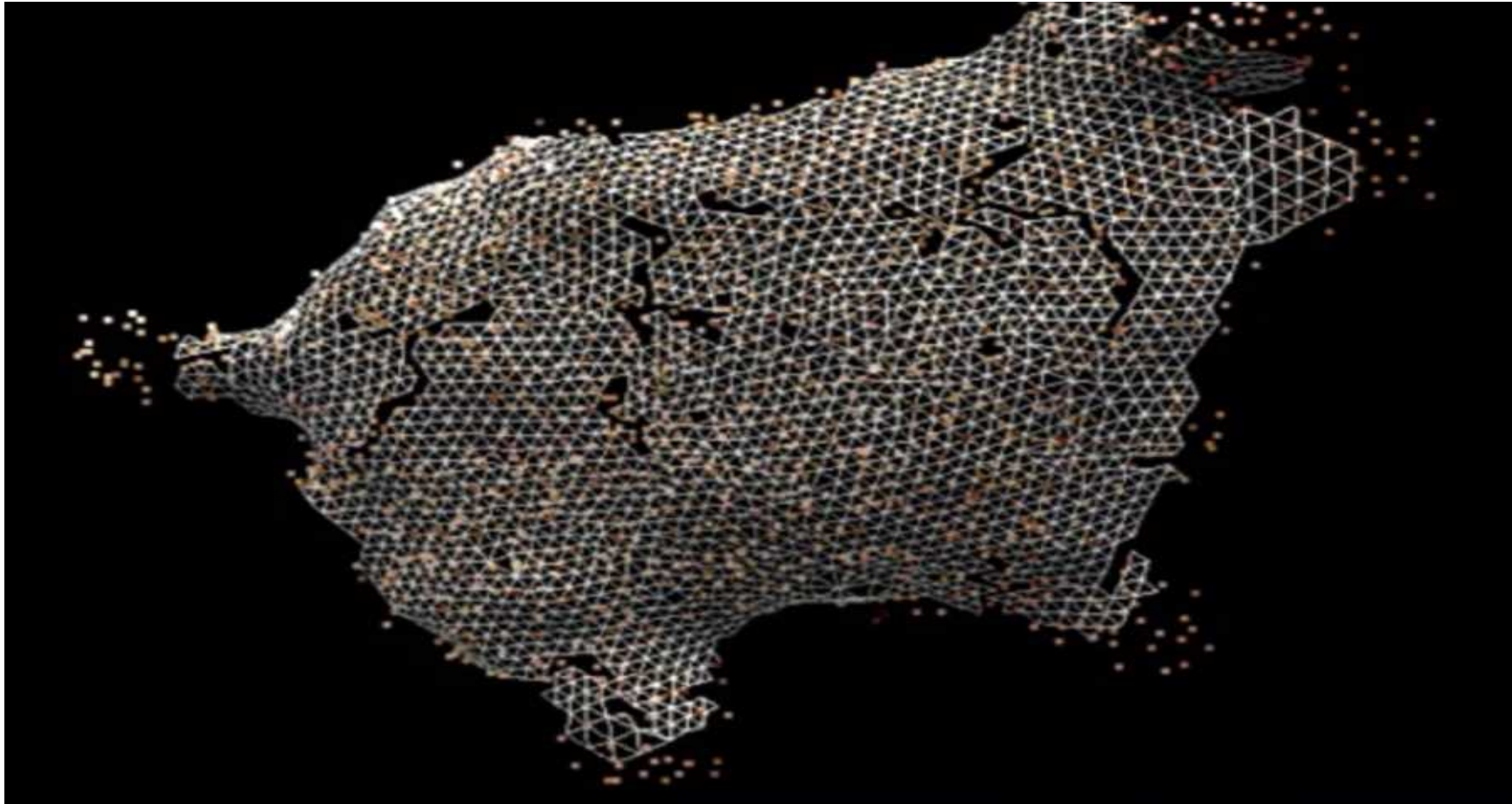
- = 3D Object의 **표면에서 임의로 수많은 점을 측정**하여 Image를 구현하는 기법입니다.
- Point Cloud를 이용해서 Triangle로 이루어진 그림을 만드는 것 같습니다.

<Example : Point Cloud 예시>



<Example : Point Cloud 이후 Triangle>

- = Point Cloud는 수많은 점들로 이루어져 있습니다. 그 점들 중에서 3개씩을 선택해서 삼각형을 만들고 , 무수한 삼각형을 Image를 구현합니다.
- 근사(Approximation) : 삼각형의 꼭짓점이 반드시 Point Cloud의 점일 필요는 없습니다. 그 점들과 근사한 위치면 됩니다.



3. Surface

1) Polygonal Mesh

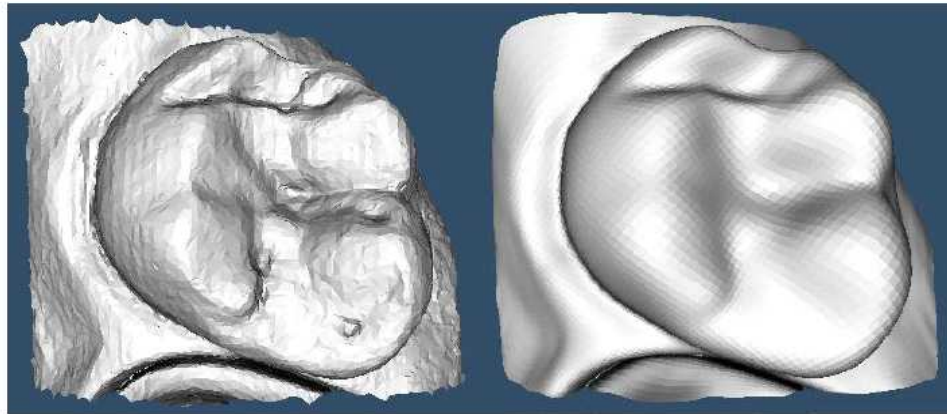
= **Triangle Mesh**가 대표적인 **Polygonal Mesh**이며 가장 많이 사용합니다.

- 3D Game에서 사용하는 대표적인 기술.
- 삼각형을 잘게 쪼갤수록 표면이 정교해지고 부드러워집니다.
- 다음 강의에서 자세하게 배웁니다.

2) Subdivision Surface

- 10주차에 자세하게 배웁니다.

3) Parametric Surface



= 장비나 부품을 만들 때 효과적 입니다.

4) Implicit Surface

= 이미 만들어진 Image를 **수학식**으로 계산하여 다시 만들 수 있습니다.

- 스캐너로 구현한 Image를 Parametric Surface 기법으로 재구성한 Image가 우측 그림입니다.

*** 여기부터는 간단하게(항목마다 30초 + 관련 영상 보여주기) 설명하시고 넘어가십시오. ***

4. Solid

1) Voxel grid

= 3D를 격자로 나눕니다.

- 물체의 밀도, 색상, 속도, 온도, 존재 유무를 표현하는 방식

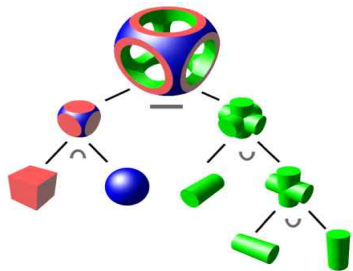
ex) CT, MRI

2) BSP Tree

= Binary Space Partition Tree

- 공간을 계속 나누어서 구현합니다.

3) CSG



= Constructive Solid Geometry

- 덩어리가 있는 기하학적 물체를 합치고 쪼개서 복잡한 물체를 구현합니다.

- 분할 정복 느낌.

4) Sweep

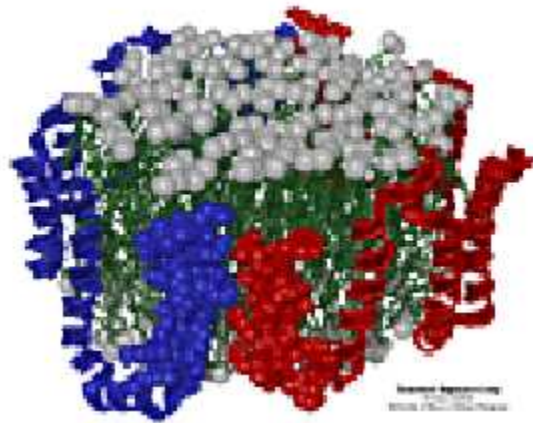
= 단면을 늘려서 물체를 구현합니다.

5. High-Level Structures

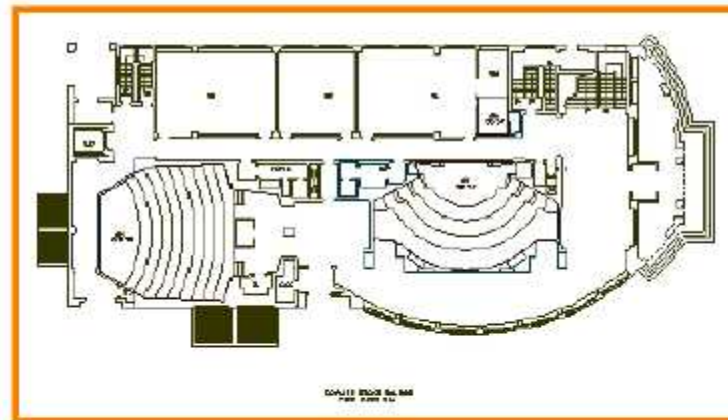
1) Scene Graph

= 방을 구성할 때 방 안에 존재하는 물체들을 계층으로 나누어서 구현.

2) Application Specific



Apo A-1
(Theoretical Biophysics Group,
University of Illinois at Urbana-Champaign)



Architectural Floorplan
(CS Building, Princeton University)

<9주차 : 대표적인 Modeling 기법_1 : Polygonal Mesh>

1. Polygon을 잘 이해하기 위한 선형대수학 지식

- 모든 개념은 3D를 기준으로 설명합니다.
- Modeling할 때 사용하는 그리기 객체를 작은 단위부터 설명합니다.

1) Point (점)

```
struct Point {  
    Coordinate x;  
    Coordinate y;  
    Coordinate z;  
};
```

 (x,y,z)

- 점의 크기는 무한대로 작습니다.
- Vertex로 이해해도 괜찮습니다.

2) Vector

```
struct Vector {  
    Coordinate dx;  
    Coordinate dy;  
    Coordinate dz;  
};
```

 (dx,dy,dz)

= 방향과 크기

- 구조체에서 크기는 저장하지 않고, 구조체의 멤버 변수로 크기를 계산할 수 있습니다.
- Point(점)를 벡터의 동족처럼 취급함으로써 벡터 공간을 확장 시켜 탄생한 공간을 '**어파인 공간**' 이라고 부릅니다.

3) 다양한 선의 개념

- ☑ 선분 : 두 점을 끝게 이은 선
- ☑ 직선 : 선분의 양쪽을 끝없이 늘린 곧은 선
- ☑ 반직선 : 한 점을 기준으로 한쪽 방향으로 끝없이 뻗어 나가는 직선

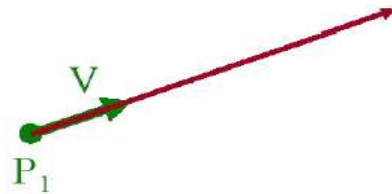
① Line Segment(선분)

```
struct Segment {  
    Point P1;  
    Point P2;  
};
```



② Ray(반직선)

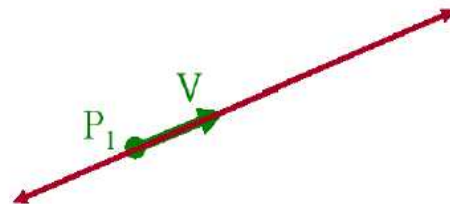
```
struct Ray {  
    Point P1;  
    Vector V;  
};
```



- 점과 벡터로 이루어져 있습니다.
- 벡터가 한 방향으로만 무한대로 뻗어나갑니다.

③ Line(직선)

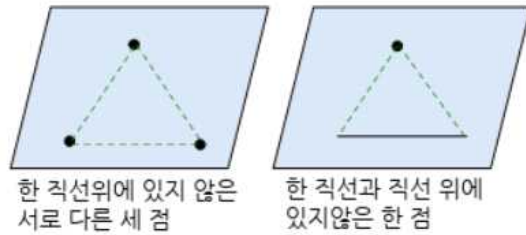
```
struct Line {  
    Point P1;  
    Vector V;  
};
```



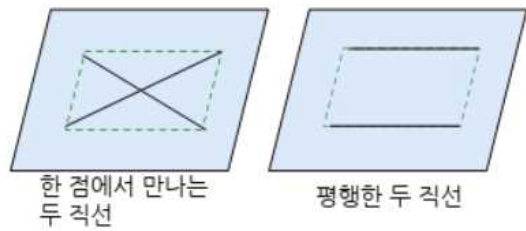
- 점과 벡터로 이루어져 있습니다.
- 벡터가 양 방향으로 무한대로 뻗어나갑니다.

4) Plane(평면)

① 평면의 결정조건 4가지

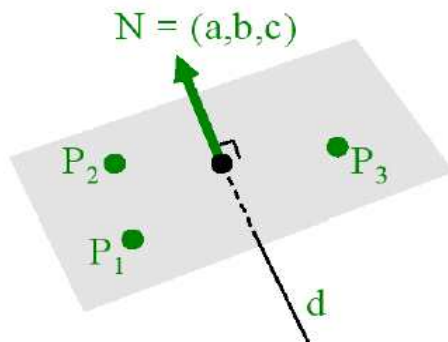


√x 수학방(mathbang.net)



② 평면

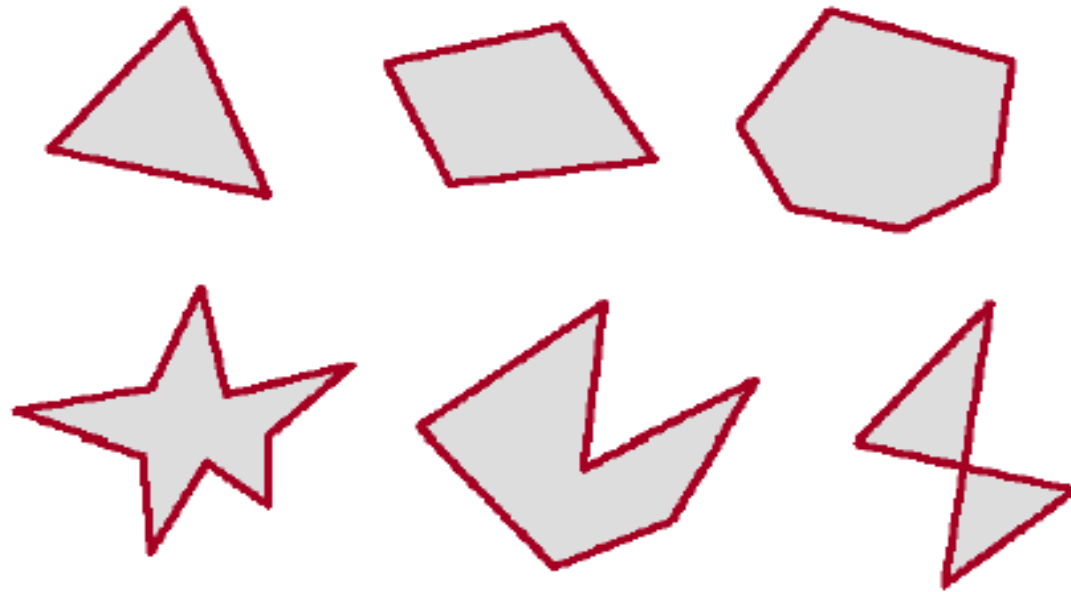
```
struct Plane {
    Vector N;
    float d;
};
```



- 점 3개가 존재합니다.
- 법선 벡터(Normal Vector) : 단위 벡터가 1이고 , 평면에 수직인 벡터
- 법선 벡터를 N 으로 표현합니다.

5) Polygon(다각형)

```
struct Polygon {  
    vector<Point> points;  
};
```



- c++ vector에 점을 저장합니다.

<Example : OpenGL 과제>

```
glBegin(GL_QUADS);
glColor3fv(colors[0]);
glVertex3fv(vertices[0]);
glVertex3fv(vertices[1]);
glVertex3fv(vertices[2]);
glVertex3fv(vertices[3]);

glColor3fv(colors[1]);
glVertex3fv(vertices[4]);
glVertex3fv(vertices[6]);
glVertex3fv(vertices[7]);
glVertex3fv(vertices[5]);

glColor3fv(colors[2]);
glVertex3fv(vertices[7]);
glVertex3fv(vertices[3]);
glVertex3fv(vertices[2]);
glVertex3fv(vertices[6]);

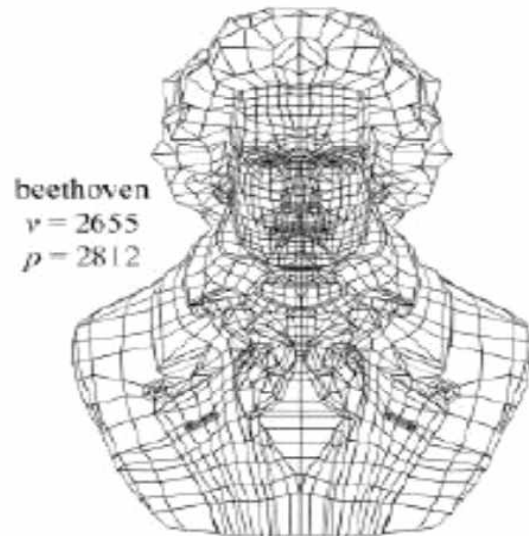
glColor3fv(colors[3]);
glVertex3fv(vertices[6]);
glVertex3fv(vertices[4]);
glVertex3fv(vertices[0]);
glVertex3fv(vertices[2]);

glColor3fv(colors[4]);
glVertex3fv(vertices[4]);
glVertex3fv(vertices[5]);
glVertex3fv(vertices[1]);
glVertex3fv(vertices[0]);

glColor3fv(colors[5]);
glVertex3fv(vertices[7]);
glVertex3fv(vertices[5]);
glVertex3fv(vertices[1]);
glVertex3fv(vertices[3]);
glEnd();
```

- GL_QUADS : Vertex(정점) 4개로 사각형을 그립니다.
- 정점 4개로 1개의 사각형을 만들고 , 6번을 반복해 6개의 사각형을 만들고 , 이를 통해 정육면체를 형성했습니다.
- 좌측 그림에서는 알 수 없지만 과제에서 모든 정점의 거리를 같게 설정했기 때문에 직육면체가 아닌 정육면체로 Modeling 됩니다.

2. Mesh (=Polygon Mesh =Polygon =Surface Mesh)



- V = Vertex = 점
- P = Polygon = 평면 다각형
- 좌측의 3D Image는 2655개의 점과 2812개의 다각형으로 이루어져 있습니다.

= 수많은 Polygon을 그물(mesh)형태로 엮어서 하나의 Image를 구현하는 방식

- 물체를 표현하는 polygon 하나하나를 mesh라고 부릅니다.
- 구성요소 : point(점), edge(모서리), face(면)
- Image를 표현하는 방법이 단순하고 그래픽 카드의 가속을 받아서 렌더링이 가능하고 , 많은 H/W Tool의 지원을 받기가 쉽기 때문에 Polygonal Mesh를 많이 사용합니다.
- 삼각형을 mesh로 사용하는 Triangular Mesh는 사각형을 mesh로 사용하는 Rectangular Mesh 보다 성능이 뛰어납니다. 그러므로 정밀한 Rendering을 요할 때는 반드시 Triangular Mesh를 사용해야 합니다.
- 복잡한 물체는 매우 많은 다각형으로 표현하며 , 다각형의 크기가 작아질수록 실제 물체와 비슷해집니다.
- 매우 많은 다각형으로 표현하면 그래픽 처리량이 늘어나고 그에 따른 연산 시간도 증가합니다.
- 정점을 조합하여 polygon mesh를 만들고 , polygon mesh를 조합하여 물체(ex : 베토벤)를 만들고 , 물체를 조합하여 Scene을 구성합니다.

3. Polygonal Mesh Acquisition

= Polygonal Mesh를 얻는 방식

- 교수님께서 거의 1시간 동안 관련 영상을 보여주시는 방식으로 강의를 진행하셨습니다.

Polygonal Mesh Acquisition

Interactive modeling

- Polygon editors
- Interchange formats

Scanners

- Laser range scanners
- Geological survey
- CAT, MRI, etc. (isosurfaces)

Simulations

- Physical processes



Digital Michelangelo Project
Stanford

1) Interactive Modeling

- 사용자가 직접 제작

ex) Google Building Maker , Blender

2) Scanner

① Laser range scanners : 스캐너로 측정

② Geological survey : 지도를 그리기 위해 실제 지형 측정

3) Simulation

= 알고리즘

<10주차_1 : 대표적인 Modeling 기법_2 : Subdivision Surface>

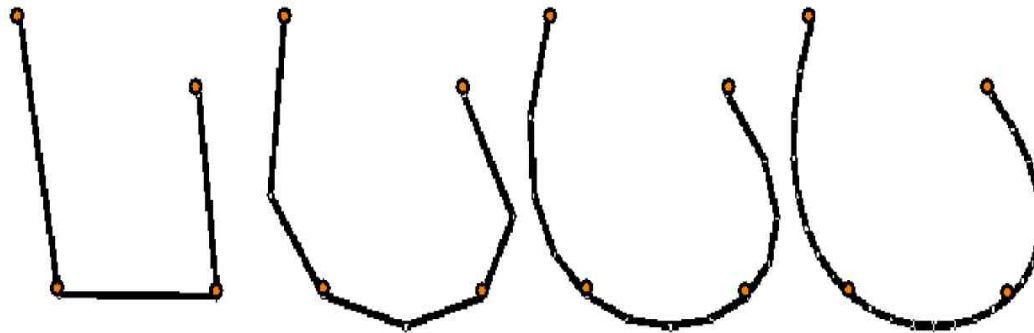
1. Continuity

= 끊이지 않고 계속되거나 지속되는 것

- 제어점 왼쪽으로 들어오는(Incoming) 곡선의 기울기가 제어점에서 나가는(Outgoing) 기울기와 같을 때를 1차 연속(=C1 Continuity)라고 합니다. 이를 수학적으로 정의하면 미분 계수(접선의 기울기)로 정의할 수 있습니다.
- 함수의 연속과 비슷하게 생각해도 괜찮을 듯 합니다.

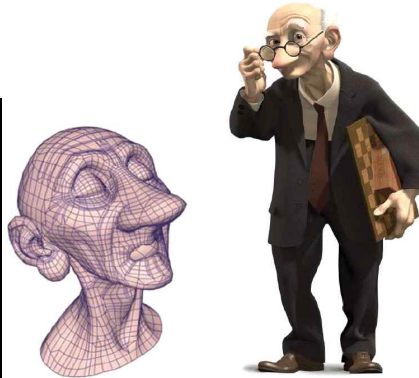
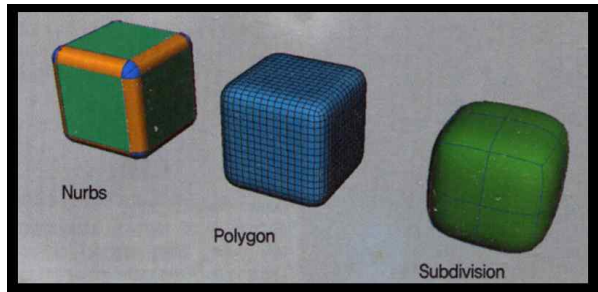
2. Subdivision을 이해하기 위한 굵음

- 선과 관련된 굵음 : Curve(=곡선)
- 면과 관련된 굵음 : Surface(=곡면)



- 1번 그림에서 curve는 존재하지 않습니다.
- 4번 그림에서 curve는 존재합니다.
- Subdivision은 3D 물체를 modeling할 때 curve를 사용합니다.

3. Subdivision의 정의



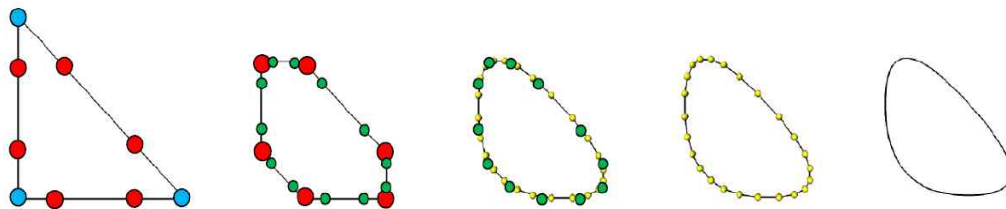
- Pixar에서 만든 Geri's Game에서 사용된 Subdivision

= 3D 물체의 곡면(=2차원의 굽은 기하학적 모양 =curve & surface)을 표현하는 방식

- Polygonal Mesh는 물체를 수많은 Polygon을 그물형태로 엮어서 하나의 Image를 구현하는 방식입니다. Subdivision 방식 또한 이와 비슷하며 Polygonal Mesh의 발전된 버전입니다. Polygonal Mesh 보다 적은 수의 mesh를 사용하고자도 같거나 높은 품질의 3D Modeling이 가능합니다.
- Subdivision은 물체를 수많은 곡면을 그물형태로 엮어서 하나의 Image를 구현하는 방식입니다.
- Subdivision이 적은 mesh를 사용해서 복잡한 방식인 Nurbs와 비슷한 수준의 품질을 만듭니다.

4. Subdivision Curve 구현 방법

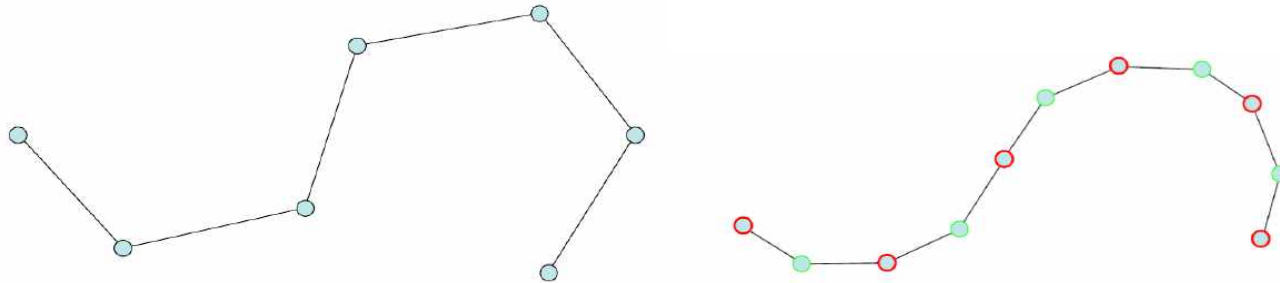
1) Corner Cutting



- 모서리의 1/4지점과 3/4지점에 점을 찍고 연결하고 , 이전에 찍은 점을 지웁니다.
- 위의 과정을 반복합니다.

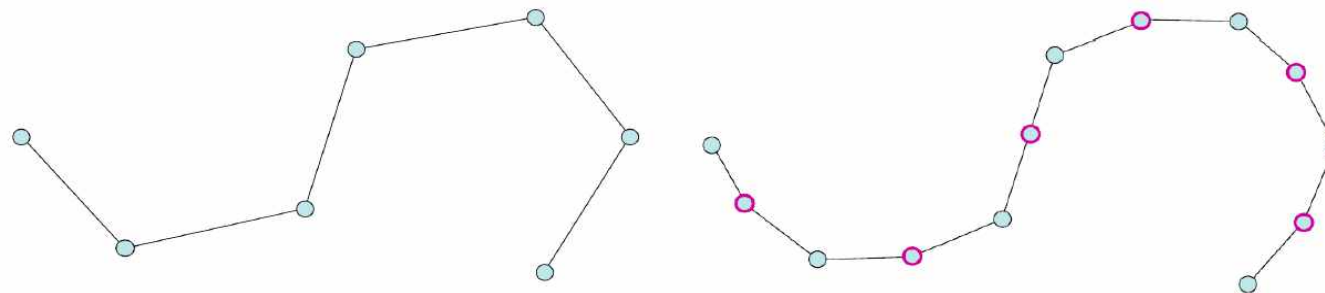
2) Interpolating (보간법)

- 점이 4개 있습니다. (P_i , P_{i+1} , P_{i+2} , P_{i+3})
- 네 점을 지나는 curve를 fitting 합니다.
- 개발자가 점을 찍어 놓으면 **찍어 놓은 점을 유지하며 Interpolating을 진행**하면서 새로운 점을 만들어냅니다.



3) Approximating

- 개발자가 점을 찍어 놓으면 **찍어 놓은 점을 비슷하게 보존하되 위치를 약간 조정하며 Approximating을 진행**하면서 새로운 점을 만들어냅니다.
- 아주 매끈한 곡선을 만들어 내는 방식입니다.

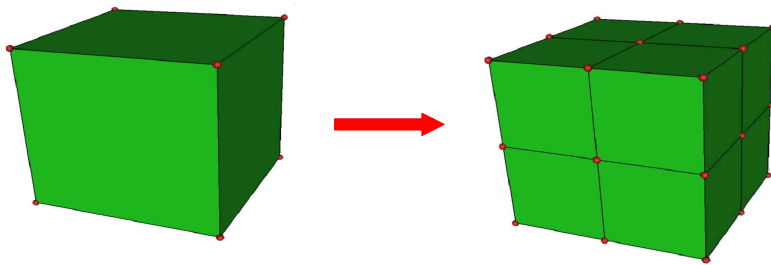


5. 대표적인 Subdivision_1 : Linear Subdivision

- 기본적으로 Quad-Mesh(사각형)들을 이용해서 3D-Model을 만듭니다.
- 먼저 Topology Refinement Rule을 하고 , 그 후에 Geometry Refinement Rule을 합니다. 이를 주어진 횟수만큼 반복하면 3D 물체가 직선형 모양에서 곡선형 모양으로 바뀝니다.

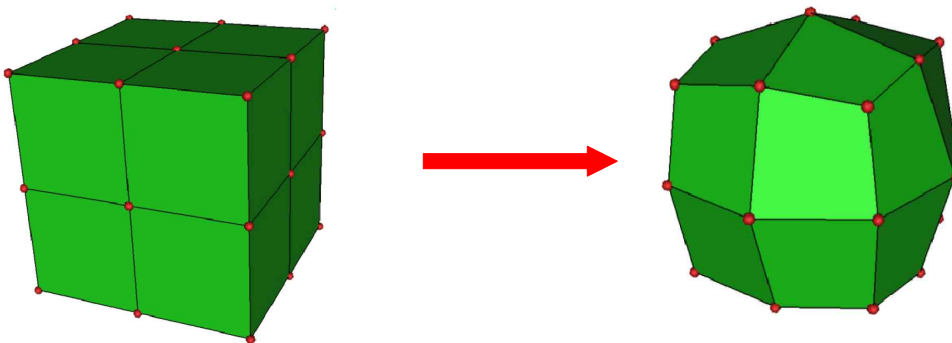
ex) (T -> G) -> (T -> G) -> (T -> G) : 3번의 Linear Subdivision을 수행합니다.

1) Topology Refinement Rule



- = Vertex(꼭짓점)의 위치를 고정하고 , 사각형을 쪼갭니다.
- 쪼개면 Vertex가 더 생깁니다.
- Vertex를 찍는 위치는 면의 무게중심 & 모서리의 중점

2) Geometry Refinement Rule

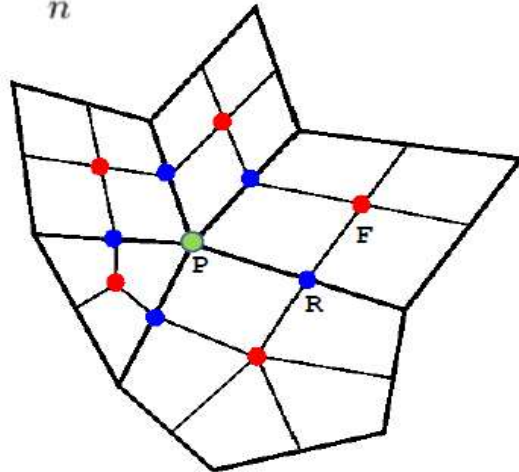


- = Vertex의 위치를 조정합니다.
- 3D 물체에서 꼭짓점을 하나 지정합니다. 그 후 해당 꼭짓점을 포함하는 면의 무게중심들을 모아서 평균을 냅니다. 처음에 지정한 꼭짓점을 무게중심들의 평균 위치의 점으로 이동시킵니다.
- Geometry Refinement Rule 과정을 반복하면 꼭짓점들이 3D 물체의 안쪽으로 점점 들어갑니다.

6. 대표적인 Subdivision_2 : Catmull-Clark Subdivision

- 영화에서 사용하는 굉장히 부드럽게 곡면을 표현하는 방식

$$\text{New } \bullet = \frac{F + 2R + (n - 3)P}{n}$$



① F 구하기

= 모든 면(네모일 필요는 없음)에 대해서 면에 대한 무게중심

- 'face-point' 라고 부릅니다.

② R 구하기

= 특정 모서리에 대해서 맞닿은 면에 존재하는 face-point와 그 모서리의 양 끝점을 모아서 평균을 낸 점

③ P 구하기

- 분모 n은 F의 개수 + R의 개수 + P의 개수(=1)이며, 그림에서의 n은 11입니다.

- 분자는 위의 식처럼 가중치를 줘서 계산합니다. 'F + 2R + (n-3)P'

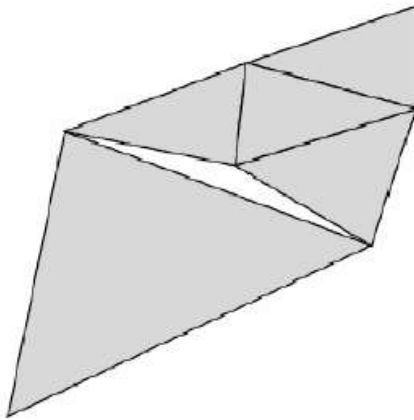
7. 대표적인 Subdivision_3 : Loop Subdivision

- 삼각형 Mesh에서만 동작합니다.
- 흰 점 : even vertex
- 검은 점 : odd vertex

8. 대표적인 Subdivision_4 : Adaptive Subdivision

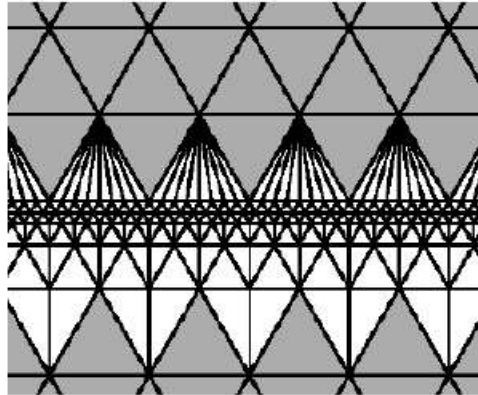
- Subdivision을 하면 삼각형이 많이 들어가서 메모리를 많이 사용하게 됩니다.
- 기준을 정해 **특정 지역에만 Subdivision을 많이 합니다.**

1) 문제점

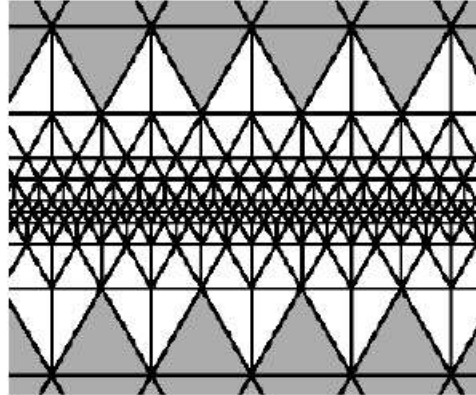


- 특정 지역만 Subdivision을 간극이 발생해 틈이 생깁니다.

2) 해결책 : Triangle Fan (Balanced Subdivision)



Unbalanced



Balanced

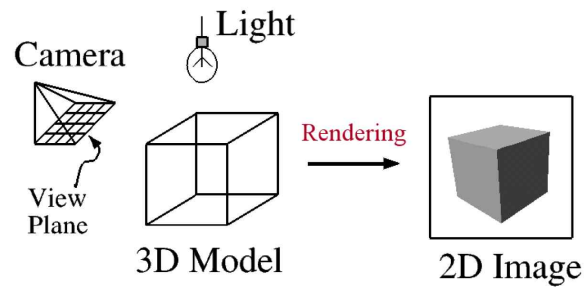
[Kobbelt 2000]

- 임의의 Mesh와 인접한 Mesh는 Subdivision level이 한 단계 까지만 다르게 해야 하며 , 두 단계이상 다르지 않게 하는 기법입니다.

<10주차_2 : Rendering>

1. Rendering 정의

Construct image from 3D model



= Constructing 2D images from 3D models

= Modeling에 의해 정의된 물체를 어떻게 그릴 것인지에 관련된 것

= 컴퓨터가 3D Model에서 물체의 정보를 얻어 최종 결과를 계산(광원과 카메라의 상호작용)하여 2D image를 형성하는 기술.

- 개발자가 화면에서 보는 모든 그림은 Rendering의 결과입니다.

- 화면에 2D Image를 구현하는 과정에서 가장 최종의 단계입니다.

ex) 조명 , Viewer의 위치 , 물체 표면의 재질

2. Rendering Scenario

1) Interactive (=Realtime)

- = 주어진 시간에서 가능한 범위 내에서 최대의 화질을 추구하는 렌더링 방식
- 게임에서 주로 사용합니다.

2) Offline (=Non-Realtime =Photo-Realism)

- = 시간이 얼마나 걸리는지 무시하고 , 무조건 화질을 최상으로 추구하는 렌더링 방식(영화)
- 영화에서 주로 사용합니다.
- 교수님께서 보여주신 예시를 보면 사진과 차이를 느낄 수 없습니다. 그러나 현대의 기술로는 사람을 사진처럼 만들기는 거의 불가능하기 때문에 해당 사진들에서는 사람이 존재하지 않았습니다.

3. Rendering 에서 고려해야 할 사항

What issues must be addressed
by a 3D rendering system?

- Camera
- Visible surface determinaton
- Lights
- Reflectance
- Shadows
- Indirect illumination
- Sampling
- etc.

<이해를 돕기 위한 나의 생각>

- 미술에서 3D 도형을 도화지에 그릴 때 , 화가가 고려할 점을 생각해봅시다.
- Unity에서 3D 게임을 구현하는 경우에도 , 결국에는 2D인 화면에 3D Model들을 나타내는 렌더링 작업이므로 이때 개발자가 고려할 점을 생각해봅시다.

1) Camera Models

Position

- Eye position (p_x, p_y, p_z)

Orientation

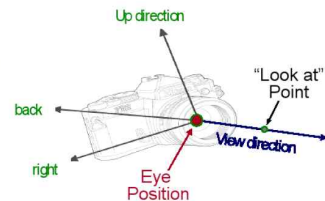
- View direction (d_x, d_y, d_z) or "look at" point
- Up direction (u_x, u_y, u_z)

Coverage

- Field of view (fov_x, fov_y)

Resolution

- In x and y



<카메라 구현을 위한 4가지 매개변수>

- ① Position = 카메라의 위치
- ② Orientation = 카메라가 보는 방향
- ③ Coverage = 카메라가 볼 수 있는 범위
- ④ Resolution = 해상도

2) Visible Surface Determination

= How find visible surfaces?

= Viewer가 물체를 볼 때 , 보이는 부분과 보이지 않는 부분을 구분하는 방법에 대해서 고려해야 합니다.

① Ray Tracing

= (다음 강의에서 자세하게 배움)

- offline 시나리오에서 사용됩니다.

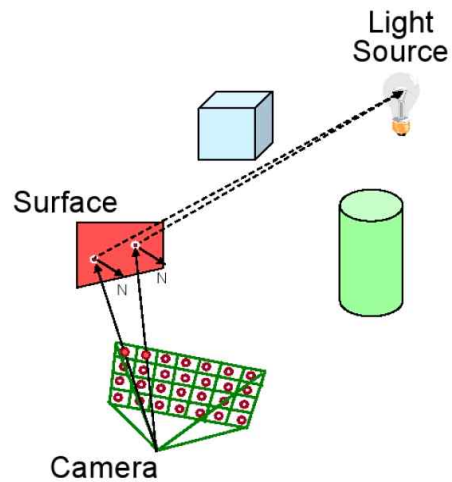
- ray casting이라는 열화 버전이 존재합니다. Ray casting은 화면의 특정 영역에 pixel 마다 ray를 쏘서 최초로 만나는 물체의 정보를 얻어내는 기술입니다.

② Rasterization

= View의 위치와 물체의 거리에 따른 색. (자세히 모름)

- Interactive 시나리오에서 사용됩니다.

3) Lighting Simulation (광원과 반사)



광원 [light source / source of light, 光源]

12월 02, 2016

빛을 내는 물체의 총칭. 태양을 비롯해서 반딧불 같은 생물, 양초, 백열전구, 레이저 발광체에 이르기까지 다양한 것들이 모두 포함된다. 일반적으로 광원이라고 할 때는 인공적으로 만들어진 것을 가리키는 경우가 많다. 태양, 전구 등과 같이 스스로 빛을 내는 발광체의 광원을 1차 광원이라 하고, 반사 또는 투과하여 빛을 발하는 발광체의 광원을 2차 광원이라 한다.

<광원에서 나오는 빛과 물체의 상호작용>

- ① Light source emission : 광원에서 나오는 빛
- ② Surface reflectance : 물체의 표면에서 일어나는 빛 반사
- ③ Atmospheric Attenuation : 밝고 어두움의 정도
- ④ Camera response : 카메라의 반응

4) Shadow

= 광원이 빛을 물체에 발사하면 물체가 빛을 가리게 되고, 그 결과 그림자가 생성됩니다.

- 광원이 여러 개일 때 생기는 그림자에 대해서도 고려해야 합니다. (Soft Shadow)

5) Indirect illumination (간접 조명)

- 영상을 보여주셨습니다.

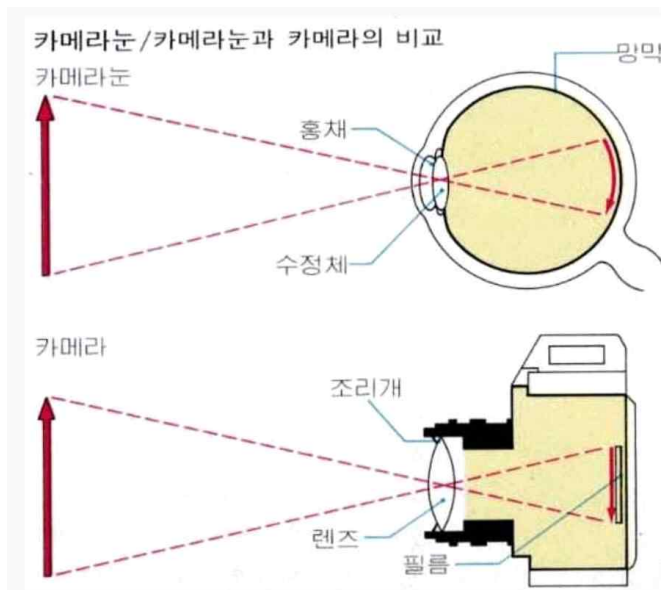
6) Sampling

= 연속적인 실세계를 이산적인 Image로 나타내는 방식입니다.

4. 사람이 물체를 보는 원리

✓ 우리가 물체를 보기 위해 필요한 것은 무엇일까? 일단, 볼 수 있는 대상인 물체, 감각을 받아들일 수 있는 눈이 있어야 할 것이다. 그리고 눈과 물체 사이를 이어 주는 무언가가 필요한데, 그 무언가가 바로 빛이다.

- 빛이 존재하지 않으면 물체를 볼 수 없습니다.
- 사람의 눈은 빛의 밝기와 색깔을 인식합니다.
- 빨간색 사과는 빛의 3원색 중에서 빨간색만 반사하고 , 초록색과 파란색은 흡수하기 때문에 사람이 볼 때 빨간색입니다.



① 사람의 눈

= 물체에서 반사된 빛은 홍채에 의해 조절되어 눈으로 들어오고 , 이 빛이 수정체를 통과하면서 굴절되어 망막에 도달합니다. 이때 망막에 맺히는 물체의 상은 실제와는 달리 거꾸로 맺히고 , 그게 전기 신호로 변환되어 뇌로 전달되고 , 우리 뇌가 재해석해서 물체를 정확하게 인식합니다.

② 카메라

= 물체에서 반사된 빛은 카메라의 볼록 렌즈(=사람의 수정체)를 통과하면서 굴절되어 필름(=사람의 망막)에 도달합니다. 이때 필름에 맺히는 물체의 상은 실제와는 달리 거꾸로 맺힙니다.

5. Path Types

= 광원에서 발사한 빛이 그 빛을 보는 사람(Viewer)까지 도달하는 과정의 종류

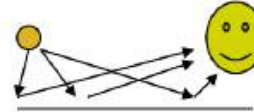
- LE

- The light goes straight from the source to the viewer



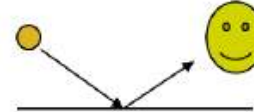
- LDE

- The light goes from the light to a diffuse surface that the viewer can see



- LSE

- The light is reflected off a mirror into the viewer's eyes



① LE

= 빛이 Viewer에게 바로 도착.

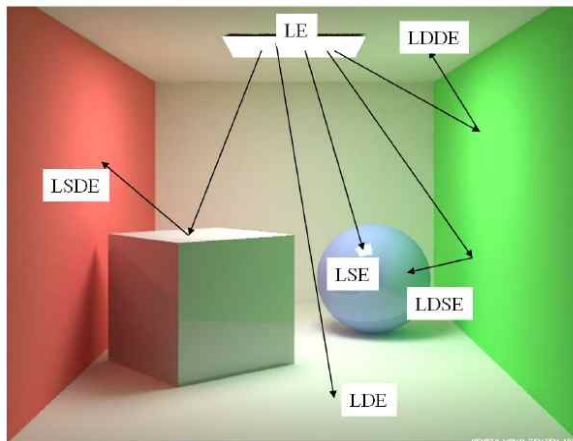
② LDE

= 빛이 불규칙한 면에 반사되어 Viewer에게 도착.

- OpenGL에서는 LDE 방식만 사용합니다.

③ LSE

= 빛이 거울에 반사되어 Viewer에게 도착.



- LSE : 빛 -> 파란 구형 거울 -> Viewer

- LDE : 빛 -> 울퉁불퉁한 바닥(현미경으로 보면 불규칙적인 면) -> Viewer

- LDDE : 빛 -> 거울 -> Viewer

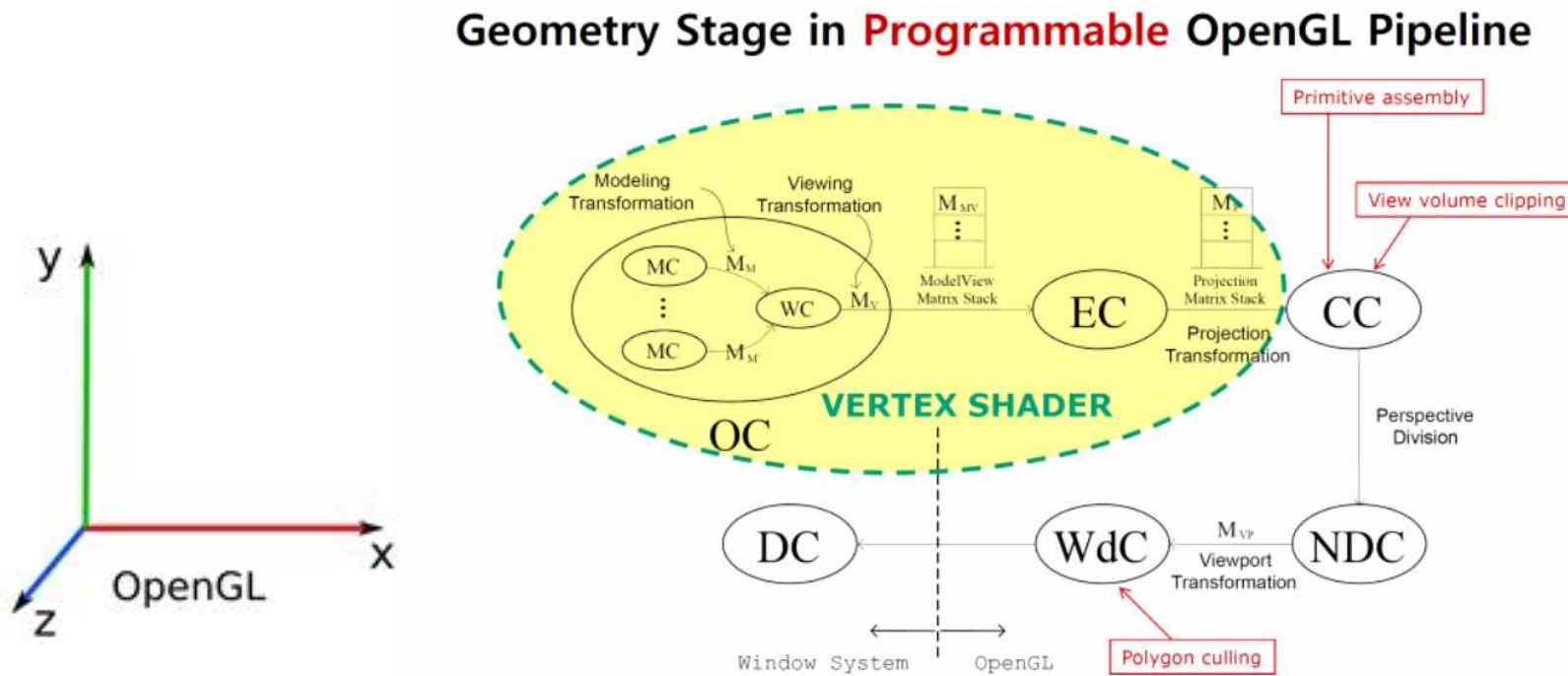
- LD(S|T)*E : 더욱 정교한 path Types

<10주차_3 : Rendering Pipeline 강의 및 기초>

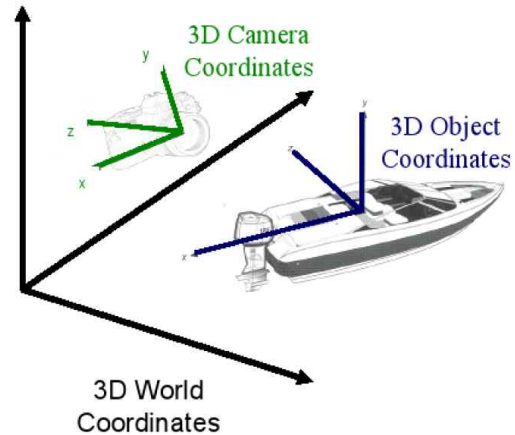
*** 강의에서 다룬 Rendering Pipeline의 기초 내용입니다. ***

*** Rendering Pipeline = Graphic(s) Pipeline ***

1. Coordinate (직교 좌표계)



1) 3D Coordinate



① Model Coordinate (=모델 좌표계 =지역 좌표계 =MC =Object Coordinate)

= 3D 공간에 존재하는 Model들의 개별 좌표계입니다.

- 좌표를 모델 1개에 대해서만 설정합니다.

ex) 사람모형의 3D model을 만들면 , 사람을 기준으로 사람의 팔이 어디에 배치되었고 , 머리는 어디에 배치되었는지에 대한 좌표.

② World Coordinate (=월드 좌표계 =전역 좌표계 =WC)

= 3D 공간에서 3D Model이 어디에 존재하는지에 대한 좌표계입니다.

ex) 사람 오브젝트에서 모델 좌표계에서 손의 위치가 (-10,0,10)이고 , 사람 오브젝트가 월드 좌표계에서 (100,100,100)에 위치한다고 가정합니다.

사람의 손은 월드 좌표계에서 (90,100,110)에 위치합니다.

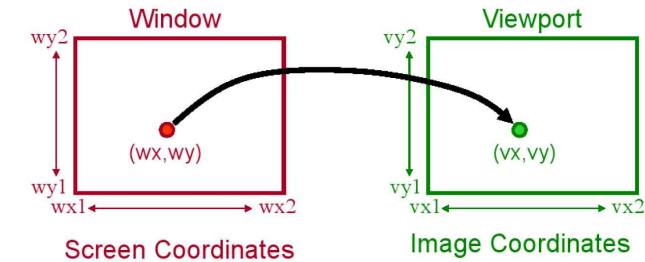
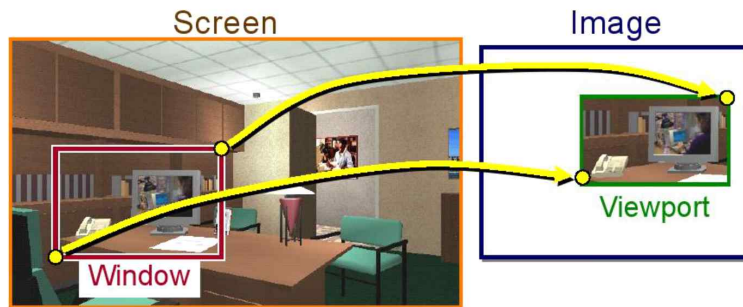
③ Camera Coordinate (=시점 좌표계 =카메라 좌표계 = CC = Eye Coordinate = WC = View Space)

= 카메라의 위치를 기준으로 하는 좌표계입니다.

- World Coordinate에서 세상에서 물체를 배치시키는 과정이라면 , Camera Coordinate는 만들어진 세상을 카메라로 바라보고 그려내는 과정입니다.

2) 2D Coordinate

- Transform 2D geometric primitives from screen coordinate system (normalized device coordinates) to image coordinate system (pixels)

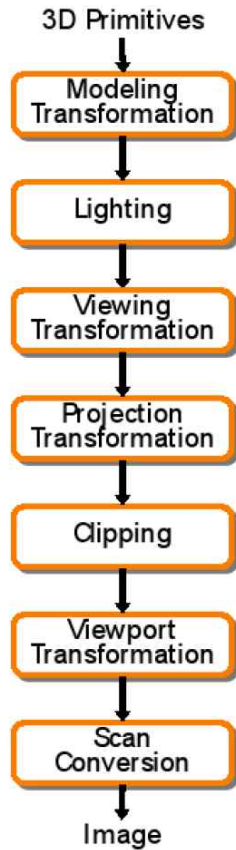


$$\begin{aligned} vx &= vx1 + (wx - wx1) * (vx2 - vx1) / (wx2 - wx1); \\ vy &= vy1 + (wy - wy1) * (vy2 - vy1) / (wy2 - wy1); \end{aligned}$$

① Screen Coordinate (=화면 좌표계 =정규 디바이스 좌표계 =Normalized Device Coordinate)

② Image Coordinate (=이미지 좌표계)

2. 3D Rendering Pipeline

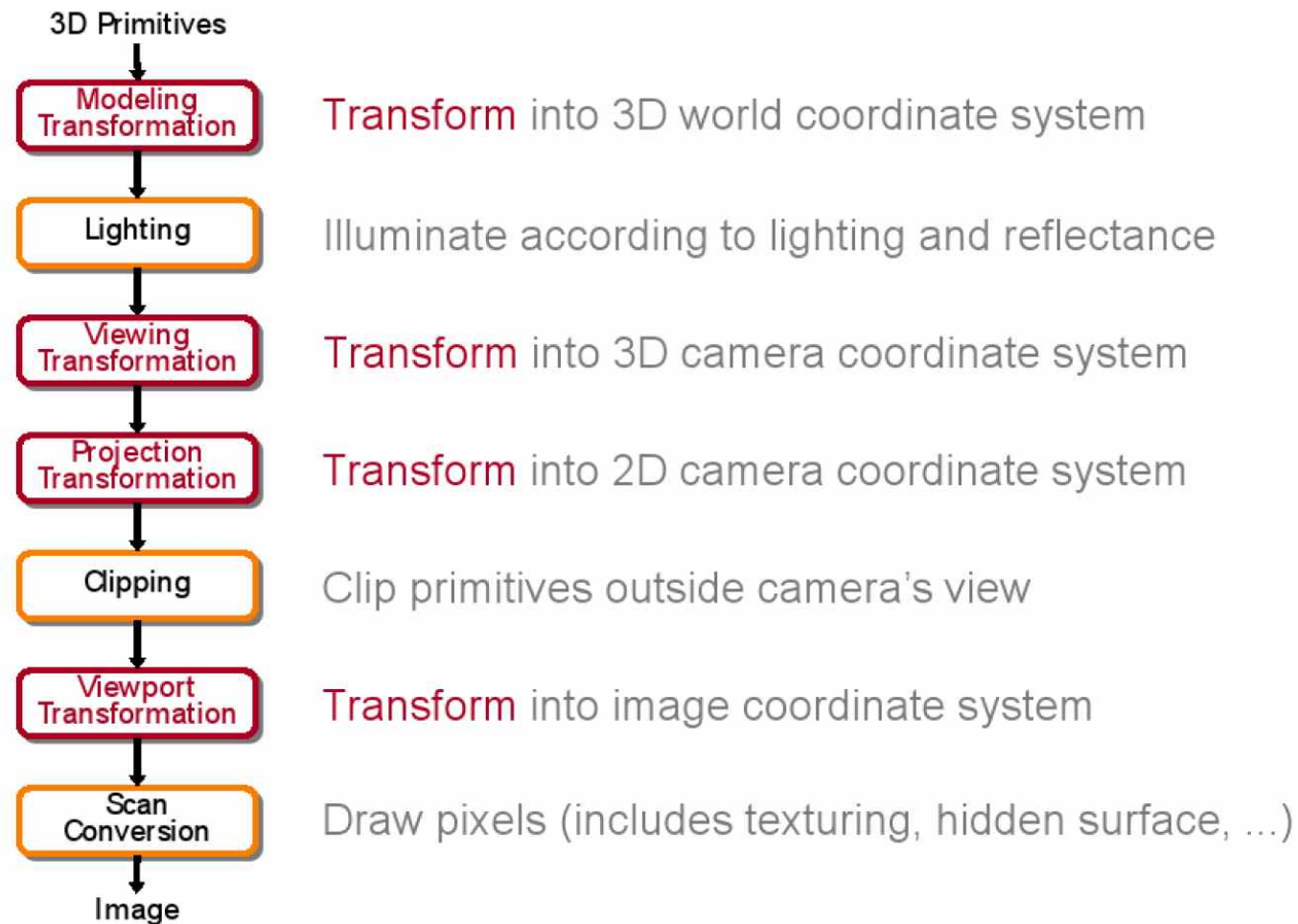


= 3D model을 7단계를 거쳐서 2D 화면의 이미지로 만드는(렌더링) 과정입니다.

- Graphics Pipeline 이라고도 불립니다.
- 3차원 이미지를 2차원 래스터 이미지로 표현하기 위한 단계적인 방법입니다.
- OpenGL과 Direct3D가 두 개의 3차원 그래픽 표준으로, 매우 비슷한 그래픽 파이프라인을 기술합니다.

<Raster>

- 13주차 강의에서 자세하게 배웁니다.
- 컴퓨터에서 화상 정보를 표현하는 한 가지 방법으로 이미지를 2차원 배열 형태의 픽셀로 구성하고 이 점들의 모습을 조합, 일정한 간격의 픽셀들로 하나의 화상 정보를 표현하는 것을 말합니다. 즉, 한 줄에서 연속된 픽셀들의 집합
- 13주차의 강의지만 , Rendering이므로 이어서 필기를 하는 것이 바람직하다고 생각하여 옮겼습니다.



- Pipeline에서 물체의 좌표는 '모델 좌표계 -> 월드 좌표계 -> 카메라 좌표계 -> 이미지 좌표계' 순서로 좌표가 변합니다.

1) Modeling Transformation (=모델 변환)

- = 3D 모델에 기하 변환(Transform, Rotation, Scale 등)을 가하여 3D 모델의 모습을 변환하는 작업
- 3D 모델마다 개인의 모델 좌표계를 가지고 있으므로 모델마다 원점이 다릅니다.
- Model Coordinate -> World Coordinate 변환

2) Lighting

- 12주차에서 배운 광원과 빛.

3) Viewing Transformation (=시점 변환)

- = 월드 좌표 기준으로 그려진 모델을 카메라의 시점 기준으로 그려진 모델로 변환하는 작업
- 카메라가 어떻게 보는지에 따라서 3D 모델이 어떻게 보이는지 달라집니다.
- World Coordinate -> Camera Coordinate 변환
- 역행렬을 이용하면 역연산도 할 수 있습니다.

<Projection>

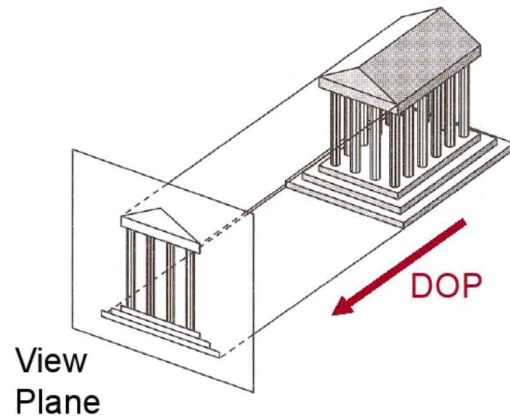
Projection

- General definition:
 - Transform points in n -space to m -space ($m < n$)
- In computer graphics:
 - Map 3D camera coordinates to 2D screen coordinates

- 3D Camera 좌표계 -> 2D 화면 좌표계

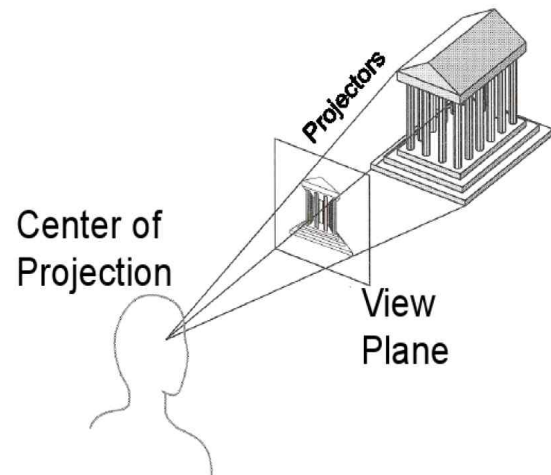
4) Projection Transformation

① Parallel Projection (평행 투상)



- = 물체의 모양에 변형이 없이 모든 점들이 투영되는 방법
- 물체의 실제 수치 측정이 중요한 작업에서 사용합니다.
- 원근감이 존재하지 않아 현실감이 떨어집니다.
- 평면도 , 입면도 , 측면도를 모두 표현하는 **Orthographic Projection**은 Parallel Projection의 한 종류입니다.

② Perspective Projection (원근 투상)



- = 시점이 물체로부터 유한한 거리에 있다고 간주하여 , 모든 투상선이 시점에서 출발하여 방사선 모양으로 퍼져가는 방법
- 거리에 비례해서 물체의 크기가 작아집니다.
- 원래 물체와 Viewer의 각도가 보존 되지 않습니다.
- 원근감이 존재해 현실감이 있습니다.
- 실제 카메라나 사람의 눈과 같은 방식입니다.

5) Clipping

- 카메라에 들어오지 않는 3D 모델의 부분은 잘라서 버립니다.
- window 안에 존재하지 않는 점은 제거 , 자른 선과 만나는 부분을 새로운 점이 생성

6) Viewport Transformation

- Scene에서 어떤 부분을 잡고 , S/W에서 특정 부분에서 2D Image를 나타냄
- 이 과정에서 Image Mapping 이용 : 6주차
- 이 과정을 거치면 최종적으로 pixel 단위의 이미지 좌표로 바뀝니다.

7) Scan Conversion

- 13주차 Rasterization에서 자세하게 배웁니다.

<Rendering Pipeline 독학 및 심화>

*** 교재 Chapter 6 & Chapter 7을 독학한 Rendering Pipeline의 심화 내용입니다. ***

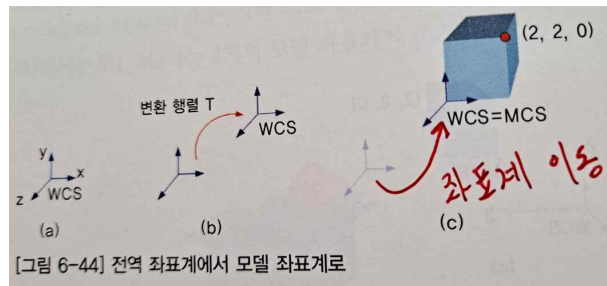
1. Modeling

1) Modeling Coordinate System (=모델 좌표계 =지역 좌표계)

= Scene에 존재하는 각각의 Object 별로 설계상의 편의를 위주로 설정된 좌표계

- OpenGL 프로그램이 최초로 실행 될 때는 기본적으로 모델 좌표계와 전역 좌표계와 시점(카메라) 좌표계가 일치되어 있습니다.
- 물체에 이동, 회전, 크기 조절 등의 변환을 가하면 모델 좌표계와 전역 좌표계가 분리 됩니다.

<Example : 정육면체 이동>



- 초기에 정육면체의 기준점(=아랫면의 좌상단 점)을 모델 좌표와 전역 좌표가 모두 (0,0,0)로 설정합니다. 이 때 그림에서 표시한 빨간 점의 모델 좌표와 전역 좌표가 모두 (2,2,0)이 됩니다. 이 물체를 (3,2,0)만큼 이동시키면 표시한 빨간 점의 전역 좌표는 (5,4,0)으로 변경되지만 모델 좌표는 (2,2,0)을 유지해야 합니다. 정육면체의 기준점을 기준으로 표시한 빨간 점의 모델 좌표는 (2,2,0)이 유지되어야 하고 좌표가 바뀌면 정육면체 모양 자체가 바뀝니다. 그러므로 전역 좌표계를 이동시켜 모델 좌표계와 전역 좌표계를 다시 동일하게 만들어서 모델 좌표를 (2,2,0)로 유지시켜야 합니다.

- 이런 과정이 결국 OpenGL이 translate를 구현할 때 '물체의 이동'으로 구현하지 않고 '좌표계의 이동'으로 구현하는 방식입니다. 이게 제가 과제를 할 때 glTranslatef()를 하면 물체가 이동한다는 느낌이 아닌 물체를 그리는 붓(붓이 결국 좌표계)을 이동시키는 느낌을 받은 것 같습니다.

- 교재 p264

2) Modeling Transformation (=모델 변환)

3) 관련 OpenGL 함수

- ① glMatrixMode(상수)
- ② glLoadIdentity()

2. Viewing

1) View Coordinate System (=월드 좌표계 =전역 좌표계)

= 동일한 위치를 서로 다른 좌표로 부르지 않기 제작된 , 장면 안의 모든 물체를 한꺼번에 일률적으로 아우를 수 있는 기준 좌표계

2) View Transformation OR Viewpoint Transformation (=시점 변환)

= 변환된 물체를 관찰하기 위해 카메라를 이동하고 회전함으로써 카메라 위치와 방향을 설정하는 작업

3. Projection Transformation (=투상 변환)

= 카메라의 렌즈를 선택하고 촬영하여 물체의 2D 영상을 필름에 맺히게 하는 작업

4. Viewport Transformation (=뷰 포트 변환)

<Viewport>

= 현재 화면에 보여 지고 있는 영역

- 기기 별로 뷰포트의 크기가 다르기 때문에 , 동일한 이미지라도 기기에 따른 배율 조정이 발생해 화면의 크기가 다르게 보이는 현상이 나타납니다.

<Example : OpenGL에서 Window 크기에 따라서 Viewport의 크기를 변화시켜 봅시다.>

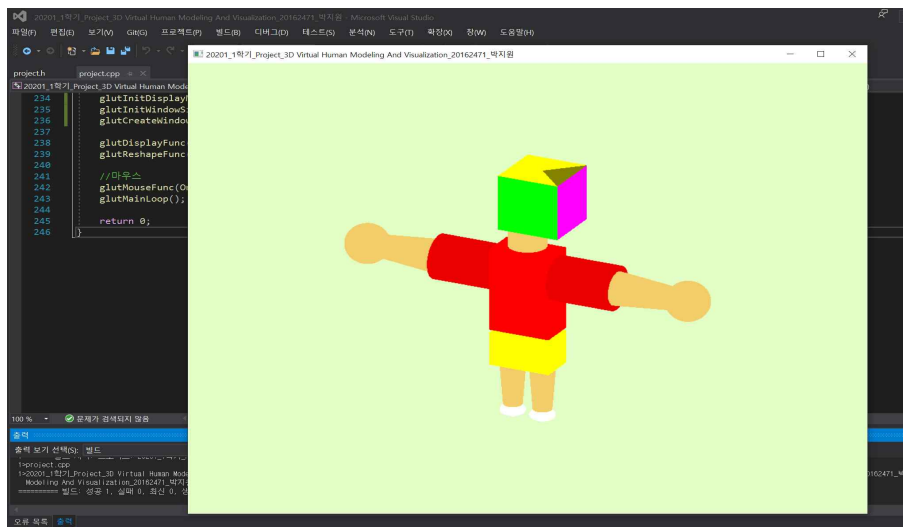
```
#define ROW 1024  
#define COL 768
```

```
glutInitWindowSize(ROW, COL);
```

- 아마도 WindowSize = ScreenSize

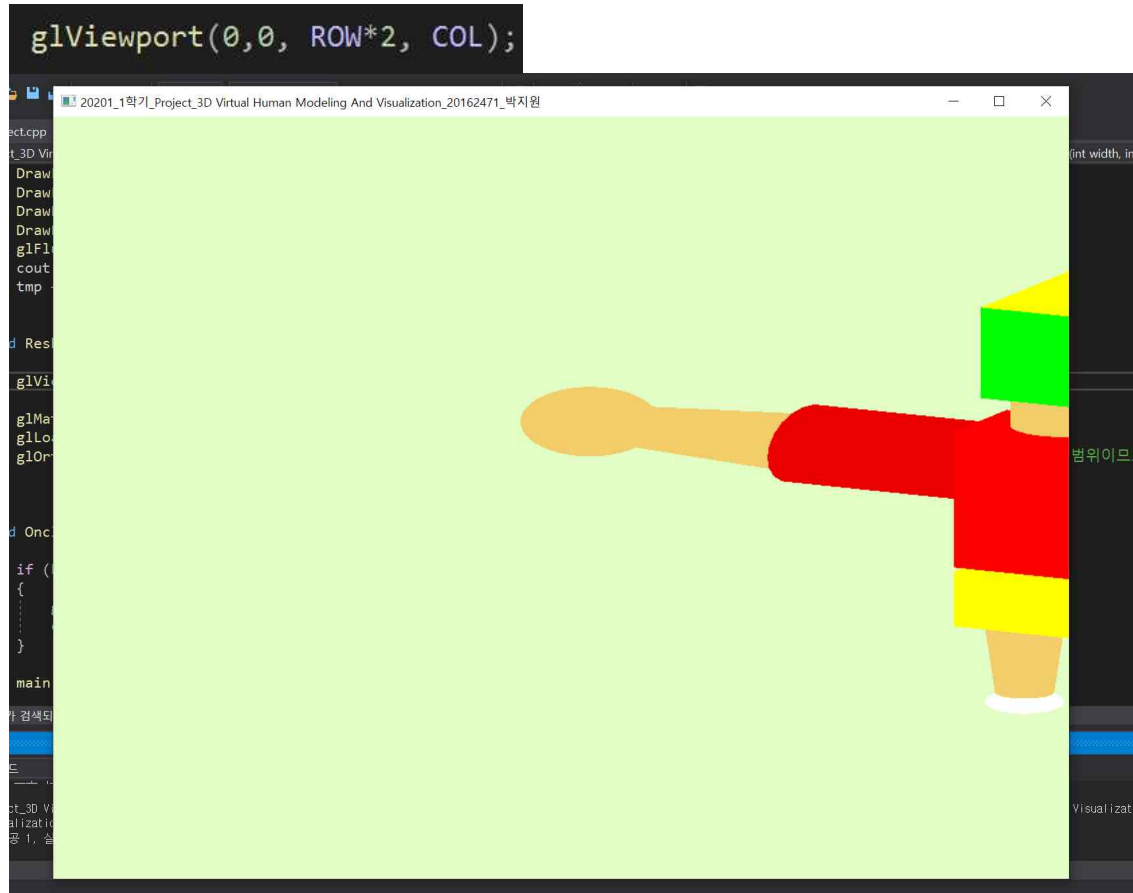
① Window 크기와 Viewport 일치시키기.

```
glViewport(0,0, ROW, COL);
```

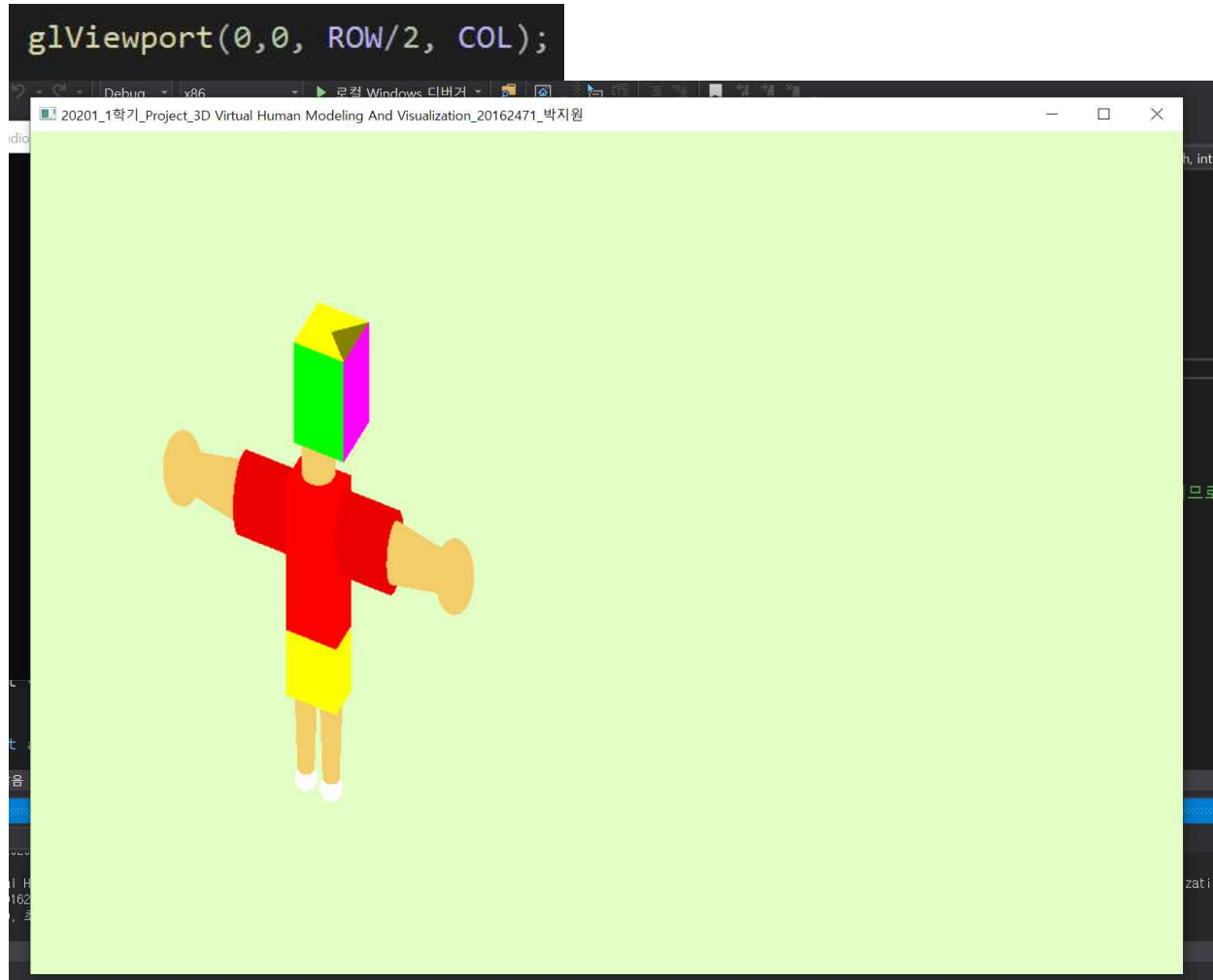


- Window와 Viewport의 크기를 갖게 하면 , 주어진 스크린에서 개발자가 화면에 보여주고 싶은 부분을 정확히 모두 보여줄 수 있습니다.

② Viewport 가로 범위를 2배 증가시킵니다.



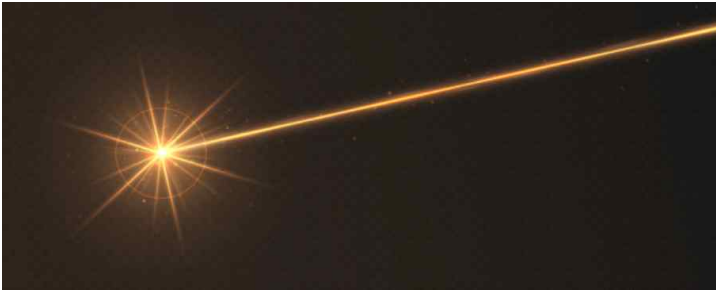
③ Viewport 가로 범위를 1/2배 증가시킵니다.



= 찍힌 사진의 크기를 줄이거나 늘리는 작업

<11주차 : Ray Tracing>

1. Ray (=광선 =반직선)



= 광원에서 나온 모든 빛이 지나는 경로

2. Ray Casting을 할 때 Scene에서 고려할 사항

- 광원의 개수
- 카메라의 위치
- Bounding Box (=Bounding Volume)
- 전반적인 배경색상
- 은은하게 밝거나 어두움을 표현하는 Ambient

3. Vector

- Ray는 Vector와 연관이 깊은 개념이라 '선형대수학'의 필기 일부분을 수정해서 다시 공부합니다.

1) Vector의 정의

- 객체의 방향과 크기를 표기하는 것을 Vector라고 합니다.

- Vector는 화살표로 표현합니다.

ex) vector(1,1) = (1,1)방향으로 루트2의 크기를 갖는 객체.

2) Vector의 크기 (magnitude)

$\mathbb{R}^n$ 의 벡터 $\mathbf{x} = (x_1, x_2, \dots, x_n)$ 에 대하여

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

을 $\mathbf{x}$ 의 **노름(norm, length, magnitude)**이라 한다.

위의 정의에서 $\|\mathbf{x}\|$ 는 원점에서 점 $P(x_1, x_2, \dots, x_n)$ 에 이르는 거리로 정의됨을 의미한다. 따라서 $\mathbb{R}^n$ 의 두 벡터 $\mathbf{x} = (x_1, x_2, \dots, x_n)$, $\mathbf{y} = (y_1, y_2, \dots, y_n)$ 에 대하여 $\|\mathbf{x} - \mathbf{y}\|$ 는 두 점 $P(x_1, x_2, \dots, x_n)$ 와 $Q(y_1, y_2, \dots, y_n)$ 사이의 **거리**로 정의한다. 즉,

$$\|\mathbf{x} - \mathbf{y}\| = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2}$$

```
gameObject.transform.position.magnitude
```

3) Vector의 내적과 두 Vector의 사이 각도

① 내적

$$\vec{a} = (a_1, a_2), \vec{b} = (b_1, b_2) \text{ 일 때}$$
$$\vec{a} \cdot \vec{b} = |\vec{a}| |\vec{b}| \cos\theta = a_1 b_1 + a_2 b_2$$

= 내적은 Vector에서의 곱을 의미합니다.

- 기호 : $\cdot$
- Unity에서는 내적의 값이 0이면 두 Vector는 수직인 성질을 검사할 때 사용합니다.
- 두 Vector의 내적 값이 0이면 두 Vector는 수직입니다.
- 두 Vector의 내적 값이 -1이면 두 Vector의 사이 각은 180도 입니다.
- **내적을 이용하면 두 Vector의 각도를 구할 수 있습니다.**

② 두 Vector의 각도

$\mathbb{R}^n$ 의 벡터 x, y 에 대하여

$$x \cdot y = \|x\| \|y\| \cos\theta, 0 \leq \theta \leq \pi$$

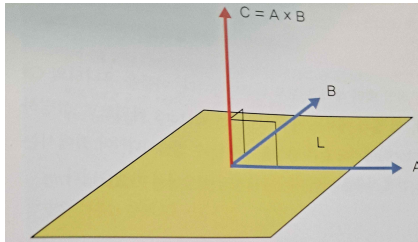
인 θ 를 x 와 y 가 이루는 **각(angle, 사잇각)**이라 한다.

4) Vector의 외적과 법선 벡터

= 두 Vector를 모두 수직으로 통과하는 Vector

- 기호 : $\times$

<법선 벡터 (=Normal Vector =Normal =노말 벡터)>



= 외적 벡터의 크기를 1로 정규화 한 벡터

- 평면은 셀 수 없이 많은 선(벡터)으로 구성되어 있습니다. 이에 외적의 성질을 조금만 응용하면 '어떤 평면에 수직인 방향'을 구할 수 있습니다.
- 위의 그림에서 평면 L에 포함된 벡터 A와 벡터 B에 대하여 동시에 수직인 벡터 C는 평면 L에 대해서도 수직입니다.
- 벡터 C의 크기를 1로 정규화하면 법선 벡터가 됩니다.
- 법선 벡터는 어떤 평면이 '바라보고 있는' 방향을 나타냅니다. 이는 카메라에서 매우 유용한 개념으로 사용됩니다.

5) Vector Normalization (Vector 정규화)

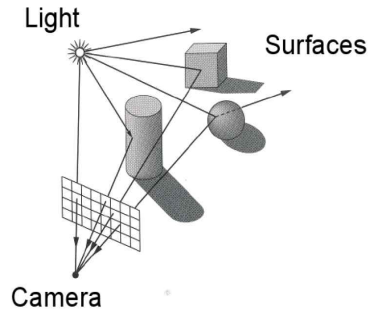
- Vector에서 크기는 무시하고 방향 정보만 필요할 때가 많습니다.
- 그 때 Vector의 크기가 1이면 편리하므로 해당 Vector의 방향은 유지하되 크기를 1로 만들어 버립니다. 이를 Vector의 정규화라고 합니다.

```
Vector3 new_vector = transform.position.normalized;  
  
float length = transform.position.magnitude;  
Vector3 new_vector_2 = transform.position / length;
```

- new_vector는 method를 사용해서 정규화를 표현한 것입니다.
- new_vector2는 일반적으로 정규화를 구하는 방식 입니다.

4. Ray Casting (광선 투사)

1) 정의



- 그림의 Light는 태양처럼 모든 방향으로 빛을 발사합니다.
- 빛이 물체에서 반사되어 Scene의 특정 픽셀로 이동하고, 픽셀에 색상을 입힙니다.
- 그림은 빛에서 ray를 쏘는 정 방향 방식보다 간단한 View Point인 Camera에서 ray를 쏘는 역 방향 방식을 사용합니다.

레이캐스팅(Raycasting)

카메라를 통한 레이의 가장 일반적인 사용 방법은 씬에 raycast를 실행하는 것입니다. 레이캐스트는 가상의 "레이저 빔"을 원점에서부터 레이에 따라 씬 안의 콜라이더에 충돌할 때까지 보냅니다. 그 다음 오브젝트와 RaycastHit 오브젝트의 충돌된 점에 대한 정보를 반환합니다. 이것은 스크린상 나타난 이미지를 기반으로 오브젝트가 어디에 위치하는지 찾는 유용한 방법입니다.

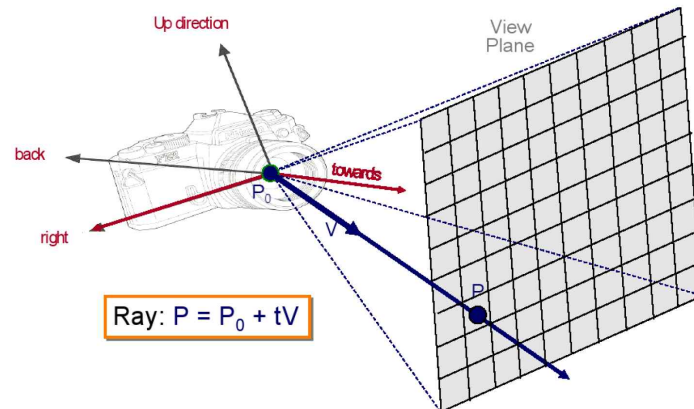
- = 컴퓨터 그래픽스와 계산기하학의 다양한 문제를 해결하기 위해 Ray와 Scene에 존재하는 물체의 교차검사를 사용하는 기법을 말한다.
- = 단순히 Camera로부터 ray가 출발한다고 하고, 결과적으로 이 빛의 진행 경로를 막고 있는 가장 가까운 물체를 찾는 기법입니다. 즉, 어떤 물체의 표면이 광원을 향하고 있을 때, 빛이 이 표면에 닿았다면 빛은 차단되지 않았거나 그림자를 만들거나 둘 중에 하나입니다.
- 불규칙한 면을 가진 물체들에 대해 빛과의 Intersection 검사는 더 간단합니다.
- 단순히 진행 방향에 대해서 Intersection을 가지고 있는지에 대해서만 판단합니다.
- Reference : <https://celdee.tistory.com/587>

2) Ray Casting 알고리즘

```
R2Image *RayCast(R3Scene *scene, int width, int height)
{
    R2Image *image = new R2Image(width, height);
    for (int i = 0; i < width; i++) {
        for (int j = 0; j < height; j++) {
            R3Ray ray = ConstructRayThroughPixel(scene->camera, i, j);
            R3Rgb radiance = ComputeRadiance(scene, &ray);
            image->SetPixel(i, j, radiance);
        }
    }
    return image;
}
```

- 모든 픽셀에 대해서 Ray Casting을 해야 하므로 이중 for문을 돌려 모든 픽셀을 검사합니다.
- Scene에 픽셀이 100개면 100개의 Ray를 발사합니다.
- &ray = 카메라에서 Scene(=View Plane)으로 발사한 Ray와 물체의 intersection(교차 지점) 정보
- Intersection에서 얻은 밝기 값 정보를 픽셀에 저장합니다.

3) 카메라에서 발사한 Ray와 Scene의 Intersection의 점의 좌표



- P : Scene에서 Ray와 만나는 점의 좌표
 - P0 : 카메라의 좌표
 - V : 카메라에서 Scene으로 발사하는 Vector
 - t : 상수
- <Example : 2차원>
- 점 P0은 (1,0)이고 t는 거리 2를 의미하고, 벡터 V가 (1,1)이면 P는 $(1,0) + 2*(1,1) = (3,2)$ 의 좌표를 갖습니다.

4) Ray Intersection

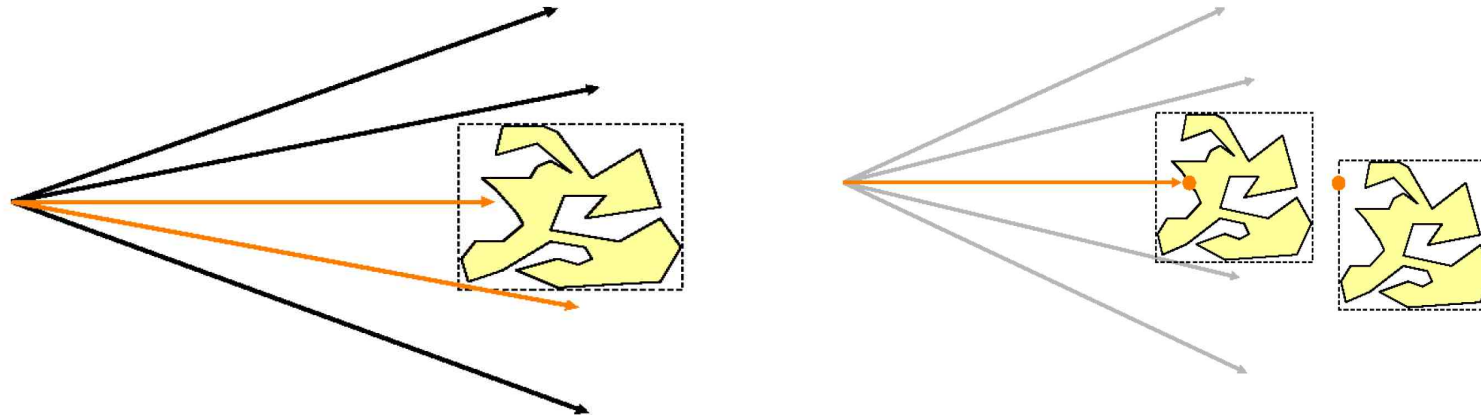
= Ray와 Scene에 존재하는 물체의 교차 지점

- ① Sphere
- ② Triangle
- ③ Box
- ④ Scene

5) Ray Intersection Acceleration

- Ray와 물체의 Intersection을 검사하는 과정은 매우 많은 계산을 요구하기 때문에 시간이 많이 사용됩니다. 이를 해결하기 위해 계산 과정을 단축시켜 가속화하는 방식을 알아보시다.

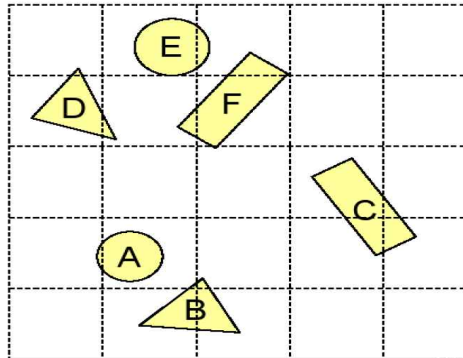
- ① Bounding Volume (=Bounding Box)



= 복잡한 물체에 대해서 자신을 둘러싼 사각형을 만들고 사각형에 대한 Intersection을 먼저 계산하는 기법

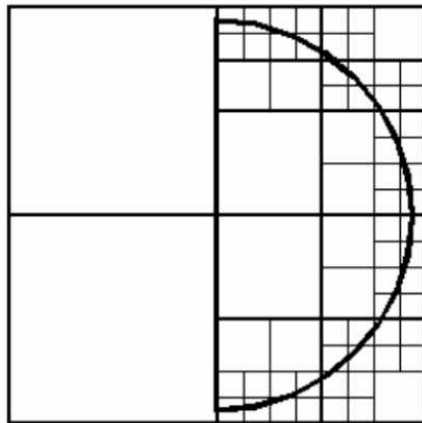
- Bounding Volume Hierarchy : 물체 한 개에 대해서 계산을 하지 않고 , 여러 물체에 대한 Bounding Volume을 만들어서 계산하는 기법
- Sort Bounding Volume Intersection : Camera 기준으로 물체가 다른 물체에 뒤에 있어서 가려진다면 Intersection 계산을 하지 않는 기법
- Cache Node Intersections : Ray가 공간적으로 유사할 때 바로 직전의 계산을 활용하는 기법

② Grid(격자) 방식_1 : Uniform Grid



- Scene을 격자로 나눕니다.
- 같은 격자에 존재하는 물체만 Intersection 계산을 수행합니다.
- 격자를 너무 잘게 쪼개거나, 너무 크게 쪼개면 둘 다 문제가 발생합니다.

③ Grid 방식_2 : Octree



- = 물체가 많은 부분은 격자를 많이 쪼개고, 적은 부분은 조금 쪼개는 방식
- Scene을 격자로 나눕니다.
- Uniform Grid 보다 뛰어난 성능을 가집니다.

5. Ray Tracing (광선 검출)

- Ray Tracing은 Ray Casting과 다르게 한번 Ray를 발사한 이후에도 빛을 추적합니다. 즉, 표면에 빛이 닿았을 때 이 표면이 빛을 받는 것인지 그림자를 만드는 것이지만 결정하는 것이 아니라, 표면에 닿는 순간 반사, 굴절, 그림자 광선과 같은 추가적인 빛이 다시 생성된다고 가정합니다. 이렇게 생성된 각각의 빛들은 또 다시 다른 표면에 대해 같은 작용을 하게 되고, 이것은 Ray Casting보다 훨씬 현실에 가깝게 재현할 수 있습니다.

- Ray Tracing은 Ray Casting에 비해 막대한 계산이 필요합니다.

- 그림자 광선을 먼저 추적하는 방법이 있다, 즉, 그림자 광선을 먼저 적용하여 보이지 않는 부분에 대해서 빛을 추적하지 않는 것이다.

Ray Tracing은, 다른 렌더링 알고리즘을 사용했을 때 구현하기 곤란한 그림자, 반사 효과 따위가 알고리즘 자체에 자연스럽게 포함되어 있다는 장점을 갖는다. 또, 상대적으로 다른 알고리즘에 비해 구현이 비교적 간단하며, 광선 하나 하나가 다른 빛과 상관없이 개별적인 계산 과정을 거치므로 병렬화에 유리하다는 장점도 있다.

- 전형적인 global illumination model

- 광원에서 나온 모든 Ray를 추적하는 방법인 순방향 레이트레이싱은 너무 많은 계산이 필요하기 때문에 우리 눈에서 광선이 발사된다고 가정하고 추적해 들어가는 방법인 **역방향 레이트레이싱**을 주로 사용합니다.

- 본질적으로 많은 연산이 필요하기 때문에 처리 속도가 느립니다.

<12주차 : Lighting and Reflectance>

1. 헛갈리는 두 개념 : Shader vs Shading

1) Shader

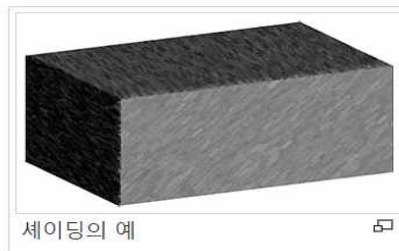
컴퓨터 그래픽 분야에서 셰이더(shader)는 소프트웨어 명령의 집합으로 주로 그래픽 하드웨어의 렌더링 효과를 계산하는 데 쓰인다. 셰이더는 그래픽 처리 장치(GPU)의 프로그래밍이 가능한 렌더링 파이프라인을 프로그래밍하는 데 쓰인다.

셰이더는 표면상으로 무한해 보이는 효과를 만들기 위해 영화 후처리, CGI, 비디오 게임에 널리 쓰인다. 단순한 광원 모델을 떠나, 더 복잡한 이용에는 영상의 색조, 채도, 밝기, 대비를 변경하는 일과 블러, 라이트 블룸, 입체 광원, 심도 효과를 위한 노멀 매핑, 보케, 셀 셰이딩, 포스터리제이션, 범프 매핑, 왜곡, 크로마 키 (이른바 블루스 크린/그린스크린 효과), 테두리 검출, 모션 감지, 사이키델리아 효과 제작 등을 포함한다.

= GPU의 렌더링 효과를 위해 사용되는 소프트웨어

- Reference : <https://www.youtube.com/watch?v=sXbdF4KjNOc>

2) Shading (=그림자 처리 =음영 처리 =명암 처리)



Shading = 미술의 데생

= Rendering 과정에서 빛의 거리와 각도에 따라 물체 표면의 조도(물체의 표면이 받는 빛의 세기)를 변화시키는 기법

- 미술 분야에서 Dessin(=데생 , =소묘)이 Shading입니다.

- 교재에서는 Lighting And Shading(=10강_조명과 음영)을 같이 다루지만 강의에서는 12강_Lighting , 13강_Rasterization으로 구분해서 강의하겠습니다.

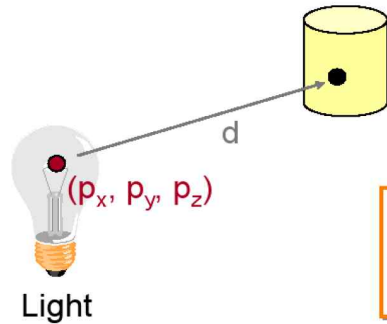
2. Direct Illumination (=Local Illumination =지역 조명 =직접 조명)

- Global Illumination을 단순화한 개념이며 , 다른 물체에서 반사된 빛은 일체 감안하지 않습니다.
- 광원에서 나오는 빛과 , 그 빛을 받은 물체에서의 반사에 대해서 설명합니다.
- 현실세계의 빛은 매우 복잡한 형태로 이루어진 에너지입니다. 그러므로 컴퓨터로 빛과 , 빛의 반사를 표현하는 것은 매우 많은 parameter를 요구하며 , 시간복잡도와 공간복잡도가 어마어마해서 표현하기가 어렵습니다. 그래서 , 컴퓨터(OpenGL)는 빛과 빛의 반사를 Direct Illumination Model을 이용해서 표현합니다.
- Global Illumination을 사용하는 모델만큼 사실적인 묘사를 하는 것이 어렵지만 처리 속도는 빨라집니다.
- Direct Illumination에서도 그림자와 주변광을 이용해서 근사적으로나마 Global Illumination과 비슷하게 구현을 합니다.
- 지금부터 설명하는 모든 개념은 OpenGL에서 빛과 빛의 반사를 표현하는 모델에 대해서 설명합니다.

1) Emission at light source

= 광원에서 나오는 빛을 3가지 수학적 모델로 표현합니다.

① Point Lights



$$I_L = \frac{I_0}{c_a + l_a d + q_a d^2}$$

= 광원에서 나온 빛이 모든 방향에 대해 동일한 밝기로 물체에 도달합니다.

- Intensity (I) : 빛의 세기

- Distance (d) : 광원과 물체 사이의 거리

- c , l , q : 상수

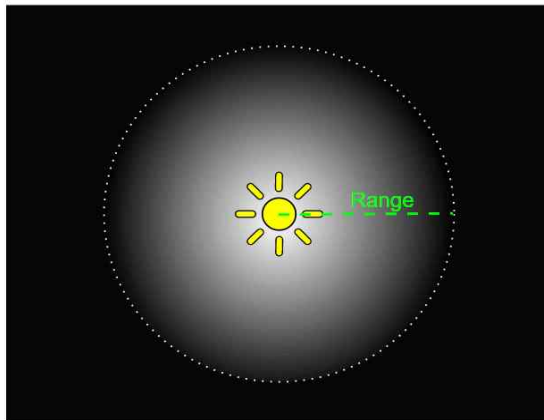
- I_L 은 물체가 받는 빛의 세기이고 , I_0 은 광원에서 나오는 빛의 세기입니다.

- 광원과 물체 사이의 거리와 빛의 세기는 반비례 합니다.

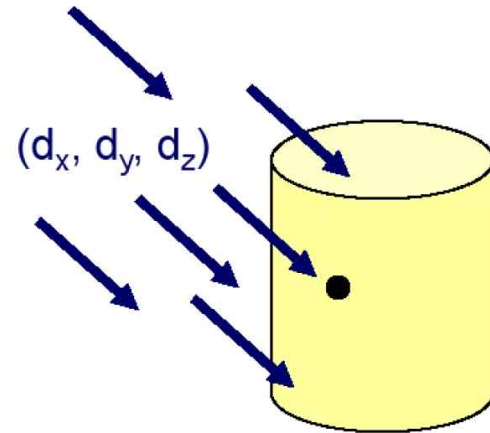
ex) 전구

Point lights

점 광원(Point lights)은 공간의 한 점에 있고, 광원을 모든 방향으로 균등하게 내보냅니다. 표면에 닿는 광원의 방향은 광원이 닿은 지점에서 라이트 오브젝트의 중심으로 다시 이어지는 선입니다. 광도는 광원에서 거리가 멀어질수록 약해지고, 지정된 거리에서 0이 됩니다. 이를 '역제곱 법칙'이라고 하며, 광원이 실제 세계에서 작용하는 방법과 유사합니다.



② Directional Light Source



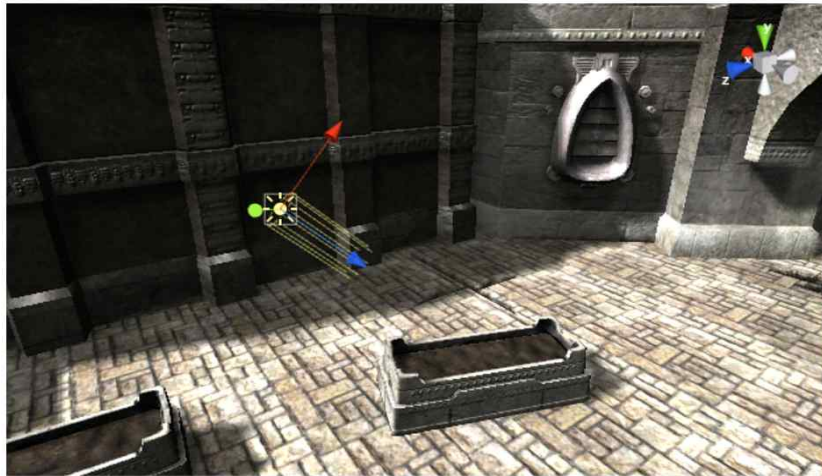
$$I_L = I_0$$

= 방향이 있는 빛을 의미합니다.

- 빛의 세기가 무한대라고 이해해도 무방합니다. 그로 인해 빛의 세기가 광원과 물체 사이의 거리와 무관하게 항상 일정합니다.

ex) 태양

방향 광원은 게임 세계의 범위 밖에 있는 위치에서 오는 크고 먼 광원을 표현합니다. 사실적인 씬에서는 방향 광원을 사용하여 하나 달을 시뮬레이션할 수 있습니다. 추상적인 게임 세계에서는 빛이 정확하게 어디서 오는지 지정하지 않고 오브젝트에 사실적인 셰이딩을 추가하는 데 유용한 방법이 될 수 있습니다.

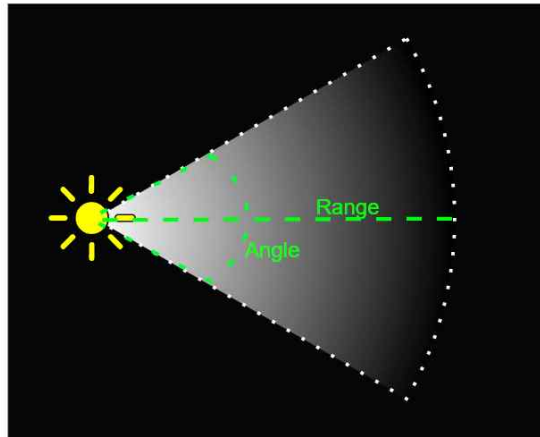


③ Spot Light Source

= 범위가 한정된 Point Light입니다.

- 주어진 각도 안에서만 빛이 존재하며 , 그 외에 부분은 빛이 존재하지 않습니다.

스팟 광원(Spot lights)은 점 광원과 마찬가지로 광원이 떨어지는 지정된 위치와 범위가 있습니다. 하지만 스팟 광원은 특정 각도로 제약되므로 조명이 비추는 원뿔 모양의 영역이 발생합니다. 원뿔의 중앙은 라이트 오브젝트의 전방(Z) 방향을 가리킵니다. 또한 광원은 스팟 광원 원뿔의 가장자리로 갈수록 약해집니다. 각도를 넓히면 원뿔의 너비가 증가하고 '페넬라'라고 하는 이 페이드(광원이 약해지는 영역)의 크기도 함께 커집니다.



스팟 광원은 일반적으로 플래시라이트, 자동차 헤드라이트 및 서치라이트 같은 인공 광원으로 사용됩니다. 스크립트나 애니메이션에서 방향을 제어하는 움직이는 스팟 광원은 씬의 작은 영역에만 광원을 비추고 극적인 조명 효과를 만들어 냅니다.

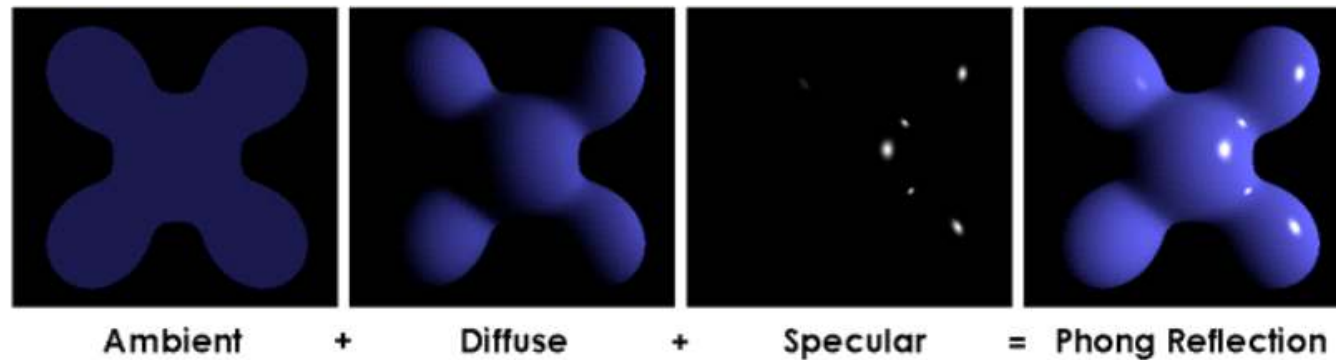


2) Scattering at surfaces

= 광원으로부터 빛을 받아 물체 표면에서 반사되어 나타나는 빛과 물체 자체의 빛을 표현합니다.

- 반사 또한 실제 세상의 물리법칙을 적용하여 구현하려면 매우 많은 parameter를 이용해야 하므로 거의 불가능합니다.
- 실제 세상의 물체는 매끈해보여도 실제로는 울퉁불퉁하기 때문에 빛이 들어오고 나가는 방향을 모두 계산해서 구현하는 것은 불가능합니다. 그렇기 때문에 3가지 요소만 고려하는 근사 모델인 'Phong Reflection Model'을 사용해서 구현합니다.
- 반사 또한 컴퓨터(OpenGL에서) "Phong" 이라는 근사모델을 통해 구현하였습니다.
- 나중에 배우는 'Phong Shading'과 'Phong Reflection Model'은 전혀 관련이 없는 개념입니다.

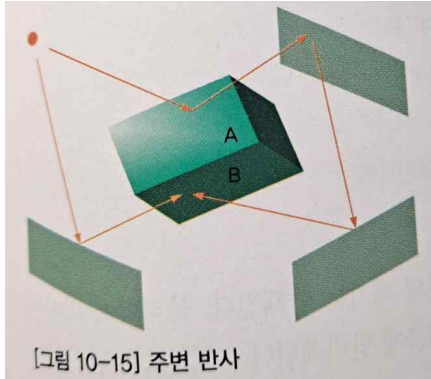
<Phong Reflection Model의 3가지 요소>



Components of Phong illumination or reflection model using RGB model:

OpenGL allows us to break this light's emitted intensity into 3 components: ambient L_a , diffuse L_d , and specular L_s . Each type of light component consists of 3 color components, so, for example, L_{rd} denotes the intensity of the red component of diffuse illumination.

① Ambient Reflection (=주변 반사)



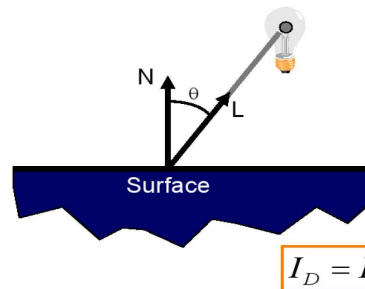
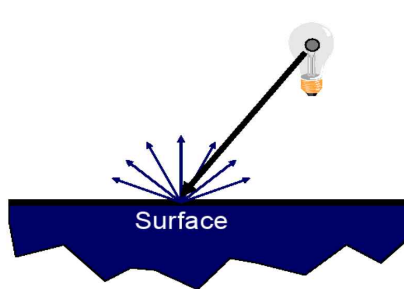
= 광원에 직접 노출되지 않는 면에 밝기를 부여한 것

- 공간의 밝고 어두운 느낌

- 어떤 공간에 들어가면 빛과는 별개로 밝거나 어두운 느낌이 있습니다.

ex) 광원에 직접 노출되지 않으므로 물체면_B에서 반사되는 빛은 없으나, 주변의 다른 물체면 으로부터 반사되어 물체면_B로 입사되는 빛

② Diffuse Reflection (=확산 반사)



$$I_D = K_D (N \cdot L) I_L$$

- N : Normal Vector = 법선 벡터 = 물체의 수직 방향의 벡터

- L : 광원의 Vector

- I_L : 광원에서의 빛의 세기

- I_D : 난반사의 빛의 세기

= 광원으로부터 빛이 거친 표면으로 들어와서 여러 방향으로 다수의 빛이 반사되어 확산 되는 반사

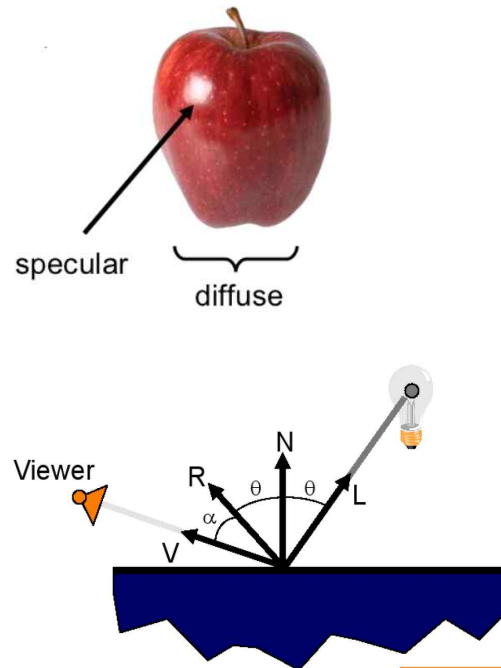
- 난반사에 해당하는 반사

- LDE와 같은 개념

- 물체의 수직 방향과, 빛이 들어오는 각도가 중요합니다.

- 물체의 표면에 따라 반사하는 방향이 다양한데, 반사하는 방향과 같은 시점에 존재한다면 더욱 밝게 보입니다.

③ Specular Reflection (=거울 반사)



$$I_S = K_S (V \cdot R)^n I_L$$

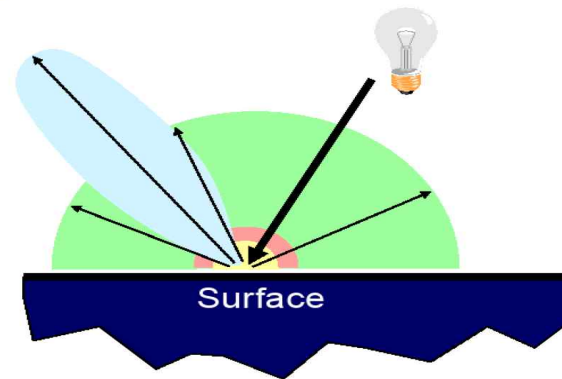
= 광원에서부터 빛이 반질반질한 표면으로 들어와서 단 하나의 방향으로 빛이 반사되어 확산 되는 반사

- 정반사에 해당하는 빛의 반사
- LSE와 같은 개념
- 반사되는 빛(R)과, Viewer가 보는 방향(V)이 중요합니다.
- 수식보다는 쉽게 생각해보면, 반사되는 빛과 Viewer의 각도가 작으면 직접적으로 보기 때문에 잘 보입니다.
- $V=R$ 일 때 가장 잘 보입니다.
- 사과 그림에서 나타는 Specular Reflection 부분이 물체(사과)의 색인 빨간색이 아니라, 광원의 백색인 이유는 광원에서 나오는 빛의 색이 우리 눈 그대로 들어오기 때문입니다.

<Phong Reflection Model 3요소 총괄한 자료>

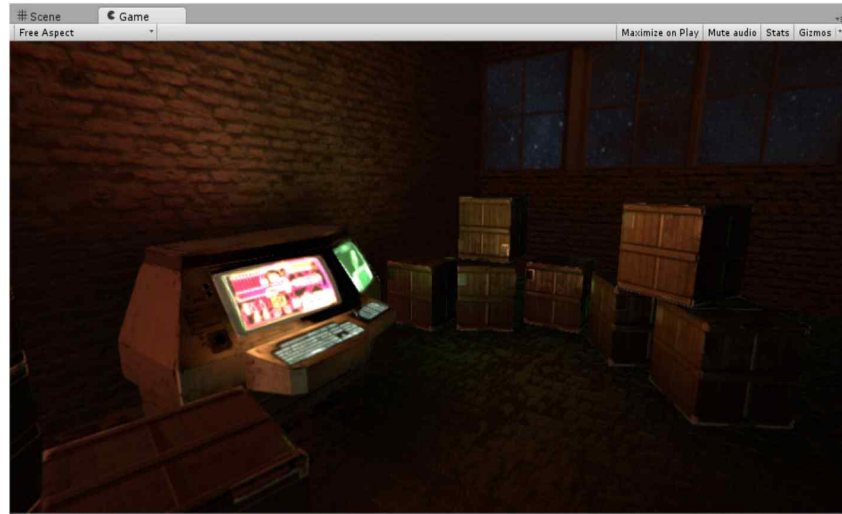


- diffuse reflection +
- specular reflection +
- emission +
- "ambient"



<Emission (=물체 자체의 빛)>

- Phong Reflection Model에는 포함이 되지 않습니다.



= 모니터 화면 같이 빛이 있는 물체에서 방출 되는 빛을 의미합니다.

- 광원은 아닙니다.

- 광원처럼 빛은 나지만 , 주변에 영향을 주지 않는 빛을 의미합니다.

- Unity에서 네온이나 광원 같은 물체를 만드는데 유용한 방법입니다.

<Reference>

<https://docs.unity3d.com/Manual/StandardShaderMaterialParameterEmission.html>

3. Global Illumination (=Indirect Illumination =전역 조명) : 너무 어려움 이해안감
= 다른 물체에서 반사되어 입사되는 빛까지 고려한 조명 모델

1) Ray Tracing
= 11주차에서 공부했습니다.

2) Shadow (=그림자)
- 사실감을 증가시킵니다.
- 기존의 빛 관련 수식에 SL(Shadow term)을 추가시킵니다.
- 물체가 가려져 있으면 SL을 0으로 설정합니다.

3) Transmission
= 물체 통과

4) Inter-Object Reflection
= 물체간의 빛 반사로 생긴 빛

<13주차 : Rasterization>

1. Rasterization (=래스터 변환 =픽셀 변환)

= 정점 , 선분 , 다각형 내부 등을 표현하기 위해 어떤 pixel을 선택해야 하는지를 결정하는 작업

- Raster는 pixel을 의미합니다.

- Rasterization에 포함된 4가지 개념을 학습합니다.

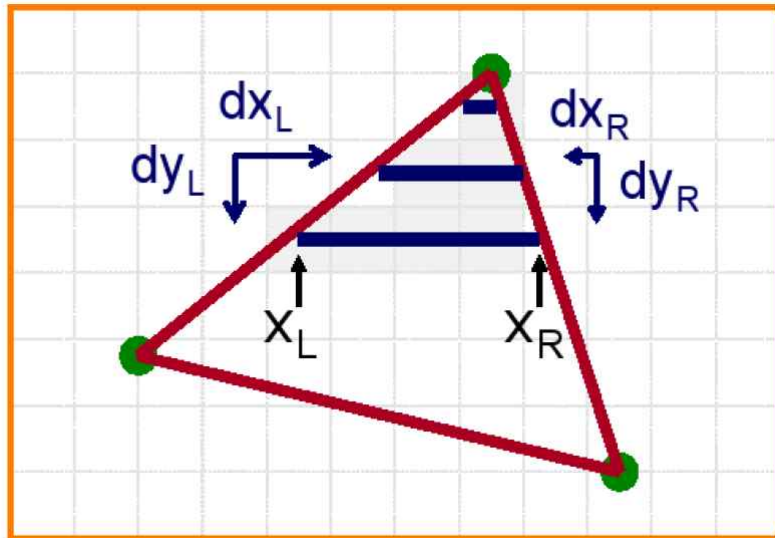
Rasterization

- Scan conversion
 - Determine which pixels to fill
- Shading
 - Determine a color for each filled pixel
- Texture mapping
 - Describe shading variation within polygon interiors
- Visible surface determination
 - Figure out which surface is front-most at every pixel

2. Scan Conversion

= 픽셀에 어떤 색깔을 칠할지를 결정하는 기술

<Triangle Sweep-Line Algorithm>

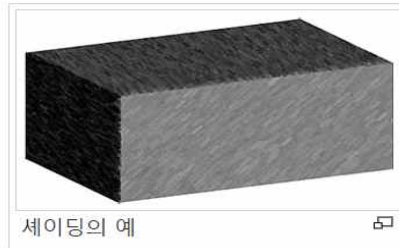


- 픽셀을 색칠할 때 Brute-Force하게 하지 말고, 효율적으로 빠르게 색칠하기 위한 알고리즘

<알고리즘 순서>

- ① 가장 위에 존재하는 꼭짓점을 잡고 왼쪽 직선과 오른쪽 직선의 기울기를 계산합니다.
- ② y축으로 한 칸 씩 내려가면서 직선 안에 존재하는 픽셀을 구하고 업데이트를 합니다.
- ③ 직선 안에 존재하는 픽셀들의 색상을 설정합니다. ex) : SetPixel(x,y,rgba)
- ④ '② ~ ③'의 과정을 반복합니다.

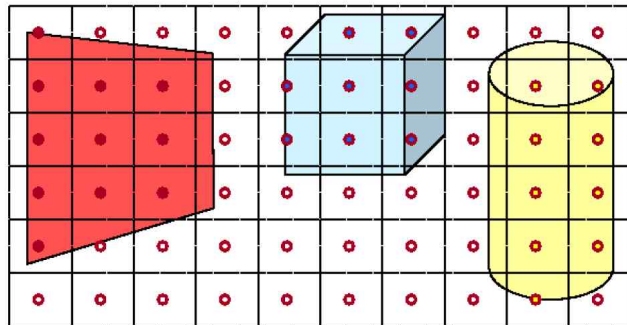
3. Shading (=그림자 처리 =음영 처리 =명암 처리)



Shading = 미술의 데생

- = Rendering 과정에서 빛의 거리와 각도에 따라 물체 표면의 조도(물체의 표면이 받는 빛의 세기)를 변화시키는 기법
- 미술 분야에서 Dessin(=데생, =소묘)이 Shading입니다.
- Surface Rendering 이라고도 부릅니다.
- 물체를 렌더링 할 때 현실감을 증폭시키는 요소입니다.

1) Polygon Shading

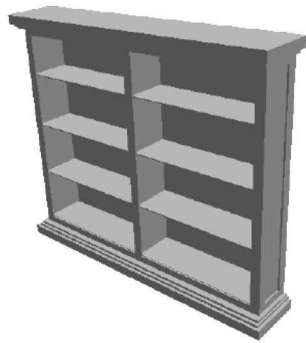


- = 인접한 픽셀은 같은 색상으로 칠하는 방식(?)
- 속도가 빠릅니다.

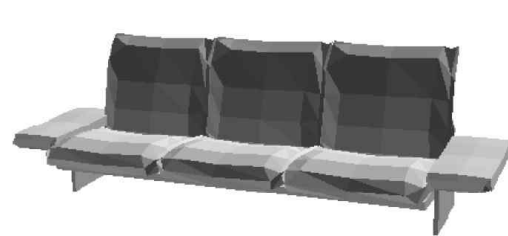
2) Flat Shading

= 다각형 하나의 색상을 모두 동일하게 칠하는 방식

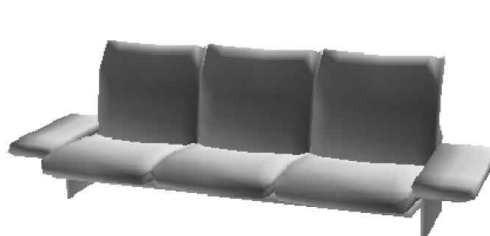
- 상수 Shading OR 깎은 면 Shading 이라고도 부릅니다.
- 계산이 매우 빠르고 간단합니다.
- 각각의 다각형 사이의 경계선이 필요 이상으로 뚜렷하게 보이는 '마하 밴드 효과'가 발생합니다. 그러므로 각이 없이 smoothing된 model에서는 적합한 방식이 아닙니다.
- 아래의 그림과 같은 각이 진 책꽂이는 Flat Shading 방식이 괜찮습니다.



← 마하 밴드 효과



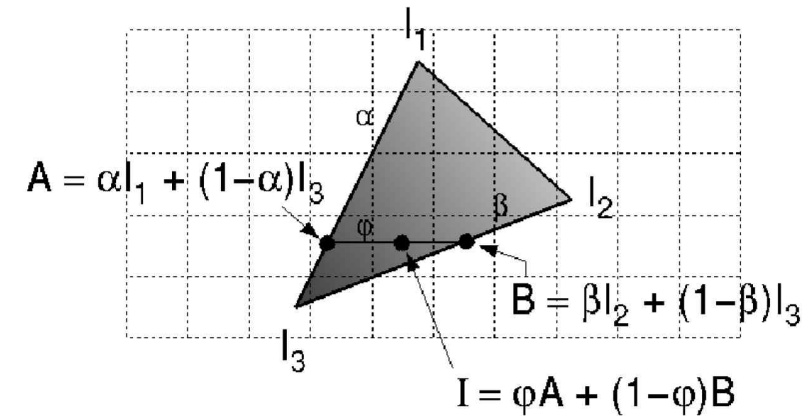
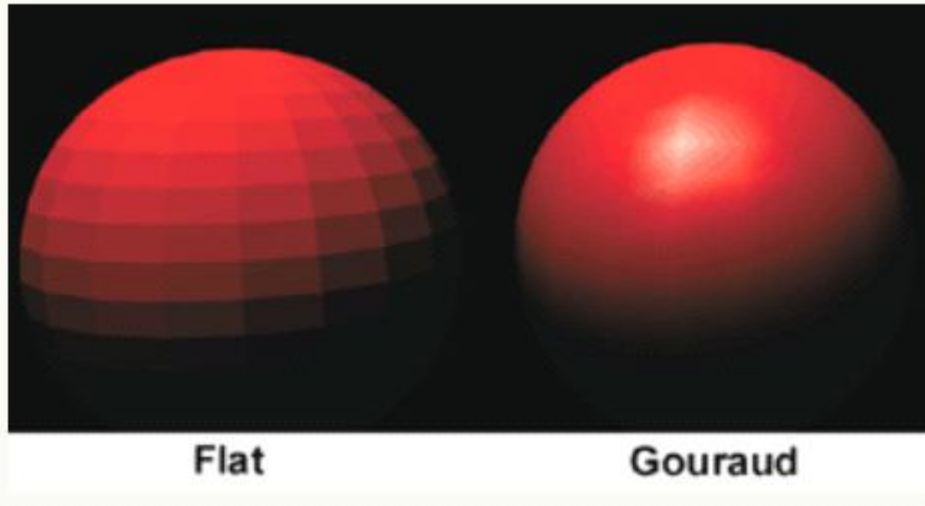
Flat Shading



Gouraud Shading

- Flat Shading에서 의자의 쿠션은 6개의 다각형으로 이루어져 있고, 각각의 다각형은 동일한 색상으로 칠해져 있습니다. 마하 밴드효과가 심하게 느껴집니다.

3) Gouraud Shading (고로 셰이딩) : 정점의 반사광 세기를 보간(=interpolation)



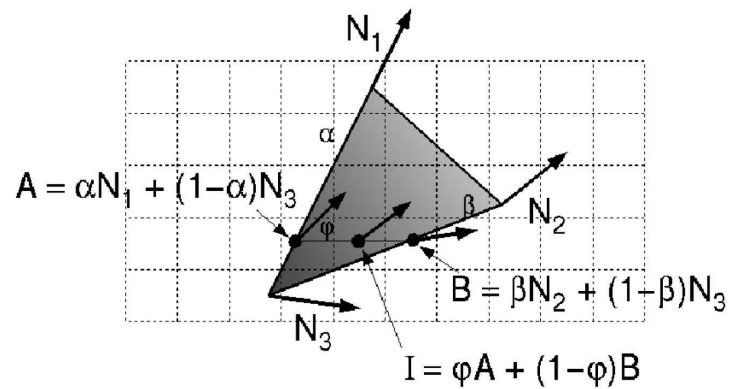
= 다각형에는 무한개의 점이 있다고 가정하고 다각형을 하나의 색으로 칠하지 않고, 서로 다른 여러 가지 색으로 칠하는 방식

- 그로 인해 마치 하나의 다각형이 다시 여러 개의 더 작은 다각형들로 나누어져서 빛이 가해진 것과 비슷한 효과를 창출합니다.

- 법선 벡터를 면에 대해서만 생각하는데 컴퓨터 그래픽스에서는 점에 법선 벡터를 저장합니다. 점의 법선 벡터와 Viewer와의 방향 벡터를 가지고 수식을 계산하여 점에 색상을 저장합니다. 점에 저장한 색상들(L1과 L3)끼리 linear interpolation(보간)을 해서 점(A)의 색상을 정합니다. 이걸 계속 반복하여 모든 점의 색상을 설정하면 물체의 음영이 점차 부드러워집니다.

ex) 점L1과 점L3을 3:1로 나눈 점A의 색상을 비례식의 결과로 지정합니다.

4) Phong Shading : 정점의 법선 벡터를 보간(=interpolation)



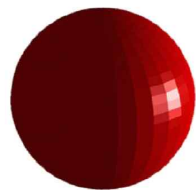
= 점에 저장된 법선 벡터들끼리 Linear Interpolation을 해서 새로운 법선벡터를 만들고 , 그걸 반복합니다.

- 원리는 Gouraud Shading과 동일하지만 , 점을 사용하여 보간을 하는 Gouraud Shading 과는 다르게 Phong Shading은 법선 벡터를 사용하여 보간을 합니다.

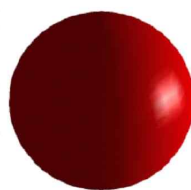
- 점에 저장하는 빛의 세기는 1개의 인자를 사용하며 , 법선 벡터는 x,y,z 3개의 인자를 사용하기 때문에 Phong Shading은 Gouraud Shading 보다 처리 시간이 3배나 더 필요합니다. 하지만 그로 인해 더 사실적입니다.

- Phong Reflection Model과는 아예 관련이 없는 개념입니다.

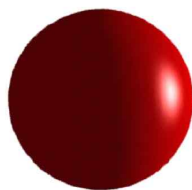
5) 3가지 Shading 비교



Flat

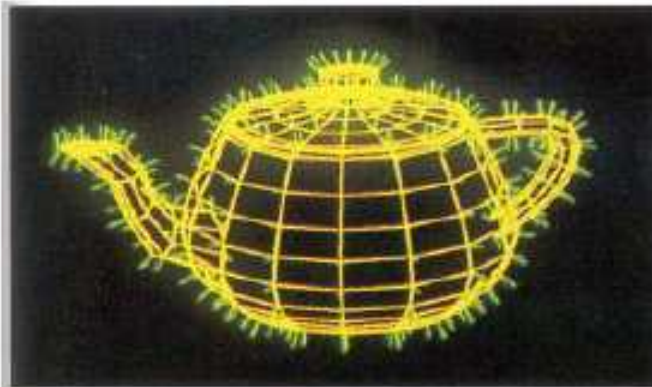


Gouraud



Phong

Wireframe



Flat



Gouraud



Phong

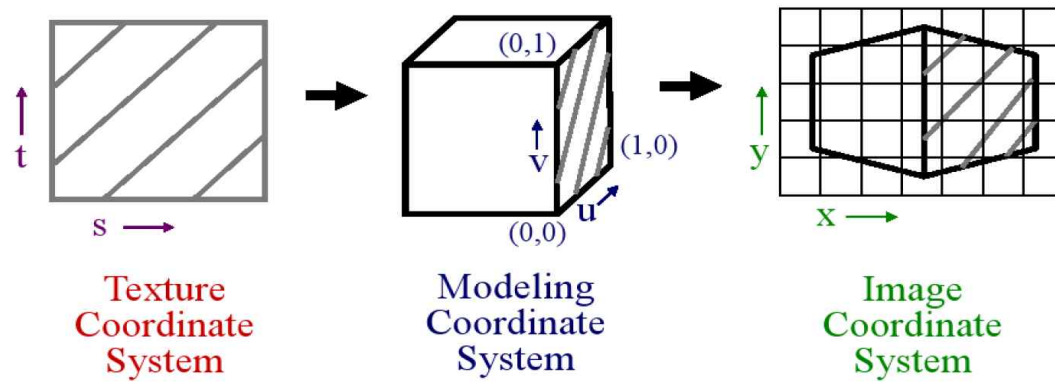
2. Texture Mapping

1) 정의

= Pixel에 사실감 있는 질감(영상)을 입히는 방식

- Shading만으로는 한계가 존재하기 때문에 3D model에 Texture를 입혀서 좀 더 실감나게 구현합니다.
- Texture(2D)를 Model(3D)에 입힙니다.

2) 방법



① Texture를 준비 (Texture 좌표계)

② 변환을 사용해서 'Texture 좌표계 -> Image 좌표계' mapping을 실시시킵니다.

3) 문제점과 해결책

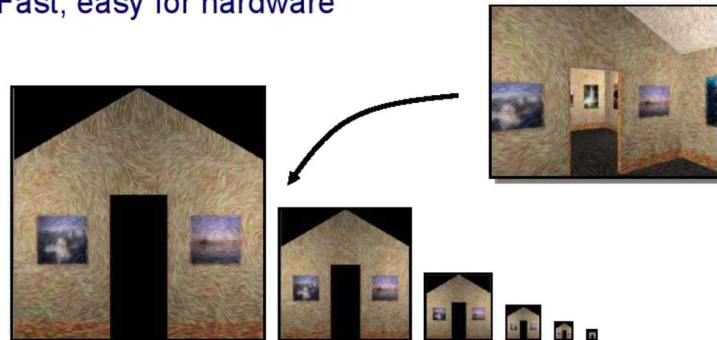
① 문제점 : Aliasing이 발생합니다.

② 해결책

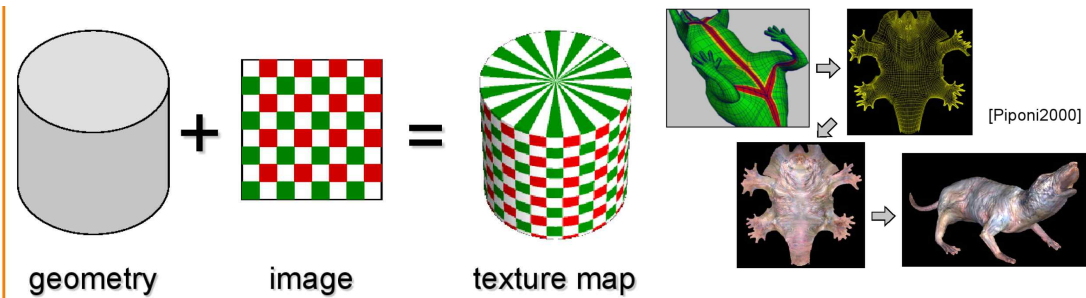
- Convolution Filtering으로 anti-aliasing
- Mipmaps : Texture를 여러 단계의 해상도로 준비하고 , 물체에 알맞은 Texture를 용합니다.

<Example : Mipmaps>

Fast, easy for hardware

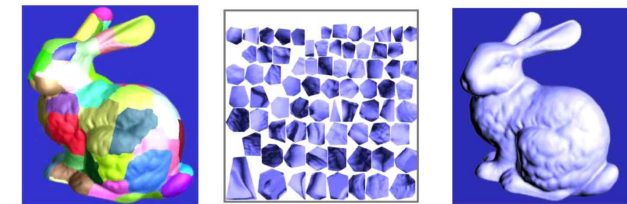


4) Parameterization



= 3D 물체(Geometry)에 어떻게 2D 이미지를 입힐지에 대한 방법

ex) Unfold The Surface(해부), Make An Atlas(3D 물체를 쪼갭니다.)



charts

atlas

surface

5) 응용 사례

- 교수님께서 다양한 사례를 영상으로 보여주셨습니다.

3. Visible Surface Determination

= 두 물체가 겹쳐 있을 때 **가려지는 부분을 화면에 출력하지 않습니다.**

1) Depth Sort (미술의 콜라주)

= 눈에서 먼 Image부터 화면에 그립니다. 그렇게 되면 가까운 Image가 미리 그린 Image에 Overwriting 됩니다.

2) Z-Buffer

= 색상과 명암 말고 **얼마나 떨어져 있는지(depth)**를 **parameter(Z)**로 추가합니다.

- Rendering 하는 과정에서 Z를 비교하면서, 보이지 않는 부분은 애시 당초 그리지 않습니다.

- 하드웨어 가속이 가능하여 속도가 빨라지지만 , Parameter Z가 존재해야 하므로 추가적인 메모리가 필요합니다.

<14주차 : 기말고사 질문>

1. 기말고사 범위의 강의 구성

- 14강을 제외하고 기말고사 범위를 크게 나누면 modeling과 rendering으로 나눌 수 있다고 생각합니다.
- 모델링의 개념을 학습 -> 모델링의 종류_1 : Polygonal Mesh -> 모델링의 종류_2 -> Subdivision Surface -> 렌더링의 개념을 학습 -> 렌더링에서 고려할 사항_1 : ray_tracing -> 렌더링에서 고려할 사항_2 : Lighting and Reflectance -> modeling과 rendering을 통합한 rasterization pipeline
- 이렇게 구분지어 이해해도 괜찮을까요? 네^^

2. direct illumination , indirect illumination , local illumination , global illumination

- 교수님께서서는 direct 와 global로 나누어서 강의해주셨습니다.
- 개인적으로 공부하는 교재에서 local 과 global로 나누어서 설명하고 있습니다.
- local = direct , global = indirect로 이해해도 괜찮을까요? 네^^
- 인터넷에서 얻은 자료들이 많지만 명쾌하지 않아서 질문 드립니다.

<참고 자료>

Local Illumination(= Direct Illumination, 로컬광, 직접광) : 광원에서 직접 빛을 받아 만들어진 음영

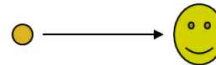
Global Illumination(GI = Indirect Illumination, 전역광, 간접광) : 광원에서 간접적으로 빛을 받아 만들어진 음영 혹은 직접광을 받은 주변의 물체에서 반사된(산란된) 빛에 의해 만들어진 음영

3. LDE와 LSE

Path Types

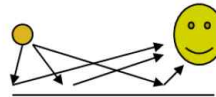
- LE

- The light goes straight from the source to the viewer



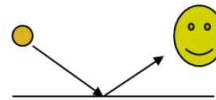
- LDE

- The light goes from the light to a diffuse surface that the viewer can see



- LSE

- The light is reflected off a mirror into the viewer's eyes



- L(S|D)E

- The light is reflected off either a diffuse surface or a specular surface toward the viewer

- Phong Reflection에서 LDE가 Diffuse Reflection이고 LSE가 Specular Reflection을 의미하는 것 인가요? 그렇다면 빛이 있는 제 방에서 책을 보는 것도 결국 LDE 이자 Diffuse Reflection인 것인가요? 네^^

<학기말 프로젝트 : OpenGL>

*** Dr.Bean의 코딩교실 -> 재생 목록 -> 컴퓨터 그래픽스를 수강했습니다. ***

1. Main 함수

```
int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGBA | GLUT_DEPTH);
    glutInitWindowSize(ROW, COL);
    glutCreateWindow("20201_1학기_Project_3D Virtual Human Modeling And Visualization_20162471_박지원");

    glutDisplayFunc(DisplayAllObjects);
    glutReshapeFunc(Reshape);
    glutMotionFunc(MouseListener);
    glutKeyboardFunc(KeyboardListener);
    glutMainLoop();

    return 0;
}
```

- glutInit : GLUT 라이브러리를 초기화한 후 윈도우 시스템과 세션을 엽니다.
- glutInitDisplayMode : 생성될 윈도우의 디스플레이 모드를 설정
- glutInitWindowSize : 프로그램의 첫 실행 시에 윈도우 크기를 설정
- glutCreateWindow : 새로운 윈도우의 이름을 지정하고 만듭니다.
- 아래의 함수는 모두 Call-Back Method입니다.
- glutDisplayFunc(내가 만든 함수) : 디스플레이 이벤트에 대한 call-back Method
- glutReshapeFunc(내가 만든 함수) : 프로그램 실행 중 사용자가 윈도우 크기를 재조정 했을 때 호출되는 함수
- glutMotionFunc(내가 만든 함수) : 마우스 드래그 관련 call-back Method
- glutKeyboardFunc(내가 만든 함수) : 키보드 입력 관련 call-back Method

2. Modeling 함수

```
void DrawLeftEyebrows(int position_x)
{
    GLUQuadricObj* qobj;
    qobj = gluNewQuadric(); // Quadric 객체 속성 설정 함수
    gluQuadricDrawStyle(qobj, GLU_FILL);
    /*
        GLU_FILL Solid
        GLU_POINT point(점)
        GLU_SILHOUETTE 외부모서리(선)만 그림
    */

    glTranslatef(position_x, 80, 95);
    glRotatef(90, 1, 0, 0);
    glRotatef(45, 0, 1, 0);
    glColor3f(0, 0, 0);
    gluCylinder(qobj, 10, 10, 10, 100, 100);
    glRotatef(-45, 0, 1, 0);
    glRotatef(-90, 1, 0, 0);
    glTranslatef(-position_x, -80, -95);
}
```

- glTranslatef(x,y,z) : 이동 함수
- glRotatef(x,y,z,u) : 회전 함수
- glOrtho(6) : 화면의 좌하단 좌표의 x값 , 화면의 우상단 좌표의 x값 , 화면의 좌하단 좌표의 y값 , 화면의 좌상단 좌표의 y값
- glOrtho(0,200,0,200,0,200)은 모든 그림들이 (0,0)을 기준을 중점으로 생성되므로 잘립니다. 그러므로 glOrtho(-100,100,-100,100,-100,100)으로 그래프시다.
- glutSolidCube(100) : 모서리의 길이가 100인 정사각형을 그립니다. 그리고 중점은 (0,0)입니다. 그러므로 glOrtho(0,200,0,200,0,200)에서 그림면 (0,0)기준으로 정사각형의 1사분면만 보이고 , 2~4사분면은 잘립니다. 그러므로 이동해주어야 합니다.
- glViewport() : 카메라가 보는 위치를 바꿔버림.

```
glClear(GL_COLOR_BUFFER_BIT);
```

: 현재 화면에 존재하는 모든 도형 삭제

3. GLUT을 이용한 간단한 3D 도형

- 세부적으로 도형을 커스텀 할 수 없는 단점이 존재합니다.

1) Sphere (구)

Definition

```
void glutWireSphere (double dRadius, GLint slices, GLint stacks);  
void glutSolidSphere(double dRadius, GLint slices, GLint stacks);
```

Arguments

`dRadius` The desired radius of the sphere

`slices` The desired number of slices (divisions in the longitudinal direction) in the sphere

`stacks` The desired number of stacks (divisions in the latitudinal direction) in the sphere. The number of points in this direction, including the north and south poles, is `stacks+1`

변수명	설명
radius	구의 반지름을 설정합니다.
slices	구의 중심을 따라 세로로 세분화할 면의 수를 지정합니다. (경선)
stacks	구의 중심에 평행하게 가로로 세분화할 면의 수를 지정합니다. (위선)

2) Cylinder (원기둥)

Definition

```
void glutWireCylinder (double base, double height, GLint slices, GLint stacks);  
void glutSolidCylinder(double base, double height, GLint slices, GLint stacks);
```

Arguments

radius The desired radius of the cylinder

height The desired height of the cylinder

slices The desired number of slices around the cylinder

stacks The desired number of segments between the base and the top of the cylinder (the number of points, including the tip, is `stacks + 1`)

3) 직육면체 + 정육면체

15.5 glutWireCube, glutSolidCube

The `glutWireCube` and `glutSolidCube` functions draw a wireframe and solid cube respectively.

Definition

```
void glutWireCube (double dSize);  
void glutSolidCube(double dSize);
```

Arguments

dSize The desired length of an edge of the cube

- `glutscalef(3)`를 이용해서 정육면체를 직육면체로 만들 수 있습니다.

4. 추가 내용

- 세세하게 도형을 커스텀하려면 GL OR GLU를 사용해서 도형을 그려야 합니다.
- 두 모델이 겹치면 어떤 부분을 나타낼지 컴퓨터가 제대로 처리하지 못하는 경우 더 위에 있음에도 불구하고 해당 부분을 뚫고 가려져야 하는 부분의 모델이 보이게 됩니다. 그러므로 최대한 겹치지 않게 합니다.
- glTranslate , glRotate , glScale 같은 함수를 사용하면 도형을 움직이는 것이 아니라 좌표축을 움직이는 것입니다. 다시 말해 , 내가 그린 그림을 움직이는 것이 아니라 붓을 움직이는 원리입니다.